

AI Agents: Patterns, Principles & Practices

An interactive, live-updating book on Agentic Design Patterns.

Author: George Kour

Source Book

Title: Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems **Author:** Antonio Gulli
Publisher: Springer **ISBN:** 978-3032014018 **URL:** <https://www.amazon.com/Agentic-Design-Patterns-Hands-Intelligent/dp/3032014018/>

Generated: 2025-11-26T15:44:36.769Z

TABLE OF CONTENTS

Introduction & Foundations

Getting started with agentic AI systems

1. About Attribution and source information.
2. Preface High-level introduction to the agentic era, the book's purpose, structure, and how to use it effectively.
3. Introduction - Foundations of Agentic Systems Understanding the shift from Generative AI to Agentic AI, and the core differences between Workflows and Agents.
4. What is an Agentic Design Pattern? Understanding what design patterns are, how they differ from implementations and frameworks, and how to use patterns effectively in agentic systems.

Core Workflow Patterns

Fundamental patterns for building agent workflows

5. Pattern: Prompt Chaining (Pipeline Pattern) Breaking down complex tasks into sequential, manageable workflows. The foundational pattern for building reliable multi-step LLM systems.
6. Pattern: Routing Dynamically selecting between multiple potential actions based on input, state, or conditions, introducing conditional logic into agent workflows.
7. Pattern: Parallelization Executing multiple independent operations simultaneously to improve efficiency and reduce latency in agent workflows.

8. Pattern: Reflection Enabling agents to review, critique, and refine their own outputs through iterative self-evaluation and improvement cycles.

Tool Use & Execution

Designing the Agent-Computer Interface

9. Pattern: Tool Use & Execution Designing the Agent-Computer Interface (ACI). Best practices for tool definitions and result management.
10. Pattern: Constrained Tool Use (Mask, Don't Remove) Managing tool availability through programmatic constraints (logit masking) rather than dynamically modifying tool definitions, preserving KV-Cache efficiency and preventing model confusion.

Reasoning & Planning

Enabling agents to plan and reason effectively

11. Reasoning Techniques Core algorithms for agency: Chain-of-Thought, ReAct Loops, and Tree-of-Thought strategies.
12. Pattern: Planning Enabling agents to create structured plans before execution, breaking down complex goals into actionable steps.
13. Pattern: Prioritization Enabling agents to assess and rank tasks, objectives, or actions based on significance, urgency, dependencies, and criteria.

Memory & Context Management

Managing context windows and externalizing memory

14. Memory Management How to manage the context window, externalize memory, and use recitation patterns to keep agents on track.
15. Pattern: Persistent Task List (Recitation) A context engineering strategy where agents maintain a running plan and continuously append it to context to maintain goal alignment in long-horizon tasks.
16. Pattern: Leverage External Memory (Filesystem as Context) Treating external persistent storage as an unlimited extension of the agent's working memory, enabling restorable compression and just-in-time retrieval of large data.

17. Context Compression: Managing the Finite Window Comprehensive techniques for fitting necessary information into the LLM's finite context window through externalization, summarization, pruning, and attention manipulation.

Multi-Agent Systems

Scaling up with multiple agents working together

18. Multi-Agent Architectures Scaling up: Orchestrator-Workers, Evaluator-Optimizers, and Swarm patterns.
19. Pattern: Orchestrator-Worker (Coordinator) A central orchestrator dynamically breaks down complex goals into subtasks, delegates to specialized workers, and synthesizes results. The most common pattern for complex multi-agent tasks.

Advanced Capabilities

Learning, protocols, goal management, and human interaction

20. Learning and Adaptation Enabling agents to improve their performance over time through experience, feedback, and adaptive mechanisms.
21. Model Context Protocol (MCP) A standardized protocol for agents to discover, access, and interact with external tools and data sources.
22. Goal Setting and Monitoring Establishing clear objectives and tracking progress toward goals, enabling agents to measure success and adapt strategies.
23. Pattern: Exception Handling and Recovery Robust error handling and recovery mechanisms that enable agents to gracefully handle failures and continue operation.
24. Pattern: Human-in-the-Loop Integrating human oversight, feedback, and decision-making into agent workflows for safety, quality, and trust.

Knowledge & Communication

Retrieving knowledge and enabling agent communication

25. Pattern: Knowledge Retrieval (RAG) Enabling LLMs to access external knowledge bases through Retrieval-Augmented Generation, vector databases, and semantic search.

26. Pattern: Inter-Agent Communication (A2A) Enabling agents to communicate, coordinate, and collaborate with each other through standardized protocols and interfaces.

Optimization & Safety

Optimizing performance and ensuring safe operation

27. Resource-Aware Optimization Optimizing agent behavior considering computational, temporal, and financial resource constraints.
28. Guardrails/Safety Patterns Implementing safety mechanisms, content filters, and compliance checks to ensure agents operate within defined boundaries.
29. Evaluation and Monitoring Systematic assessment of agent performance, monitoring progress, and detecting operational anomalies in production environments.
30. Exploration and Discovery Enabling agents to actively seek out novel information, uncover new possibilities, and identify unknown unknowns.
-

PART I

Introduction & Foundations

Getting started with agentic AI systems

About

Attribution and source information.

Module ID: module-0

ABOUT THIS BOOK

This interactive book is a comprehensive guide to building intelligent, goal-oriented AI systems. As we transition from the era of Generative AI—where models simply respond to prompts—to the era of Agentic AI—where systems actively pursue objectives and interact with their environment—developers need practical patterns and principles to construct reliable, scalable agentic systems. This book provides exactly that: a hands-on collection of 21 design patterns that cover everything from foundational workflow patterns like prompt chaining and routing, to advanced capabilities such as multi-agent collaboration, memory management, and safety mechanisms.

Each pattern is presented with clear explanations, practical guidelines for when to use it, and real-world examples that demonstrate how to implement these concepts in production systems. Whether you're building simple single-agent workflows or complex multi-agent architectures, this book serves as both a reference guide and a practical handbook for navigating the rapidly evolving landscape of agentic AI development.

Book Structure

This interactive book contains **23 modules**:

1. **About** (this module) - Attribution and book information
2. **Preface** - High-level introduction to the agentic era
3. **Introduction** - Foundations and core concepts of agentic systems
4. **21 Pattern Modules** - Each dedicated to a specific design pattern

The patterns cover the full spectrum of agentic system design, from foundational concepts like prompt chaining and routing to advanced topics like multi-agent collaboration and exploration strategies.

How Content Was Adapted

The adaptation process involved:

- **Reorganization:** Content from the original 21 chapters was restructured into a consistent module format
- **Standardization:** Each pattern module follows a standardized structure for better accessibility
- **Enhancement:** Added structured sections like “When to Use This Pattern” with clear decision guidelines
- **Preservation:** Maintained the original code examples, concepts, and technical accuracy
- **Integration:** Combined related content (e.g., memory and context engineering) into cohesive modules

All technical content, code examples, and pattern descriptions remain based on the original work, ensuring that readers receive the same authoritative information in a more accessible format.

AI-Assisted Content Generation

In a fitting demonstration of the book’s subject matter, much of this adaptation was created with the assistance of AI writing agents. This approach reflects the very principles and patterns discussed throughout the book—using intelligent agents to structure, organize, and present complex information.

The content has been carefully reviewed and validated by the authors to ensure accuracy and quality. However, given the collaborative nature of AI-assisted content creation, there may occasionally be errors, inconsistencies, or areas that could benefit from improvement.

If you encounter any issues, have suggestions for improvement, or notice any errors, we would greatly appreciate your feedback. Please contact the author at kourgeorge@gmail.com. Your input helps us maintain and improve the quality of this resource for the entire community.

Bibliography

Much of the content in this book is based on “**Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems**” by **Antonio Gulli**, published by Springer.

Source Reference:

- **Book:** Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems
- **Author:** Antonio Gulli
- **Publisher:** Springer
- **ISBN:** 978-3032014018
- **Available at:** <https://www.amazon.com/Agentic-Design-Patterns-Hands-Intelligent/dp/3032014018/>

Journal Articles

Liu, Yue, et al. "Agent Design Pattern Catalogue: A Collection of Architectural Patterns for Foundation Model Based Agents." *Journal of Systems and Software*, vol. 220, 2025, p. 112278. Available at: <https://www.sciencedirect.com/science/article/pii/S0164121224003224>

Online Articles and Blog Posts

Ji, Yichao 'Peak'. "Context Engineering for AI Agents: Lessons from Building Manus." *Manus Blog*, July 18, 2025. Available at: <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>

Huang, Nick. "How Agents Can Use Filesystems for Context Engineering." *LangChain Blog*, November 21, 2025. Available at: <https://blog.langchain.com/how-agents-can-use-filesystems-for-context-engineering/>

Anthropic. "How We Built Our Multi-Agent Research System." *Anthropic Engineering Blog*, June 13, 2025. Available at: <https://www.anthropic.com/engineering/multi-agent-research-system>

Anthropic. "Building Effective AI Agents." *Anthropic Engineering Blog*, December 19, 2024. Available at: <https://www.anthropic.com/engineering/building-effective-agents>

Google Cloud. "Choose a Design Pattern for Your Agentic AI System." *Google Cloud Architecture Center*, 2025. Available at: <https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system>

Preface

High-level introduction to the agentic era, the book's purpose, structure, and how to use it effectively.

Module ID: module-1

PREFACE

Welcome to the Agentic Era

Welcome to “Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems.” We are living through a remarkable moment in the history of artificial intelligence. The last few years have witnessed an unprecedented transformation—from simple, reactive programs to sophisticated, autonomous entities capable of understanding context, making decisions, and interacting dynamically with their environment and other systems. These are the intelligent agents and the agentic systems they comprise.

The advent of powerful large language models (LLMs) has provided unprecedented capabilities for understanding and generating human-like content, serving as the cognitive engine for many of these agents. However, orchestrating these capabilities into systems that can reliably achieve complex goals requires more than just a powerful model. It requires structure, design, and a thoughtful approach to how the agent perceives, plans, acts, and interacts.

We are at an inflection point. If the last eighteen months were about the engine—the breathtaking ascent of Large Language Models—the next era is about the frameworks we build around them. It’s about transforming these generators of plausible text into true agents of action. This transformation is happening at an extraordinary pace: AI agent startups raised over \$2 billion by the end of 2024, with the market valued at \$5.2 billion and projected to reach nearly \$200 billion by 2034. According to recent studies, a majority of large IT companies are actively using agents, with a fifth of them starting within just the past year.

The Canvas of Agentic Systems

Think of building intelligent systems as creating a complex work of art or engineering on a canvas. This canvas isn't a blank visual space, but rather the underlying infrastructure and frameworks that provide the environment and tools for your agents to exist and operate. It's the foundation upon which you'll build your intelligent application, managing state, communication, tool access, and the flow of logic.

Building effectively on this agentic canvas demands more than just throwing components together. It requires understanding proven techniques – patterns – that address common challenges in designing and implementing agent behavior. Just as architectural patterns guide the construction of a building, or design patterns structure software, agentic design patterns provide reusable solutions for the recurring problems you'll face when bringing intelligent agents to life on your chosen canvas.

What Are Agentic Systems?

At its core, an agentic system is a computational entity designed to perceive its environment (both digital and potentially physical), make informed decisions based on those perceptions and a set of predefined or learned goals, and execute actions to achieve those goals autonomously. Unlike traditional software, which follows rigid, step-by-step instructions, agents exhibit a degree of flexibility and initiative.

Imagine you need a system to manage customer inquiries. A traditional system might follow a fixed script. An agentic system, however, could perceive the nuances of a customer's query, access knowledge bases, interact with other internal systems (like order management), potentially ask clarifying questions, and proactively resolve the issue, perhaps even anticipating future needs. These agents operate on the canvas of your application's infrastructure, utilizing the services and data available to them.

Agentic systems are characterized by autonomy, allowing them to act without constant human oversight; proactiveness, initiating actions towards their goals; and reactivity, responding effectively to changes in their environment. They are fundamentally goal-oriented, constantly working towards objectives. A critical capability is tool use, enabling them to interact with external APIs, databases, or services. They possess memory, retain information across interactions, and can engage in communication with users, other systems, or even other agents.

Effectively realizing these characteristics introduces significant complexity. How does the agent maintain state across multiple steps? How does it decide when and how to use a tool? How is communication between different agents managed? How do you build resilience into the system to handle unexpected outcomes or errors?

Why Patterns Matter in Agent Development

This complexity is precisely why agentic design patterns are indispensable. They are not rigid rules, but rather battle-tested templates or blueprints that offer proven approaches to standard design and implementation challenges in the agentic domain. By recognizing and applying these design patterns, you gain access to solutions that enhance the structure, maintainability, reliability, and efficiency of the agents you build.

Using design patterns helps you avoid reinventing fundamental solutions for tasks like managing conversational flow, integrating external capabilities, or coordinating multiple agent actions. They provide a common language and structure that makes your agent's logic clearer and easier for others (and yourself in the future) to understand and maintain. Implementing patterns designed for error handling or state management directly contributes to building more robust and reliable systems. Leveraging these established approaches accelerates your development process, allowing you to focus on the unique aspects of your application rather than the foundational mechanics of agent behavior.

A great question we often hear is: "With AI changing so fast, why write a book that could be quickly outdated?" Our motivation is actually the opposite. It's precisely because things are moving so quickly that we need to step back and identify the underlying principles that are solidifying. Patterns like RAG, Reflection, Routing, Memory, and the others we discuss are becoming fundamental building blocks. This book is an invitation to reflect on these core ideas, which provide the foundation we need to build upon. Humans need these reflection moments on foundation patterns.

About This Book

This book explores **21 essential agentic design patterns** that represent fundamental building blocks for constructing sophisticated agents. Each pattern is a battle-tested template offering proven approaches to standard design and implementation challenges in the agentic domain.

Book Structure

The book is organized into **23 modules**:

1. **About** - Attribution, source information, and details about this adaptation
2. **Preface** (this module) - High-level introduction to the era and the book
3. **Introduction** - Foundations and core concepts of agentic systems

4. 21 Pattern Modules - Each dedicated to a specific design pattern

The patterns are organized to build concepts progressively, but you can also use this book as a reference, jumping to patterns that address specific challenges you face in your agent development projects.

Pattern Organization

The 21 patterns are grouped conceptually:

Foundation Patterns (Modules 3-8):

- Prompt Chaining - Sequential workflows
- Routing - Dynamic decision-making
- Parallelization - Concurrent execution
- Reflection - Self-evaluation and improvement
- Tool Use - External interaction
- Planning - Strategic goal decomposition

Advanced Patterns (Modules 9-16):

- Multi-Agent - Collaborative systems
- Memory Management - Context and state
- Learning and Adaptation - Continuous improvement
- Model Context Protocol - Standardized tool access
- Goal Setting and Monitoring - Progress tracking
- Exception Handling - Error recovery
- Human-in-the-Loop - Human oversight
- Knowledge Retrieval (RAG) - External knowledge access

Enterprise Patterns (Modules 17-23):

- Inter-Agent Communication - Agent coordination
- Resource-Aware Optimization - Efficiency and cost management
- Reasoning Techniques - Advanced thinking strategies
- Guardrails/Safety - Safety and compliance
- Evaluation and Monitoring - Performance assessment

- Prioritization - Task management
- Exploration and Discovery - Novel solution finding

Each Pattern Module Includes

- **Pattern Overview** - What it is, when to use it, and why it matters
- **When to Use This Pattern** - Decision criteria with  /  guidelines
- **Practical Applications & Use Cases** - Real-world scenarios
- **Implementation** - Hands-on code examples demonstrating the pattern
- **Key Takeaways** - Summary of crucial points
- **Related Patterns** - Connections to other patterns
- **References** - Resources for further exploration

Frameworks Used

Throughout this book, we demonstrate patterns using prominent frameworks:

- **LangChain & LangGraph** - Flexible chaining and stateful graph-based agent construction
- **Google ADK (Agent Development Kit)** - Tools and components for building, evaluating, and deploying agents
- **Crew AI** - Structured framework for orchestrating multiple AI agents

These frameworks represent different approaches to the agent development “canvas,” each with its strengths. By showing examples across these tools, you’ll gain a broader understanding of how patterns can be applied regardless of your chosen technical environment.

How to Use This Book

This book is crafted to be a practical and accessible resource. Its primary focus is on clearly explaining each agentic pattern and providing concrete, runnable code examples to demonstrate its implementation.

For Beginners

Start with the About, Preface, and Introduction modules to understand the foundations and context. Then work through the foundation patterns (Modules 3-8) in order. These build upon each other and establish core concepts. Practice with the code examples to build understanding. Don't rush—take time to experiment with each pattern before moving to the next.

For Experienced Developers

Use this as a reference guide. Jump to patterns that address your specific challenges. Each module is self-contained, though cross-references help you understand relationships. Read the Pattern Overview and Key Takeaways for quick understanding, then dive into Implementation sections when you need to build something.

For Teams

Use patterns as a common language for discussing agent architecture. Reference specific patterns when designing systems to ensure consistent approaches across your organization. This shared vocabulary will improve communication and reduce misunderstandings about system design.

For Learning

Read the Pattern Overview and Key Takeaways for quick understanding. Dive into Implementation sections when you need to build something. Review Related Patterns to understand how patterns work together. We strongly encourage you to run the code examples, experiment with them, and adapt them to build your own intelligent systems.

Reading Strategies

- **Sequential Reading:** Follow the module order to build concepts progressively
- **Reference Reading:** Jump to specific patterns when you encounter challenges
- **Deep Dive:** Focus on Implementation sections when you need to build
- **Quick Scan:** Read Pattern Overview and Key Takeaways for rapid understanding

The Emphasis on Practical Application

Throughout this book, the emphasis is on practical application. Every pattern includes runnable code examples that you can execute, modify, and learn from. We encourage you to:

- **Run the examples** - Don't just read them; execute them and see how they work
- **Experiment** - Modify the code, try different inputs, break things and fix them
- **Adapt** - Use the patterns as starting points for your own applications
- **Build** - Apply patterns to real problems you're trying to solve

The code examples are designed to clearly illustrate each pattern's core logic and its implementation, focusing on clarity and practicality over production-ready complexity.

What You'll Gain

By the end of this book, you will:

- Understand the fundamental concepts behind 21 essential agentic patterns
- Possess practical knowledge and code examples to apply them effectively
- Have a common language for discussing agent architecture with your team
- Be able to build more intelligent, capable, and autonomous systems
- Understand when to use each pattern and when to avoid it
- Know how patterns work together to create sophisticated agentic systems

A Note on the Rapidly Evolving Landscape

The field of agentic AI is evolving at an extraordinary pace. New frameworks emerge, models improve, and techniques advance. However, the patterns in this book represent stable, foundational principles that transcend specific implementations. They are the architectural decisions and design approaches that will remain relevant even as specific technologies change.

Think of these patterns as the grammar of agentic systems—the fundamental structures that enable effective communication and problem-solving, regardless of the specific “words” (frameworks, models, tools) you choose to use.

Let's Begin

This book is your guide to building intelligent, agentic systems. Whether you're just starting your journey into agentic AI or looking to deepen your understanding of proven patterns, we hope this resource empowers you to create systems that are robust, reliable, and effective.

The journey ahead is exciting. You're about to learn patterns that will enable you to build systems that can reason, plan, act, and collaborate. These are the building blocks of the next generation of AI applications.

Let's begin this hands-on journey into building intelligent, agentic systems!

Introduction - Foundations of Agentic Systems

Understanding the shift from Generative AI to Agentic AI, and the core differences between Workflows and Agents.

Module ID: module-2

INTRODUCTION: FOUNDATIONS OF AGENTIC SYSTEMS

The Paradigm Shift

*We are standing on the precipice of a transformation in artificial intelligence that is as significant as the move from command-line interfaces to graphical user interfaces. We are moving from models that simply generate content to systems that **achieve goals**. This is the fundamental shift from passive tools to active participants in problem-solving.*

For the past few years, the spotlight has been on **Generative AI**. These models are impressive; they can write poetry, debug code, and summarize history. However, they remain fundamentally reactive. They wait for input, process it, and return output. They are stateless oracles.

Agentic AI changes the equation. It moves beyond text generation into the realm of action. An agent doesn't just say what *should* be done; it attempts to *do* it.

Generative AI

- Flow: Prompt → LLM → Response

- **Nature:** Passive, one-shot, content-focused.
- **Role:** The human drives the process. The human must verify the output and perform the subsequent action. The model is a fancy autocomplete engine.

Agentic AI

- **Flow:** Goal → Agent (Think → Act → Observe Loop) → Environment → Goal Achieved
- **Nature:** Active, iterative, goal-focused.
- **Role:** The agent drives the process. It acts as a collaborator or an employee. It doesn't just answer a question; it navigates a problem space, corrects its own errors, and interacts with external software.

"Agentic systems represent a new paradigm that breaks traditional barriers. More than tools, agents act as collaborators, assisting humans in dynamic environments and automating decision-making." - Aatrbn



Agentic Systems Overview

The Evolution of AI Systems

The journey from simple language models to sophisticated agentic systems has been rapid and transformative. In just a few years, we've witnessed a dramatic evolution:

Phase 1: Basic LLMs - Simple prompt-response interactions with no external capabilities.

Phase 2: RAG (Retrieval-Augmented Generation) - Enhanced reliability by grounding models on factual information from knowledge bases.

Phase 3: Individual AI Agents - Agents capable of using various tools, planning, and executing multi-step tasks.

Phase 4: Agentic AI - Teams of specialized agents working in concert to achieve complex goals, marking a significant leap in AI's collaborative power.

This evolution reflects a fundamental shift from static automation to dynamic, intelligent systems that can adapt, learn, and collaborate. The market reflects this transformation: AI agent startups raised over \$2 billion by the end of 2024, with the market valued at \$5.2 billion and projected to reach nearly \$200 billion by 2034. According to recent studies, a majority of large IT companies are actively using agents, with a fifth of them starting within just the past year.

Defining the Landscape: Workflows vs. Agents

Before building, we must understand the architectural distinction. In the rush to adopt “agents,” many engineers misclassify their systems. The difference lies in who controls the flow of execution: the code or the model. One follows a script; the other writes its own.

Workflows (Pipelines)

Workflows are systems where LLMs and tools are orchestrated through predefined code paths. Imagine a rigid flow chart. The LLM is used as a processing unit (a “cognitive engine”) at specific nodes, but the edges between nodes are hardcoded.

- **Structure:** `Input → Step A → Step B → Step C → Output`
- **Characteristics:** Predictable, consistent, ideal for well-defined tasks. If step A fails, the system likely throws a standard exception.
- **Control:** The path is hardcoded by the engineer.
- **Example:** A system that takes a PDF, summarizes it using an LLM, translates the summary, and emails it. The LLM never decides *what* to do next; it only does what the script commands.

Agents

Agents are systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks. The engineer defines the *tools* and the *goal*, but the agent defines the *path*.

- **Structure:** `Start → Decision → Task A OR Task B → Observation → Next Decision → Goal`
- **Characteristics:** Flexible, adaptive, ideal for open-ended problems where the solution path is unknown.

- **Control:** The LLM decides the path at runtime.
- **Example:** A system asked to “Find out why the server crashed.” The agent might choose to check logs, or it might choose to check recent code commits. It decides its next step based on the result of the previous step.

***Note:** Simply augmenting an LLM with modules, tools, or predefined steps does not make it an agent; in any case, that would make it a workflow. If the “if/else” logic is in your Python code, it’s a workflow. If the “if/else” logic is generated by the LLM, it’s an agent.*

What Makes an AI System an Agent?

In simple terms, an AI agent is a system designed to perceive its environment and take actions to achieve a specific goal. It’s an evolution from a standard Large Language Model (LLM), enhanced with the abilities to plan, use tools, and interact with its surroundings. Think of an Agentic AI as a smart assistant that learns on the job. It follows a simple, five-step loop to get things done:

1. **Get the Mission:** You give it a goal, like “organize my schedule.”
2. **Scan the Scene:** It gathers all the necessary information—reading emails, checking calendars, and accessing contacts—to understand what’s happening.
3. **Think It Through:** It devises a plan of action by considering the optimal approach to achieve the goal.
4. **Take Action:** It executes the plan by sending invitations, scheduling meetings, and updating your calendar.
5. **Learn and Get Better:** It observes successful outcomes and adapts accordingly. For example, if a meeting is rescheduled, the system learns from this event to enhance its future performance.

Core Characteristics of Agents

Agents are characterized by several key capabilities:

- **Autonomy:** Agents can operate without constant human oversight, making decisions and taking actions independently.
- **Proactiveness:** Agents initiate actions toward their goals rather than merely reacting to inputs.
- **Reactivity:** Agents respond effectively to changes in their environment.

- **Goal-Oriented:** Agents are fundamentally focused on achieving specific objectives.
 - **Tool Use:** Agents can interact with external APIs, databases, or services, extending beyond their immediate context.
 - **Memory:** Agents retain information across interactions, maintaining context and learning from experience.
 - **Communication:** Agents can engage with users, other systems, or other agents.
-

The Anatomy of an Agent

We can understand agentic patterns by mapping them to the core components of an intelligent system. This book is structured around building an agent, piece by piece, mirroring biological cognition.

1. Memory & Context (The Mind)

An agent is useless if it cannot remember previous actions or retrieve relevant knowledge.

- **Context Window:** The immediate “short-term” working memory.
- **History Management:** Mechanisms to summarize or truncate long conversations to fit within the window.
- **Retrieval (RAG):** Long-term memory. This allows the agent to query vector databases to access documentation or past experiences, grounding its decisions in data rather than hallucinations.

2. Reasoning & Planning (The Brain)

How the agent thinks, plans, and makes decisions. This is the core loop.

- **Chain-of-Thought (CoT):** Encouraging the model to “show its work” before answering.
- **ReAct (Reason + Act):** A paradigm where the model generates a thought, performs an action, observes the output, and then reasons again.
- **Tree-of-Thoughts:** Exploring multiple possible future paths before committing to one, allowing for strategic foresight.

3. Tool Use & Execution (The Hands)

How the agent interacts with the world. Without tools, an agent is a brain in a jar.

- **Function Calling:** The mechanism by which an LLM generates structured JSON to execute code.
- **Capabilities:** This involves API calls (e.g., Stripe, Slack), file system manipulation (reading/writing code), and web browsing to fetch real-time data.

4. Multi-Agent Collaboration (Society of Minds)

For complex tasks, a single agent often fails due to context overload or lack of specialization.

- **Orchestration:** A “manager” agent delegating tasks to “specialist” agents (e.g., a Coder, a Researcher, and a Reviewer).
- **Swarm Architectures:** Autonomous agents interacting to solve problems through consensus or division of labor.

Levels of Agent Complexity

Agents can be categorized into different levels of sophistication, each building upon the previous:

Level 0: The Core Reasoning Engine

While an LLM is not an agent in itself, it can serve as the reasoning core of a basic agentic system. In a ‘Level 0’ configuration, the LLM operates without tools, memory, or environment interaction, responding solely based on its pretrained knowledge. Its strength lies in leveraging its extensive training data to explain established concepts. The trade-off for this powerful internal reasoning is a complete lack of current-event awareness.

Level 1: The Connected Problem-Solver

At this level, the LLM becomes a functional agent by connecting to and utilizing external tools. Its problem-solving is no longer limited to its pre-trained knowledge. Instead, it can execute a sequence of actions to gather and process information from sources like the internet (via search) or databases (via Retrieval Augmented Generation, or RAG). This ability to interact with the outside world across multiple steps is the core capability of a Level 1 agent.

Level 2: The Strategic Problem-Solver

At this level, an agent's capabilities expand significantly, encompassing strategic planning, proactive assistance, and self-improvement. The agent moves beyond single-tool use to tackle complex, multi-part problems through strategic problem-solving. It performs context engineering: the strategic process of selecting, packaging, and managing the most relevant information for each step. This level leads to proactive and continuous operation, and the agent achieves self-improvement by refining its own context engineering processes.

Level 3: The Rise of Collaborative Multi-Agent Systems

At Level 3, we see a significant paradigm shift, moving away from the pursuit of a single, all-powerful super-agent and towards sophisticated, collaborative multi-agent systems. This approach recognizes that complex challenges are often best solved not by a single generalist, but by a team of specialists working in concert. The collective strength of such a system lies in the division of labor and the synergy created through coordinated effort.

Common Challenges in Agent Development

Building effective agentic systems introduces unique challenges that traditional software development doesn't face:

1. Reliability and Consistency

Agents operate probabilistically, making them inherently less predictable than deterministic code. The same input can produce different outputs, making testing and validation more complex.

2. Context Management

Agents must manage limited context windows while maintaining relevant information across long interactions. Balancing detail with efficiency is a constant challenge.

3. Error Handling and Recovery

When agents make mistakes or encounter unexpected situations, they need robust mechanisms to detect, understand, and recover from errors without human intervention.

4. Cost and Latency

Each LLM call consumes tokens and time. Complex agents making many sequential calls can become expensive and slow, requiring careful optimization.

5. Hallucination and Factual Accuracy

Agents can generate plausible but incorrect information. Ensuring accuracy requires careful design, grounding mechanisms, and validation.

6. Tool Integration Complexity

Agents must interact with diverse external systems, each with different APIs, error formats, and behaviors. Creating a consistent interface is challenging.

7. Coordination in Multi-Agent Systems

When multiple agents work together, managing communication, state sharing, and task coordination becomes complex.

8. Safety and Guardrails

Agents operating autonomously need boundaries to prevent harmful actions, ensure compliance, and maintain ethical standards.

These challenges are precisely why design patterns matter. They provide proven solutions to these recurring problems.

Guiding Principles of Agentic Engineering

Success in the LLM space isn't about building the most sophisticated system. It's about building the *right* system for your needs. Agentic systems are prone to loops, hallucinations, and high costs. Adhering to these principles mitigates those risks.

1. Start Simple

Find the simplest solution possible, and only increase complexity when needed. Do not build an autonomous agent if a linear workflow suffices.

- **Tradeoff:** Agentic systems trade latency (time) and cost (tokens) for better task performance and flexibility. They are slower and more expensive than workflows.
- **Advice:** Start with a prompt. If that fails, try a chain (workflow). Only if the path to the solution is highly variable should you build an agent.

2. Prioritize Transparency

Explicitly show the agent’s planning steps (its “inner monologue”).

- **Why:** This helps with debugging and builds user trust. If an agent fails, you need to know if it failed because it *reasoned* poorly or because a *tool* returned an error.
- **User Trust:** If users can see *why* an agent made a decision, they are more likely to accept the outcome, even if it takes longer to generate.

3. Craft the Agent-Computer Interface (ACI)

We have spent decades perfecting the Human-Computer Interface (HCI). We must now invest as much effort in creating a good Agent-Computer Interface (ACI).

- **The Concept:** Tools and APIs are the UI for the model. If an API is messy or poorly documented, the human developer might figure it out, but the Agent will hallucinate or fail.
- **Implementation:** This involves writing thorough tool documentation (which the LLM reads), clear docstrings, and robust error handling that returns meaningful error messages to the agent so it can self-correct.

4. Design for Failure

Assume things will go wrong and build resilience into your system from the start.

- **Error Recovery:** Implement retry mechanisms, fallback strategies, and graceful degradation.
- **Validation:** Check outputs before using them, validate tool results, and verify goal achievement.
- **Monitoring:** Track agent behavior, detect anomalies, and measure performance continuously.

5. Balance Autonomy with Control

Give agents enough autonomy to be effective, but maintain appropriate oversight and control mechanisms.

- **Human-in-the-Loop:** Know when to involve humans for critical decisions or validation.
 - **Guardrails:** Set clear boundaries and constraints to prevent harmful actions.
 - **Observability:** Make agent decisions and actions visible and auditable.
-

Why Design Patterns Matter

Design patterns are battle-tested templates offering proven approaches to standard design and implementation challenges. In agentic systems, they address fundamental questions:

- How do you structure sequential operations?
- How do you manage state and context?
- How do you handle errors and unexpected situations?
- How do you coordinate multiple agents?
- How do you ensure quality and safety?

By recognizing and applying these patterns, you gain access to solutions that enhance structure, maintainability, reliability, and efficiency. Patterns provide a common language that makes your agent's logic clearer and easier to understand, maintain, and extend.

Using design patterns helps you avoid reinventing fundamental solutions and accelerates development, allowing you to focus on the unique aspects of your application rather than foundational mechanics.

The Future of Agentic Systems

As we look ahead, several trends are shaping the future of agentic AI:

- **Generalist Agents** - Evolution from narrow specialists to true generalists capable of managing complex, ambiguous, long-term goals

- **Deep Personalization** - Agents that become proactive partners, learning from patterns and anticipating needs
- **Embodiment** - Agents breaking free from digital confines to operate in the physical world through robotics
- **Agent-Driven Economy** - Highly autonomous agents becoming active participants in economic systems
- **Metamorphic Systems** - Goal-driven multi-agent systems that can modify their own architecture and improve autonomously

The patterns in this book provide the foundation for building these future systems. They represent the stable principles that will remain relevant even as specific technologies evolve.

Next Steps

Now that you understand the foundations of agentic systems, you're ready to explore the 21 design patterns. Each pattern module provides clear explanations, practical guidance, real-world applications, and hands-on code examples.

Remember the guiding principles: start simple, prioritize transparency, craft good interfaces, design for failure, and balance autonomy with control. These principles, combined with the patterns you'll learn, will enable you to build robust, reliable, and effective agentic systems.

Proceed to the pattern modules to begin building intelligent, agentic systems!

What is an Agentic Design Pattern?

Understanding what design patterns are, how they differ from implementations and frameworks, and how to use patterns effectively in agentic systems.

Module ID: module-2a

WHAT IS AN AGENTIC DESIGN PATTERN?

Before diving into specific patterns, it's essential to understand what we mean by an “agentic design pattern” and how it differs from concrete implementations, frameworks, or libraries.

Understanding Design Patterns

A **design pattern** is an abstract, reusable solution to a recurring problem in system design. It's not a specific piece of code or a library you can import. Instead, it's a **template** or **blueprint** that describes:

1. **The Problem:** A recurring challenge that appears across different agentic systems
2. **The Solution Structure:** An abstract approach to solving that problem
3. **The Trade-offs:** Benefits and limitations of applying this solution
4. **When to Use It:** Contexts where this pattern is appropriate
5. **When Not to Use It:** Situations where alternative approaches are better

Design patterns are **technology-agnostic**. The same pattern can be implemented using different frameworks (LangChain, LangGraph, Google ADK, CrewAI), different programming languages, or even different model providers. The pattern describes the *what* and *why*, while implementations show the *how*.

What Makes a Pattern “Agentic”?

An **agentic design pattern** is a design pattern specifically tailored to the unique challenges of building AI agent systems. These patterns address problems that arise when:

- **LLMs make autonomous decisions** about tool usage and action sequences
- **Systems operate in dynamic, unpredictable environments** where the solution path isn’t predetermined
- **Probabilistic models** require different reliability and error-handling approaches than deterministic code
- **Context windows are finite** and must be managed strategically
- **Multiple agents collaborate** and need coordination mechanisms
- **Human oversight** must be integrated into autonomous workflows

Agentic design patterns differ from traditional software design patterns (like Singleton, Factory, Observer) because they account for the unique characteristics of LLM-based systems: non-determinism, context limitations, tool integration, and the need for transparency in decision-making.

The Relationship Between Patterns and Implementations

It’s crucial to distinguish between:

- **The Pattern (Abstract):** The reusable solution template
 - Example: “The Reflection pattern enables agents to evaluate and refine their outputs through iterative feedback loops”
- **The Implementation (Concrete):** A specific realization of the pattern using particular technologies
 - Example: “Using LangGraph to implement a Producer-Critic reflection loop with Gemini 2.0”
- **The Framework (Tool):** A library or system that provides abstractions for implementing patterns
 - Example: “LangGraph provides state management and conditional edges that make it easier to implement the Reflection pattern”

In this book, each pattern module includes:

- **Pattern Overview:** The abstract description of the problem and solution
- **When to Use:** Guidance on recognizing the recurring problem
- **Implementation Examples:** Concrete code showing how the pattern can be realized
- **Framework-Specific Examples:** How different tools can be used to implement the same pattern

Characteristics of Good Design Patterns

Effective agentic design patterns share these characteristics:

1. **Abstraction:** They describe solutions at a conceptual level, not tied to specific technologies
2. **Reusability:** They can be applied across different domains, use cases, and technical stacks
3. **Proven:** They represent solutions that have been tested and refined through real-world application
4. **Composable:** They can be combined with other patterns to solve complex problems
5. **Documented Trade-offs:** They clearly explain benefits, limitations, and when alternatives are better

Patterns vs. Frameworks vs. Libraries

Understanding these distinctions helps you choose the right tool for the right job:

ASPECT	DESIGN PATTERN	FRAMEWORK	LIBRARY
Nature	Abstract solution template	Concrete implementation tool	Reusable code components
Level	Conceptual/Architectural	Application structure	Code utilities
Portability	Technology-agnostic	Framework-specific	Language/library-specific
Example	“Orchestrator-Worker pattern”	“LangGraph framework”	“LangChain tools library”
Purpose	Solve recurring problems	Provide structure for apps	Provide reusable functions

Patterns tell you *what* to build and *why*. **Frameworks** help you *how* to build it. **Libraries** give you the *pieces* to build with.

How to Use This Book's Patterns

When you encounter a pattern in this book:

1. **Understand the Problem:** Recognize the recurring challenge the pattern addresses
2. **Learn the Solution Structure:** Understand the abstract approach, not just the code
3. **Identify When It Applies:** Determine if your situation matches the problem context
4. **Adapt the Implementation:** Use the examples as starting points, but adapt them to your specific needs, framework, and constraints
5. **Combine Patterns:** Real systems often use multiple patterns together

Remember: **The pattern is the abstraction. The code examples are illustrations.** Your implementation will differ based on your specific requirements, but the core pattern structure remains the same.

Next Steps

Now that you understand what agentic design patterns are, you're ready to explore the specific patterns in this book. Each pattern module follows a consistent structure:

- **Pattern Overview:** What the pattern is and why it matters
- **When to Use:** Guidance on recognizing when this pattern applies
- **Practical Applications:** Real-world use cases
- **Implementation:** Code examples showing how to realize the pattern
- **Key Takeaways:** Summary of the pattern's core concepts

Proceed to the pattern modules to begin learning how to apply these abstract solutions to your specific agentic system challenges.

PART II

Core Workflow Patterns

Fundamental patterns for building agent workflows

Pattern: Prompt Chaining (Pipeline Pattern)

Breaking down complex tasks into sequential, manageable workflows. The foundational pattern for building reliable multi-step LLM systems.

Module ID: module-3

PROMPT CHAINING (PIPELINE PATTERN)

Motivation

When cooking a complex recipe, you don't try to do everything at once. You first gather ingredients, then prep them, then cook in stages, using the output of each step as input for the next. Similarly, when assembling furniture, you follow numbered steps sequentially, where each step builds on the previous one. Prompt chaining mirrors this natural human approach: breaking complex tasks into manageable, sequential steps where each stage produces results that inform the next.

Pattern Overview

What it is: Prompt Chaining, sometimes referred to as the Pipeline Pattern, is a technique for handling intricate tasks with LLMs by breaking down complex problems into a sequence of smaller, manageable sub-problems. Each sub-problem is addressed through a specifically designed prompt, and the output from one prompt is strategically fed as input into the subsequent prompt in the chain.

When to use: Use prompt chaining when a task is too complex for a single prompt, involves multiple distinct processing stages, requires interaction with external tools between steps, or when building workflows that need to perform multi-step reasoning with a predetermined sequence.

Why it matters: Complex tasks often overwhelm LLMs when handled within a single prompt, leading to instruction neglect, contextual drift, error propagation, and hallucinations. Prompt chaining addresses these challenges by breaking complex tasks into focused, sequential workflows, significantly improving reliability and control. This modular, divide-and-conquer strategy makes the process more manageable, easier to debug, and allows for the integration of external tools or structured data formats between steps.

This sequential processing technique inherently introduces modularity and clarity into the interaction with LLMs. By decomposing a complex task, it becomes easier to understand and debug each individual step, making the overall process more robust and interpretable. Each step in the chain can be meticulously crafted and optimized to focus on a specific aspect of the larger problem, leading to more accurate and focused outputs.

The output of one step acting as the input for the next is crucial. This passing of information establishes a dependency chain, where the context and results of previous operations guide the subsequent processing. This allows the LLM to build on its previous work, refine its understanding, and progressively move closer to the desired solution.

Key Distinction from Agents: In prompt chaining, the engineer hardcodes the sequence. The LLM never decides *what* to do next—it only processes the input according to the predefined prompt. If the “if/else” logic is in your Python code, it’s a workflow using prompt chaining. If the “if/else” logic is generated by the LLM, it’s an agent.

Key Concepts

- **Sequential Decomposition:** Breaking complex tasks into a sequence of smaller, focused sub-tasks that build upon each other.
- **Output-to-Input Chaining:** The output from one prompt step becomes the input for the next, creating a logical workflow progression.
- **Structured Output:** Using formats like JSON or XML between steps to ensure data integrity and machine-readability.
- **Modular Design:** Each step in the chain can be independently optimized, tested, and debugged.
- **Tool Integration:** External tools, APIs, or databases can be integrated at any step in the chain.

- **Workflow Pattern:** This is a workflow pattern, not an agentic pattern—the sequence is predetermined by the engineer.

How It Works

Prompt chaining operates through a sequential workflow where each step processes input and produces output that feeds into the next step. First, the complex task is decomposed into logical sub-tasks, each with a specific purpose (e.g., extraction, transformation, synthesis). Second, each sub-task is assigned a focused prompt that instructs the LLM to perform that specific operation. Third, the output from each step is captured, potentially validated or transformed, and passed as input to the next prompt. Fourth, structured output formats (JSON, XML) are used between steps to ensure data integrity and prevent parsing errors. Finally, the chain executes sequentially, with each step building upon the results of previous steps until the final output is produced.

When to Use This Pattern

✅ Use this pattern when:

- **Complex multi-step tasks:** Tasks that require multiple distinct processing stages that build upon each other.
- **Single prompt is insufficient:** The task is too complex or has too many constraints to handle reliably in a single prompt.
- **Tool integration needed:** You need to interact with external tools, APIs, or databases between processing steps.
- **Structured data transformation:** Converting unstructured data through multiple transformation stages (e.g., extract → normalize → format).
- **Sequential reasoning required:** Tasks that require multi-step reasoning with a predetermined sequence.
- **Debugging and reliability:** You need granular control and the ability to debug individual steps in a complex process.

❌ Avoid this pattern when:

- **Simple single-step tasks:** Tasks that can be reliably completed with a single, well-crafted prompt.

- **Dynamic decision-making needed:** When the sequence of steps should be determined at runtime by the LLM (use agents instead).
- **Parallel processing possible:** When sub-tasks are independent and can be executed in parallel (use parallelization pattern instead).
- **Low-latency requirements:** When the overhead of multiple sequential LLM calls is prohibitive.
- **Minimal complexity:** When the added complexity of chaining doesn't provide sufficient benefit over a single prompt.

Decision Guidelines

Choose prompt chaining when the benefits of modularity, reliability, and control outweigh the added complexity and latency of multiple sequential calls. Consider the task complexity: if a single prompt consistently fails or produces unreliable results, chaining is likely beneficial. Consider the processing stages: if the task naturally decomposes into distinct stages (extract → transform → synthesize), chaining is appropriate. Consider tool integration: if you need to use external tools between steps, chaining provides a natural structure. However, if the sequence should be dynamic or determined by the LLM, consider using an agentic pattern instead.

Practical Applications & Use Cases

Prompt chaining is a versatile pattern applicable in a wide range of scenarios when building LLM-powered workflows. Common applications include information processing, complex query answering, and content generation.

- **Information Processing Workflows:** Processing raw information through multiple transformations (extract text → summarize → extract entities → query database → generate report).
- **Complex Query Answering:** Answering questions that require multiple steps of reasoning or information retrieval (identify sub-questions → research each → synthesize answer).
- **Data Extraction and Transformation:** Converting unstructured text into structured formats through iterative refinement (extract fields → validate → refine missing fields → output structured data).
- **Content Generation Workflows:** Composing complex content through distinct phases (generate ideas → create outline → draft sections → review and refine).
- **Conversational Agents with State:** Maintaining conversational continuity by incorporating previous context into each turn (process utterance → update state → generate response).

- **Code Generation and Refinement:** Generating functional code through multiple stages (understand request → generate outline → write code → identify errors → refine → add documentation).
- **Multimodal and Multi-Step Reasoning:** Analyzing datasets with diverse modalities through sequential processing (extract text → link with labels → interpret with tables).

Implementation

Prerequisites

```
pip install langchain langchain-community langchain-openai langgraph
```

Note: `langchain-openai` can be substituted with the appropriate package for a different model provider (e.g., `langchain-google-genai` for Gemini).

Basic Example

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Initialize the Language Model
llm = ChatOpenAI(temperature=0)

# --- Prompt 1: Extract Information ---
prompt_extract = ChatPromptTemplate.from_template(
    "Extract the technical specifications from the following text:\n\n{text_input}"
)

# --- Prompt 2: Transform to JSON ---
prompt_transform = ChatPromptTemplate.from_template(
    "Transform the following specifications into a JSON object with 'cpu', 'memory', and"
)

# --- Build the Chain using LCEL ---
# The StrOutputParser() converts the LLM's message output to a simple string.
extraction_chain = prompt_extract | llm | StrOutputParser()

# The full chain passes the output of the extraction chain into the 'specifications'
# variable for the transformation prompt.
full_chain = (
    {"specifications": extraction_chain}
    | prompt_transform
    | llm
    | StrOutputParser()
)

# --- Run the Chain ---
input_text = "The new laptop model features a 3.5 GHz octa-core processor, 16GB of RAM,
```

```
# Execute the chain with the input text dictionary.  
final_result = full_chain.invoke({"text_input": input_text})  
  
print("\n--- Final JSON Output ---")  
print(final_result)
```

Explanation: This example demonstrates a two-step prompt chain that functions as a data processing pipeline. The initial stage extracts technical specifications from unstructured text, and the subsequent stage transforms the extracted output into a structured JSON format. The LangChain Expression Language (LCEL) elegantly chains these prompts together, with the output of the first chain automatically feeding into the second prompt.

Advanced Example: Multi-Step Workflow

```

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.pydantic_v1 import BaseModel, Field
from typing import List

llm = ChatOpenAI(temperature=0)

# Define structured output for trend extraction
class Trend(BaseModel):
    trend_name: str = Field(description="Name of the trend")
    supporting_data: str = Field(description="Data point supporting the trend")

class TrendsResponse(BaseModel):
    trends: List[Trend]

# Step 1: Summarize document
summarize_prompt = ChatPromptTemplate.from_template(
    "Summarize the key findings of the following market research report:\n\n{report_text}"
)

# Step 2: Extract trends with structured output
extract_trends_prompt = ChatPromptTemplate.from_template(
    "Using the following summary, identify the top three emerging trends and extract sp"
)

# Step 3: Generate email
email_prompt = ChatPromptTemplate.from_template(
    "Draft a concise email to the marketing team that outlines the following trends and"
)

# Build chains
summarize_chain = summarize_prompt | llm | StrOutputParser()

```



```

extract_chain = extract_trends_prompt | llm | JsonOutputParser(pydantic_object=TrendsRe
email_chain = email_prompt | llm | StrOutputParser()

# Full workflow
def process_report(report_text: str):
    # Step 1: Summarize
    summary = summarize_chain.invoke({"report_text": report_text})

    # Step 2: Extract trends
    trends_data = extract_chain.invoke({"summary": summary})
    trends_json = trends_data.json() if hasattr(trends_data, 'json') else str(trends_da

    # Step 3: Generate email
    email = email_chain.invoke({"trends_json": trends_json})

    return {
        "summary": summary,
        "trends": trends_data,
        "email": email
    }

# Example usage
report = "Market research shows 73% of consumers prefer personalized experiences..."
result = process_report(report)
print(result["email"])

```

Explanation: This advanced example demonstrates a three-step workflow for processing a market research report. The chain summarizes the report, extracts trends with structured output, and generates an email. Using Pydantic models for structured output ensures data integrity between steps, and each step can be independently tested and optimized.

Framework-Specific Examples

LANGGRAPH

```

from langgraph.graph import StateGraph, END
from typing import TypedDict

class ChainState(TypedDict):
    input_text: str
    extracted: str
    transformed: str
    final_output: str

def extract_step(state: ChainState) → ChainState:
    # Extract information
    result = llm.invoke(f"Extract specs from: {state['input_text']}")
    return {**state, "extracted": result.content}

def transform_step(state: ChainState) → ChainState:
    # Transform to JSON
    result = llm.invoke(f"Transform to JSON: {state['extracted']}")
    return {**state, "transformed": result.content}

def finalize_step(state: ChainState) → ChainState:
    # Final processing
    result = llm.invoke(f"Finalize: {state['transformed']}")
    return {**state, "final_output": result.content}

# Build graph
graph = StateGraph(ChainState)
graph.add_node("extract", extract_step)
graph.add_node("transform", transform_step)
graph.add_node("finalize", finalize_step)

graph.set_entry_point("extract")
graph.add_edge("extract", "transform")

```

```
graph.add_edge("transform", "finalize")
graph.add_edge("finalize", END)

# Execute
result = graph.invoke({"input_text": "Laptop specs: 3.5GHz CPU, 16GB RAM, 1TB SSD"})
```

GOOGLE ADK

```
from google.adk.agents import Agent
from google.adk.tools import FunctionTool

# Define tools for each step
extract_tool = FunctionTool(
    name="extract_specs",
    description="Extract technical specifications from text",
    func=lambda text: extract_specifications(text)
)

transform_tool = FunctionTool(
    name="transform_to_json",
    description="Transform specifications to JSON",
    func=lambda specs: transform_specs(specs)
)

# Create agent with chained workflow
agent = Agent(
    name="SpecProcessor",
    model="gemini-2.0-flash",
    instruction="""Process technical specifications in two steps:
1. Extract specifications from input text
2. Transform extracted specs to JSON format""",
    tools=[extract_tool, transform_tool]
)
```

Context Engineering

Context Engineering is the systematic discipline of designing, constructing, and delivering a complete informational environment to an AI model prior to token generation. This methodology asserts that the quality of a model's output is less dependent on the model's architecture itself and more on the richness of the context provided.

Context Engineering expands beyond traditional prompt engineering to include several layers of information: system prompts defining operational parameters, retrieved documents from knowledge bases, tool outputs from external APIs, and implicit data such as user identity, interaction history, and environmental state. The core principle is that even advanced models underperform when provided with a limited or poorly constructed view of the operational environment.

In prompt chaining, context engineering is crucial at each step. Each prompt in the chain should receive well-structured, relevant context from previous steps, along with any necessary external information. This ensures that each step has the information it needs to perform its specific operation effectively.

Tools like Google's Vertex AI prompt optimizer can automate the improvement process at scale, systematically evaluating responses against sample inputs and predefined metrics to refine contextual inputs across different models.

Key Takeaways

- **Core Concept:** Prompt chaining breaks down complex tasks into a sequence of smaller, focused steps, improving reliability and manageability.
- **Best Practice:** Use structured output formats (JSON, XML) between steps to ensure data integrity and prevent parsing errors.
- **Common Pitfall:** Over-chaining can add unnecessary latency; only chain when the complexity justifies multiple steps.
- **Performance Note:** Each step in the chain requires an LLM call, increasing latency and cost; consider caching intermediate results when possible.
- **Key Distinction:** This is a workflow pattern, not an agentic pattern—the sequence is predetermined by the engineer, not dynamically decided by the LLM.

Related Patterns

This pattern works well with:

- **Parallelization** - Independent sub-tasks can be processed in parallel before chaining dependent steps
- **Routing** - Chains can branch to different paths based on conditions
- **Tool Use** - External tools can be integrated at any step in the chain

This pattern is often combined with:

- **Structured Output** - Using JSON/XML formats between steps ensures data integrity
- **Context Engineering** - Each step benefits from well-engineered context
- **Reflection** - Chains can include reflection steps to review and refine outputs

References

- LangChain Documentation on LCEL: https://python.langchain.com/v0.2/docs/core_modules/expression_language/
 - LangGraph Documentation: <https://langchain-ai.github.io/langgraph/>
 - Prompt Engineering Guide - Chaining Prompts: <https://www.promptingguide.ai/techniques/chaining>
 - OpenAI API Documentation: <https://platform.openai.com/docs/guides/gpt/prompting>
 - Crew AI Documentation: <https://docs.crewai.com/>
 - Google AI for Developers: <https://cloud.google.com/discover/what-is-prompt-engineering?hl=en>
 - Vertex Prompt Optimizer: <https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/prompt-optimizer>
-

Pattern: Routing

Dynamically selecting between multiple potential actions based on input, state, or conditions, introducing conditional logic into agent workflows.

Module ID: module-4

ROUTING

Motivation

A receptionist listens to each caller and routes them to the right department—sales, support, or billing. A triage nurse assesses patients and directs them to appropriate specialists. We constantly make routing decisions: choosing which tool to use, which person to ask, or which path to take. Routing in agents works the same way: evaluating the situation and directing tasks to the most appropriate handler, tool, or workflow path.

Pattern Overview

What it is: Routing is a mechanism that enables agents to dynamically select between multiple potential actions based on input, state, or conditions, introducing conditional logic into agent workflows.

When to use: Use routing when your agent needs to adapt its behavior based on variable inputs, user intent, system state, or the outcome of previous operations, rather than following a fixed sequence.

Why it matters: Routing transforms agents from static executors of predetermined sequences into dynamic systems capable of making context-aware decisions. It enables specialization, where different tasks are handled by appropriate sub-agents or tools, improving both accuracy and efficiency.

While sequential processing via prompt chaining is foundational for deterministic, linear workflows, real-world agentic systems often need to arbitrate between multiple potential actions. Routing introduces conditional logic that governs the flow of control to different specialized functions, tools, or sub-processes based on contingent factors such as the state of the environment, user input, or the outcome of a preceding operation.

The core mechanism of routing involves evaluating specific criteria to select from a set of possible subsequent actions. For instance, a customer service agent might first classify an incoming query to determine user intent, then route it to a specialized agent for question-answering, a database retrieval tool for account information, or an escalation procedure for complex issues—rather than defaulting to a single predetermined response pathway.

Routing can be implemented at multiple junctures within an agent’s operational cycle: at the outset to classify a primary task, at intermediate points within a processing chain to determine subsequent actions, or during subroutines to select the most appropriate tool from a given set. This flexibility makes routing essential for building adaptive, context-aware agentic systems.

Key Concepts

- **Dynamic Decision-Making:** Routing enables agents to evaluate conditions and make runtime decisions about execution paths, moving beyond fixed sequences.
- **Intent Classification:** The pattern often begins with classifying user input or system state to determine the appropriate route, enabling specialized handling.
- **Conditional Flow Control:** Routing introduces if-then-else logic into agent workflows, where different conditions lead to different execution paths.
- **Multiple Implementation Methods:** Routing can be achieved through LLM-based analysis, embedding similarity, rule-based logic, or trained ML models, each with different trade-offs.
- **Complexity-Based Routing:** A specialized form of routing that classifies task complexity and routes to appropriate models or tools based on resource constraints, enabling cost-effective optimization by using lightweight models for simple tasks and powerful models for complex ones.

How It Works

Routing operates through a three-step process: evaluation, decision, and delegation. First, the routing mechanism evaluates the input, state, or condition using one of several methods. LLM-based routing uses the language model to analyze input and output a category or identifier. Embedding-based routing converts input to vectors and compares them to route embeddings using semantic similarity. Rule-based routing uses predefined logic (if-else statements) based on keywords or patterns. ML model-based routing employs a trained classifier to make routing decisions.

Based on this evaluation, the router makes a decision about which route to take. Finally, it delegates the task to the appropriate handler—whether that’s a specialized agent, a specific tool, or a different workflow path. The chosen route then receives the original input or a transformed version of it, processes the task,

and returns results that may feed back into the routing system for subsequent decisions.

When to Use This Pattern

✓ Use this pattern when:

- **Multiple specialized handlers exist:** You have different agents, tools, or workflows optimized for different types of tasks, and need to direct inputs to the right one.
- **User intent varies significantly:** User queries or inputs can have fundamentally different intents (e.g., booking vs. information vs. support), requiring different processing paths.
- **System state affects behavior:** The agent's behavior should change based on current state, previous outcomes, or environmental conditions.
- **Tool selection is dynamic:** You need to select from multiple available tools based on the task at hand, rather than using a fixed tool sequence.
- **Workflow branching is needed:** Different inputs or conditions require fundamentally different processing workflows, not just parameter variations.
- **Resource optimization is required:** You need to route tasks to different models or tools based on complexity, budget, or performance requirements (e.g., simple queries to fast/cheap models, complex tasks to powerful models).

✗ Avoid this pattern when:

- **Sequential processing is sufficient:** If all inputs follow the same processing steps in the same order, prompt chaining is simpler and more appropriate.
- **Routing logic is trivial:** If the routing decision can be made with a simple if-else based on a single, easily extractable feature, consider rule-based routing or even hardcoding the logic.
- **Latency is critical:** Routing adds an extra decision step, which can increase latency. For high-speed, low-latency requirements, consider pre-classification or simpler approaches.

Decision Guidelines

Choose routing when the benefits of specialization and adaptability outweigh the added complexity. Consider the routing method: LLM-based routing offers flexibility but adds latency and cost; embedding-based routing is fast and semantic but requires pre-computed route embeddings; rule-based routing is

deterministic and fast but less flexible; ML model-based routing offers good balance but requires training data. The choice depends on your accuracy requirements, latency constraints, and the complexity of the routing decision.

Practical Applications & Use Cases

Routing is essential for building adaptive agentic systems that can handle diverse inputs and contexts. Common applications include intent-based request handling, dynamic tool selection, and multi-agent coordination.

- **Customer Service Agents:** Route user queries to specialized handlers for orders, products, support, or general information based on intent classification.
- **Document Processing Pipelines:** Route incoming documents (emails, tickets, forms) to appropriate processing workflows based on content, format, or metadata.
- **Multi-Agent Systems:** Route tasks to specialized agents (researcher, writer, reviewer) based on task type and current workload.
- **AI Coding Assistants:** Route code snippets to different tools based on programming language, intent (debug, explain, translate), or complexity.
- **Content Classification Systems:** Route content to appropriate moderation, analysis, or processing pipelines based on type, topic, or risk level.
- **Resource-Aware Model Routing:** Route requests to different LLM models based on task complexity, budget constraints, or latency requirements. Simple, common questions are routed to fast, cost-efficient models (e.g., Claude Haiku, Gemini Flash), while complex or unusual questions are routed to more capable models (e.g., Claude Sonnet, Gemini Pro). This enables automatic optimization of cost and performance without manual intervention.

Implementation

Prerequisites

```
pip install langchain langchain-google-genai langgraph
# or
pip install google-adk
```

Basic Example

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableBranch, RunnablePassthrough

# Initialize LLM
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)

# Define router prompt
router_prompt = ChatPromptTemplate.from_messages([
    ("system", """Analyze the user's request and output ONE word:
    - 'booking' for flight/hotel bookings
    - 'info' for general questions
    - 'support' for technical issues
    Output only: booking, info, or support"""),
    ("user", "{request}")
])

# Define handlers
def booking_handler(request: str) → str:
    return f"Processing booking: {request}"

def info_handler(request: str) → str:
    return f"Answering question: {request}"

def support_handler(request: str) → str:
    return f"Escalating support: {request}"

# Create router chain
router_chain = router_prompt | llm | StrOutputParser()

# Create routing branches
routing_branch = RunnableBranch(

```

```

        (lambda x: "booking" in x['decision'].lower(),
         RunnablePassthrough.assign(output=lambda x: booking_handler(x['request']))),
        (lambda x: "support" in x['decision'].lower(),
         RunnablePassthrough.assign(output=lambda x: support_handler(x['request']))),
        RunnablePassthrough.assign(output=lambda x: info_handler(x['request']))
    )

# Combine into agent
agent = {
    "decision": router_chain,
    "request": RunnablePassthrough()
} | routing_branch

# Use
result = agent.invoke({"request": "Book me a flight to Paris"})
print(result['output'])

```

Explanation: This example demonstrates LLM-based routing. The router chain uses an LLM to classify the user's request into one of three categories. The RunnableBranch then routes to the appropriate handler based on the classification. This pattern enables dynamic decision-making while keeping the code structure clear and maintainable.

Advanced Example

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableBranch, RunnablePassthrough
from typing import Literal
import json

llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)

# Advanced router with structured output and fallback
router_prompt = ChatPromptTemplate.from_messages([
    ("system", """Analyze the request and return JSON with:
    - "route": one of ["booking", "info", "support", "unclear"]
    - "confidence": 0.0-1.0
    - "reasoning": brief explanation

    Return ONLY valid JSON."""),
    ("user", "{request}")
])

def parse_route(response: str) → dict:
    try:
        return json.loads(response)
    except:
        return {"route": "unclear", "confidence": 0.0, "reasoning": "Parse error"}

def route_with_confidence(route_data: dict, request: str) → dict:
    route = route_data.get("route", "unclear")
    confidence = route_data.get("confidence", 0.0)

    # Low confidence threshold
    if confidence < 0.7:
        return {"output": f"Unclear request. Please clarify: {request}", "route": "unclear"}
```

```

handlers = {
    "booking": lambda r: f"Booking system: {r}",
    "info": lambda r: f"Information service: {r}",
    "support": lambda r: f"Support ticket created: {r}",
    "unclear": lambda r: f"Need clarification: {r}"
}

handler = handlers.get(route, handlers["unclear"])
return {"output": handler(request), "route": route, "confidence": confidence}

# Create advanced routing chain
advanced_router = (
    router_prompt
    | llm
    | parse_route
    | (lambda x: route_with_confidence(x, x.get("request", "")))
)

# Usage with error handling
def route_request(request: str) → str:
    try:
        result = advanced_router.invoke({"request": request})
        return result.get("output", "Error processing request")
    except Exception as e:
        return f"Routing error: {str(e)}"

```

Explanation: This advanced example adds confidence scoring, structured JSON output, and error handling. The router returns not just a route decision but also confidence and reasoning, enabling the system to handle low-confidence cases by asking for clarification. This makes the routing more robust and transparent.

Complexity-Based Model Routing Example

This example demonstrates routing based on task complexity to optimize resource usage—using fast, cost-efficient models for simple tasks and powerful models for complex ones.

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableBranch, RunnablePassthrough

# Initialize different models for different complexity levels
fast_model = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)
powerful_model = ChatGoogleGenerativeAI(model="gemini-2.0-flash-exp", temperature=0)

# Complexity classifier
complexity_prompt = ChatPromptTemplate.from_messages([
    ("system", """Analyze the user's question and classify its complexity:
    - 'simple' for straightforward questions with clear answers (FAQs, definitions, basic facts)
    - 'complex' for questions requiring reasoning, analysis, or multi-step thinking

    Consider:
    - Simple: "What is Python?", "How do I install package X?", "What's the weather?"
    - Complex: "Explain the trade-offs between microservices and monoliths", "Debug this code"

    Output only: simple or complex"""),
    ("user", "{question}")
])

# Handlers using different models
def handle_simple(question: str) → str:
    """Use fast, cost-efficient model for simple questions"""
    prompt = ChatPromptTemplate.from_template("Answer this question concisely: {question}")
    chain = prompt | fast_model | StrOutputParser()
    return chain.invoke({"question": question})

def handle_complex(question: str) → str:
    """Use powerful model for complex questions requiring reasoning"""
    prompt = ChatPromptTemplate.from_template(
        "Think step-by-step and provide a detailed answer: {question}"
    )
    chain = prompt | powerful_model | StrOutputParser()
    return chain.invoke({"question": question})

```



```

    chain = prompt | powerful_model | StrOutputParser()

    return chain.invoke({"question": question})

# Create routing chain
complexity_classifier = complexity_prompt | fast_model | StrOutputParser()

# Route based on complexity
routing_branch = RunnableBranch(
    (lambda x: "simple" in x['complexity'].lower(),
     RunnablePassthrough.assign(
         answer=lambda x: handle_simple(x['question']),
         model_used="fast_model"
     )),
    RunnablePassthrough.assign(
        answer=lambda x: handle_complex(x['question']),
        model_used="powerful_model"
    )
)

# Combine into agent
complexity_router = {
    "complexity": complexity_classifier,
    "question": RunnablePassthrough()
} | routing_branch

# Usage
if __name__ == "__main__":
    # Simple question - will route to fast_model
    result = complexity_router.invoke({
        "question": "What is Python?"
    })
    print(f"Answer: {result['answer']}")
    print(f"Model used: {result['model_used']}\n")

    # Complex question - will route to powerful_model
    result = complexity_router.invoke({

```

```
        "question": "Explain the architectural trade-offs between event-driven and requ  
    })  
    print(f"Answer: {result['answer']}")  
    print(f"Model used: {result['model_used']}")
```

Explanation: This example demonstrates complexity-based routing for resource optimization. The router first classifies the question's complexity using a lightweight model, then routes simple questions to a fast, cost-efficient model and complex questions to a more capable (and expensive) model. This pattern enables automatic cost and performance optimization by matching task complexity to model capability.

Advanced Complexity Routing with Budget Awareness

This example extends complexity-based routing with budget tracking, enabling graceful degradation when budget constraints are tight.

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
import json
from typing import Dict, Any

# Models with different cost/performance profiles
models = {
    "haiku": ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0),
    "sonnet": ChatGoogleGenerativeAI(model="gemini-2.0-flash-exp", temperature=0),
}

# Budget tracker
class BudgetTracker:
    def __init__(self, budget: float = 100.0):
        self.budget = budget
        self.used = 0.0
        self.costs = {"haiku": 0.25, "sonnet": 1.0} # per request

    def can_afford(self, model: str) → bool:
        return (self.used + self.costs[model]) ≤ self.budget

    def use(self, model: str):
        if model in self.costs:
            self.used += self.costs[model]

    def get_remaining(self) → float:
        return self.budget - self.used

# Complexity and budget-aware router
router_prompt = ChatPromptTemplate.from_messages([
    ("system", """Analyze the question and return JSON:
    {{
        "complexity": "simple" | "complex",
        "estimated_tokens": number,

```

```

        "reasoning": "brief explanation"
    }}

    Consider:
    - Simple: clear, factual, single-step questions
    - Complex: requires reasoning, analysis, multi-step thinking, or creative solutions

    Return ONLY valid JSON.""",
    ("user", "{question}")
])

def parse_json_response(response: str) → Dict[str, Any]:
    """Safely parse JSON response from LLM"""
    try:
        # Try to extract JSON from response if it contains markdown code blocks
        if "```json" in response:
            start = response.find("```json") + 7
            end = response.find("```", start)
            response = response[start:end].strip()
        elif "```" in response:
            start = response.find("```") + 3
            end = response.find("```", start)
            response = response[start:end].strip()
        return json.loads(response)
    except (json.JSONDecodeError, ValueError):
        # Default to simple if parsing fails
        return {"complexity": "simple", "estimated_tokens": 100, "reasoning": "Parse er

def route_with_budget(question: str, budget_tracker: BudgetTracker) → Dict[str, Any]:
    """Route question based on complexity and available budget"""
    # Classify complexity using the cheaper model
    classifier_chain = router_prompt | models["haiku"] | StrOutputParser()
    response = classifier_chain.invoke({"question": question})
    classification = parse_json_response(response)

    complexity = classification.get("complexity", "simple")

```

```
# Route based on complexity and budget
if complexity == "simple":
    model_name = "haiku"
else:
    # Check if we can afford the powerful model
    if budget_tracker.can_afford("sonnet"):
        model_name = "sonnet"
    else:
        # Fallback to cheaper model if budget is low
        model_name = "haiku"

# Use the selected model
model = models[model_name]
answer_prompt = ChatPromptTemplate.from_template(
    "Answer this question: {question}"
)
answer_chain = answer_prompt | model | StrOutputParser()
answer = answer_chain.invoke({"question": question})

# Track usage
budget_tracker.use(model_name)

return {
    "answer": answer,
    "model_used": model_name,
    "complexity": complexity,
    "budget_remaining": budget_tracker.get_remaining()
}

# Usage
if __name__ == "__main__":
    tracker = BudgetTracker(budget=10.0)

    # Simple question
    result = route_with_budget("What is machine learning?", tracker)
```

```

print(f"Question: What is machine learning?")
print(f"Model: {result['model_used']}")
print(f"Complexity: {result['complexity']}")
print(f"Budget remaining: ${result['budget_remaining']:.2f}\n")

# Complex question
result = route_with_budget(
    "Design a distributed system architecture for handling 1 million concurrent use
    tracker
)
print(f"Question: Design a distributed system...")
print(f"Model: {result['model_used']}")
print(f"Complexity: {result['complexity']}")
print(f"Budget remaining: ${result['budget_remaining']:.2f}")

```

Explanation: This advanced example adds budget awareness to complexity-based routing. The router classifies complexity and checks available budget before selecting a model. If budget is constrained, it may route complex tasks to cheaper models, enabling graceful degradation while staying within resource limits. The budget tracker monitors usage and prevents overspending.

Framework-Specific Examples

LANGGRAPH

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class RouterState(TypedDict):
    request: str
    route: str
    output: str

def route_node(state: RouterState) → RouterState:
    # LLM-based routing logic
    route = llm.invoke(f"Route this: {state['request']}").content
    return {**state, "route": route}

def booking_node(state: RouterState) → RouterState:
    return {**state, "output": f"Booking: {state['request']}"}

def info_node(state: RouterState) → RouterState:
    return {**state, "output": f"Info: {state['request']}"}

# Build graph
graph = StateGraph(RouterState)
graph.add_node("route", route_node)
graph.add_node("booking", booking_node)
graph.add_node("info", info_node)

graph.add_conditional_edges(
    "route",
    lambda state: state["route"],
    {"booking": "booking", "info": "info"}
```

```
)
graph.add_edge("booking", END)
graph.add_edge("info", END)
```

GOOGLE ADK

```
from google.adk.agents import Agent
from google.adk.tools import FunctionTool

# Define specialized agents
booking_agent = Agent(
    name="Booker",
    model="gemini-2.0-flash",
    description="Handles all booking requests",
    tools=[booking_tool]
)

info_agent = Agent(
    name="Info",
    model="gemini-2.0-flash",
    description="Answers general questions",
    tools=[info_tool]
)

# Coordinator with routing
coordinator = Agent(
    name="Coordinator",
    model="gemini-2.0-flash",
    instruction="""Route requests:
- Booking requests → Booker agent
- Questions → Info agent""",
    sub_agents=[booking_agent, info_agent]
)
```


Key Takeaways

- **Core Concept:** Routing enables dynamic, conditional flow control in agent workflows, allowing specialization and adaptability.
- **Best Practice:** Use LLM-based routing for flexibility, embedding-based for speed, rule-based for determinism, or ML-based for balance.
- **Common Pitfall:** Over-routing can add unnecessary complexity; ensure routing decisions are meaningful and improve outcomes.
- **Performance Note:** Routing adds latency (especially LLM-based); consider caching route decisions for repeated similar inputs.
- **Resource Optimization:** Complexity-based routing enables automatic cost and performance optimization by matching task complexity to model capability, significantly reducing costs for simple tasks while maintaining quality for complex ones.

Related Patterns

This pattern works well with:

- **Multi-Agent** - Routing often directs tasks to specialized agents in multi-agent systems
- **Tool Use** - Routing can select appropriate tools based on task requirements
- **Planning** - Routing decisions can be part of a larger planning process

This pattern is often combined with:

- **Prompt Chaining** - Routes can lead to different prompt chains for different scenarios
- **Reflection** - Routing decisions can be reviewed and corrected through reflection

References

- LangChain Routing Documentation: https://python.langchain.com/docs/expression_language/how_to/routing
- LangGraph Conditional Edges: <https://langchain-ai.github.io/langgraph/how-tos/routing/>
- Google ADK Agents: <https://github.com/google/generative-ai-python/tree/main/google/adk>

Pattern: Parallelization

Executing multiple independent operations simultaneously to improve efficiency and reduce latency in agent workflows.

Module ID: module-5

PARALLELIZATION

Motivation

When hosting a dinner party, you don't cook dishes one at a time. You chop vegetables while the pasta boils, set the table while the sauce simmers, and delegate tasks to others. In a team project, people work on different parts simultaneously. Parallelization in agents mirrors this: executing independent operations concurrently to save time and increase efficiency, just as humans naturally multitask and coordinate parallel efforts.

Pattern Overview

What it is: Parallelization is a pattern for executing multiple independent tasks concurrently rather than sequentially, significantly reducing overall execution time for complex workflows.

When to use: Use parallelization when your workflow contains multiple independent operations that don't depend on each other's outputs and can be executed simultaneously.

Why it matters: Parallelization dramatically improves efficiency and responsiveness of agentic systems by leveraging concurrent execution. Instead of waiting for one task to complete before starting the next, independent tasks run simultaneously, reducing total execution time from the sum of all task durations to approximately the duration of the longest task.

While sequential processing via prompt chaining is foundational and routing enables dynamic decision-making, many complex agentic tasks involve multiple sub-tasks that can be executed simultaneously rather than one after another. Parallelization involves executing multiple components, such as LLM calls, tool usages, or even entire sub-agents, concurrently. Instead of waiting for one step to complete before starting the next, parallel execution allows independent tasks to run at the same time.

Consider an agent designed to research a topic and summarize its findings. A sequential approach might search for Source A, summarize it, then search for Source B, summarize it, and finally synthesize. A parallel approach could search for both sources simultaneously, then summarize both simultaneously, before synthesizing the final answer. The core idea is to identify parts of the workflow that do not depend on the output of other parts and execute them in parallel.

This pattern is particularly effective when dealing with external services (like APIs or databases) that have latency, as you can issue multiple requests concurrently. Implementing parallelization often requires frameworks that support asynchronous execution or multi-threading/multi-processing. Modern agentic frameworks are designed with asynchronous operations in mind, allowing you to easily define steps that can run in parallel.

Key Concepts

- **Concurrent Execution:** Multiple independent tasks run simultaneously rather than sequentially, reducing total execution time.
- **Independence Requirement:** Tasks must be independent—they cannot depend on each other’s outputs to run in parallel.
- **Asynchronous Operations:** Parallelization leverages `async/await` patterns or multi-threading to manage concurrent execution.
- **Convergence Points:** Parallel branches typically converge at a synthesis or aggregation step that combines their results.

How It Works

Parallelization works by identifying independent tasks in a workflow and executing them concurrently. The process typically involves: (1) identifying tasks that can run in parallel (no dependencies between them), (2) initiating all independent tasks simultaneously, (3) waiting for all tasks to complete, and (4) aggregating or synthesizing the results at a convergence point.

Frameworks provide different mechanisms for this. LangChain uses `RunnableParallel` to bundle multiple runnables that execute concurrently. LangGraph allows defining multiple nodes that can be executed from a single state transition, enabling parallel branches. Google ADK provides `ParallelAgent` and `SequentialAgent` constructs, where a `ParallelAgent` runs multiple sub-agents concurrently and stores their results in shared state for later synthesis.

When to Use This Pattern

✓ Use this pattern when:

- **Multiple independent lookups:** You need to gather information from multiple sources (APIs, databases, search engines) that don't depend on each other.
- **Batch processing:** You're processing multiple independent items (documents, queries, data points) that can be handled simultaneously.
- **Multi-modal processing:** You're analyzing different aspects or modalities of the same input concurrently (text sentiment + image analysis).
- **Validation checks:** You're performing multiple independent validation or verification steps that can run in parallel.
- **Content generation:** You're generating multiple independent components (headlines, body text, images) that will be combined later.

✗ Avoid this pattern when:

- **Tasks have dependencies:** If one task requires the output of another, they must run sequentially.
- **Resource constraints:** If you're limited by API rate limits, memory, or computational resources that can't handle concurrent execution.
- **Simple workflows:** If your workflow has only one or two steps, the overhead of parallelization may not be worth it.
- **Synchronization complexity:** If managing concurrent execution and result aggregation adds more complexity than benefit.

Decision Guidelines

Use parallelization when the time savings from concurrent execution outweigh the added complexity. Consider: the number of independent tasks (more tasks = more benefit), the latency of each task (higher latency = more time saved), and the framework's support for concurrent execution. Be aware that parallelization increases complexity in debugging, error handling, and logging. Also consider cost implications—running multiple LLM calls in parallel increases token usage, though it may reduce total wall-clock time.

Practical Applications & Use Cases

Parallelization is essential for optimizing agent performance across various applications where multiple independent operations can be executed simultaneously.

- **Information Gathering:** Collect data from multiple sources (news, APIs, databases) concurrently to build comprehensive views faster.
- **Data Processing:** Run multiple analysis techniques (sentiment, keywords, categorization) simultaneously on different data segments.
- **Multi-API Interaction:** Call multiple independent APIs (flights, hotels, events) concurrently to assemble complete information sets.
- **Content Generation:** Generate different components (subject lines, body text, images) in parallel for later assembly.
- **Validation:** Perform multiple independent checks (format, database, content filters) concurrently for faster feedback.

Implementation

Prerequisites

```
pip install langchain langchain-openai langgraph
# or
pip install google-adk
```

Basic Example

```
import asyncio

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# Define independent chains
summarize_chain = (
    ChatPromptTemplate.from_template("Summarize: {topic}")
    | llm
    | StrOutputParser()
)

questions_chain = (
    ChatPromptTemplate.from_template("Generate 3 questions about: {topic}")
    | llm
    | StrOutputParser()
)

terms_chain = (
    ChatPromptTemplate.from_template("List key terms from: {topic}")
    | llm
    | StrOutputParser()
)

# Execute in parallel
parallel_chain = RunnableParallel({
    "summary": summarize_chain,
    "questions": questions_chain,
    "key_terms": terms_chain,
    "topic": RunnablePassthrough()
})
```

```
})  
  
# Run  
result = parallel_chain.invoke("artificial intelligence")  
print(result)
```

Explanation: This example demonstrates parallel execution using LangChain's `RunnableParallel`. Three independent chains (`summarize`, `questions`, `terms`) execute concurrently on the same input topic. The results are collected in a dictionary, with all three operations completing in approximately the time of the slowest one, rather than the sum of all three.

Advanced Example

```
import asyncio
from typing import Dict, List
from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnableParallel

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

async def process_multiple_topics(topics: List[str]) → Dict[str, Dict]:
    """Process multiple topics in parallel with error handling."""

    # Create parallel chains for each topic
    chains = {}
    for topic in topics:
        chains[topic] = RunnableParallel({
            "summary": (lambda t: f"Summarize: {t}") | llm | StrOutputParser(),
            "analysis": (lambda t: f"Analyze: {t}") | llm | StrOutputParser(),
        })

    # Execute all in parallel with error handling
    tasks = [chain.ainvoke(topic) for topic, chain in chains.items()]
    results = await asyncio.gather(*tasks, return_exceptions=True)

    # Combine results
    return {
        topic: result if not isinstance(result, Exception) else {"error": str(result)}
        for topic, result in zip(topics, results)
    }

# Usage
topics = ["AI", "blockchain", "quantum computing"]
results = asyncio.run(process_multiple_topics(topics))
```

Explanation: This advanced example processes multiple topics in parallel, each with multiple sub-tasks. It includes error handling using `asyncio.gather` with `return_exceptions`, allowing the workflow to continue even if some tasks fail. This demonstrates production-ready parallelization with robust error management.

Framework-Specific Examples

LANGGRAPH

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class ParallelState(TypedDict):
    input: str
    result_a: str
    result_b: str
    result_c: str

def task_a(state: ParallelState) → ParallelState:
    return {**state, "result_a": f"Processed A: {state['input']}"}

def task_b(state: ParallelState) → ParallelState:
    return {**state, "result_b": f"Processed B: {state['input']}"}

def task_c(state: ParallelState) → ParallelState:
    return {**state, "result_c": f"Processed C: {state['input']}"}

# Build graph with parallel execution
graph = StateGraph(ParallelState)
graph.add_node("task_a", task_a)
graph.add_node("task_b", task_b)
graph.add_node("task_c", task_c)

# All three tasks start from the same entry point
graph.set_entry_point("task_a")
graph.add_edge("task_a", "task_b")
graph.add_edge("task_a", "task_c")
# Both b and c converge before END
```

GOOGLE ADK

```
from google.adk.agents import LlmAgent, ParallelAgent, SequentialAgent

# Define sub-agents that run in parallel
researcher_1 = LlmAgent(
    name="Researcher1",
    model="gemini-2.0-flash",
    instruction="Research topic A",
    output_key="result_a"
)

researcher_2 = LlmAgent(
    name="Researcher2",
    model="gemini-2.0-flash",
    instruction="Research topic B",
    output_key="result_b"
)

# Parallel execution
parallel_agent = ParallelAgent(
    name="ParallelResearch",
    sub_agents=[researcher_1, researcher_2]
)

# Synthesis agent runs after parallel completion
synthesis_agent = LlmAgent(
    name="Synthesis",
    model="gemini-2.0-flash",
    instruction="Combine {result_a} and {result_b}"
)

# Sequential orchestration
main_agent = SequentialAgent(
```

```

    name="Main",
    sub_agents=[parallel_agent, synthesis_agent]
)

```

Key Takeaways

- **Core Concept:** Parallelization executes independent tasks concurrently to reduce total execution time from sum to maximum.
- **Best Practice:** Identify truly independent tasks—those with no data dependencies—before parallelizing.
- **Common Pitfall:** Parallelizing dependent tasks leads to errors; ensure tasks are truly independent.
- **Performance Note:** Parallelization reduces latency but increases complexity and may increase total token costs (though reduces wall-clock time).

Related Patterns

This pattern works well with:

- **Prompt Chaining** - Parallel tasks often feed into sequential synthesis steps
- **Routing** - Different routes can execute in parallel when independent
- **Multi-Agent** - Multiple agents can work in parallel on independent sub-tasks

This pattern is often combined with:

- **Planning** - Plans can identify which tasks can run in parallel
- **Reflection** - Parallel results can be evaluated and refined

References

- LangChain Expression Language (LCEL) Documentation: <https://python.langchain.com/docs/concepts/lcel/>
- Google ADK Multi-Agent Systems: <https://google.github.io/adk-docs/agents/multi-agents/>
- Python asyncio Documentation: <https://docs.python.org/3/library/asyncio.html>

Pattern: Reflection

Enabling agents to review, critique, and refine their own outputs through iterative self-evaluation and improvement cycles.

Module ID: module-6

REFLECTION

Motivation

Writers revise their drafts, checking for clarity and flow. Scientists review their experiments before publication. Students review their answers before submitting. Reflection is our built-in quality control: stepping back, evaluating our work, identifying improvements, and refining until it meets our standards. The Reflection pattern gives agents this same ability: to review, critique, and improve their own outputs through iterative self-evaluation.

Pattern Overview

What it is: Reflection is a pattern where an agent evaluates its own work, output, or internal state and uses that evaluation to improve its performance or refine its response through iterative feedback loops.

When to use: Use reflection when output quality, accuracy, or adherence to complex constraints is critical, and you're willing to trade speed and cost for higher quality results.

Why it matters: Reflection enables self-correction and iterative refinement, transforming agents from single-pass executors into systems capable of improving their own outputs. This pattern is essential for producing high-quality, accurate results that meet complex requirements.

Even with sophisticated workflows using chaining, routing, and parallelization, an agent's initial output might not be optimal, accurate, or complete. The Reflection pattern introduces a feedback loop where the agent doesn't just produce an output; it examines that output, identifies potential issues or areas for improvement, and uses those insights to generate a better version or modify its future actions.

The process typically involves: (1) Execution—the agent performs a task or generates an initial output, (2) Evaluation/Critique—the agent analyzes the result checking for accuracy, coherence, completeness, or adherence to instructions, (3) Reflection/Refinement—based on the critique, the agent determines how to improve, and (4) Iteration—the refined output can be executed again, with the reflection process repeating until satisfactory or a stopping condition is met.

A key and highly effective implementation separates the process into two distinct roles: a Producer and a Critic (Generator-Critic model). While a single agent can perform self-reflection, using two specialized agents (or two separate LLM calls with distinct system prompts) often yields more robust and unbiased results. The Producer focuses on generating content, while the Critic evaluates it with a fresh perspective, dedicated entirely to finding errors and areas for improvement.

Key Concepts

- **Feedback Loop:** Reflection introduces a cyclical process of generation, evaluation, and refinement rather than linear execution.
- **Producer-Critic Model:** Separating generation and evaluation into distinct agents or roles prevents cognitive bias and improves objectivity.
- **Iterative Refinement:** The pattern enables multiple passes of improvement, with each iteration building on previous critiques.
- **Quality vs. Speed Trade-off:** Reflection improves quality but increases latency and cost due to multiple LLM calls.

How It Works

Reflection works through a structured feedback cycle. First, a Producer agent generates initial output based on the task. Then, a Critic agent (or the same agent in a different role) evaluates this output against specific criteria—factual accuracy, code quality, stylistic requirements, completeness, or adherence to instructions. The Critic provides structured feedback identifying flaws and suggesting improvements.

This feedback is then passed back to the Producer, which uses it to generate a refined version. The cycle can repeat until the Critic determines the output is satisfactory (often signaled by a “PERFECT” or “SATISFACTORY” status) or a maximum iteration limit is reached. The separation of Producer and Critic roles is powerful because it prevents the cognitive bias of an agent reviewing its own work—the Critic approaches with a fresh perspective.

Implementing reflection requires structuring workflows to include these feedback loops, often using state management and conditional transitions based on evaluation results. Frameworks like LangGraph support iterative loops natively, while LangChain can implement single reflection steps, and Google ADK facilitates reflection through sequential workflows where one agent's output is critiqued by another.

When to Use This Pattern

✅ Use this pattern when:

- **Quality is paramount:** The task requires high-quality, accurate, or polished outputs where errors are costly.
- **Complex requirements:** The task has multiple, nuanced requirements that are difficult to meet in a single pass.
- **Iterative improvement is possible:** The output can be refined based on feedback without starting completely over.
- **Specialized evaluation needed:** The task benefits from objective, specialized critique (code review, fact-checking, style analysis).
- **Error correction is critical:** The domain requires high accuracy and the ability to catch and fix errors.

❌ Avoid this pattern when:

- **Speed is critical:** Real-time or low-latency requirements make iterative refinement impractical.
- **Cost constraints:** Budget limitations make multiple LLM calls per task prohibitive.
- **Simple tasks:** The task is straightforward enough that a single pass produces adequate results.
- **Non-refinable outputs:** The output type doesn't benefit from iterative improvement (e.g., simple lookups).
- **Context window limits:** The iterative process would exceed context window capacity.

Decision Guidelines

Use reflection when the quality improvement justifies the added cost and latency. Consider: the complexity of requirements (more complex = more benefit), the cost of errors (high-stakes = worth the cost), and the refinability of the output (some outputs improve with iteration, others don't). The Producer-Critic model is

particularly valuable when objectivity is important or when specialized evaluation expertise is needed. Be mindful of iteration limits to prevent infinite loops and manage costs.

Practical Applications & Use Cases

Reflection is valuable in scenarios where output quality, accuracy, or adherence to complex constraints is critical.

- **Creative Writing:** Refine generated text, stories, or marketing copy through iterative critique and revision cycles.
- **Code Generation:** Write code, identify errors through testing or static analysis, and refine based on findings.
- **Complex Problem Solving:** Evaluate intermediate steps in multi-step reasoning, backtracking when needed.
- **Summarization:** Refine summaries for accuracy and completeness by comparing against source material.
- **Planning:** Evaluate proposed plans for feasibility and effectiveness, revising based on critique.

Implementation

Prerequisites

```
pip install langchain langchain-openai langgraph
# or
pip install google-adk
```

Basic Example

```

from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

llm = ChatOpenAI(model="gpt-4o", temperature=0.1)

def reflect_and_refine(task: str, max_iterations: int = 3):
    """Simple reflection loop for code generation."""

    current_output = ""
    message_history = [HumanMessage(content=task)]

    for i in range(max_iterations):
        # Producer: Generate or refine
        if i == 0:
            response = llm.invoke(message_history)
            current_output = response.content
        else:
            message_history.append(HumanMessage(
                content="Please refine based on the critique."
            ))
            response = llm.invoke(message_history)
            current_output = response.content

        message_history.append(response)

    # Critic: Evaluate
    critique_prompt = [
        SystemMessage(content="""You are a senior code reviewer.
        Evaluate the code. If perfect, say 'PERFECT'.
        Otherwise, provide critiques."""),
        HumanMessage(content=f"Task: {task}\n\nCode: {current_output}")
    ]

```

```
critique = llm.invoke(critique_prompt).content

if "PERFECT" in critique:
    break

message_history.append(HumanMessage(
    content=f"Critique: {critique}"
))

return current_output

# Use
code = reflect_and_refine("Write a Python function to calculate factorial")
print(code)
```

Explanation: This example demonstrates a basic reflection loop. The Producer generates code, the Critic evaluates it, and the Producer refines based on feedback. The loop continues until the Critic approves or max iterations are reached. This shows the core feedback mechanism of the Reflection pattern.

Advanced Example

```

from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage
from typing import Dict, List

llm = ChatOpenAI(model="gpt-4o", temperature=0.1)

class ReflectionAgent:
    def __init__(self, max_iterations: int = 5, quality_threshold: float = 0.9):
        self.max_iterations = max_iterations
        self.quality_threshold = quality_threshold
        self.llm = llm

    def produce(self, task: str, context: List) → str:
        """Producer agent generates output."""
        messages = context + [HumanMessage(content=task)]
        response = self.llm.invoke(messages)
        return response.content

    def critique(self, output: str, criteria: str) → Dict:
        """Critic agent evaluates output."""
        prompt = [
            SystemMessage(content=f"""You are an expert evaluator.
            Rate the output 0.0-1.0 and provide specific feedback.
            Criteria: {criteria}
            Return JSON: {{{"score": float, "feedback": str, "status": "PASS"|"FAIL"}}}"""),
            HumanMessage(content=output)
        ]
        response = self.llm.invoke(prompt)
        # Parse JSON response
        import json
        return json.loads(response.content)

    def refine(self, task: str, output: str, feedback: str, context: List) → str:

```

```

    """Producer refines based on critique."""
    messages = context + [
        HumanMessage(content=task),
        HumanMessage(content=f"Previous output: {output}"),
        HumanMessage(content=f"Feedback: {feedback}"),
        HumanMessage(content="Please refine the output.")
    ]
    response = self.llm.invoke(messages)
    return response.content

def reflect(self, task: str, criteria: str) → Dict:
    """Main reflection loop."""
    context = []
    iterations = []

    for i in range(self.max_iterations):
        # Produce
        output = self.produce(task, context)

        # Critique
        evaluation = self.critique(output, criteria)
        iterations.append({
            "iteration": i + 1,
            "output": output,
            "score": evaluation.get("score", 0.0),
            "feedback": evaluation.get("feedback", "")
        })

        # Check stopping condition
        if evaluation.get("status") == "PASS" or \
            evaluation.get("score", 0.0) ≥ self.quality_threshold:
            break

        # Refine
        output = self.refine(task, output, evaluation["feedback"], context)
        context.append(HumanMessage(content=f"Iteration {i+1} output: {output}"))

```

```
        return {
            "final_output": output,
            "iterations": iterations,
            "total_iterations": len(iterations)
        }

# Usage
agent = ReflectionAgent(max_iterations=5, quality_threshold=0.9)
result = agent.reflect(
    task="Write a comprehensive blog post about AI agents",
    criteria="Accuracy, clarity, engagement, completeness"
)
print(f"Final output after {result['total_iterations']} iterations")
print(result['final_output'])
```

Explanation: This advanced example implements a full `ReflectionAgent` class with structured evaluation, quality scoring, and detailed iteration tracking. The Critic provides structured JSON feedback with scores, enabling more sophisticated stopping conditions and quality monitoring. This demonstrates production-ready reflection with quality thresholds and comprehensive iteration history.

Framework-Specific Examples

LANGGRAPH

```

from langgraph.graph import StateGraph, END
from typing import TypedDict

class ReflectionState(TypedDict):
    task: str
    output: str
    critique: str
    iteration: int
    max_iterations: int

def produce_node(state: ReflectionState) → ReflectionState:
    # Producer generates output
    output = llm.invoke(f"Task: {state['task']}").content
    return {**state, "output": output}

def critique_node(state: ReflectionState) → ReflectionState:
    # Critic evaluates
    critique = llm.invoke(
        f"Evaluate: {state['output']}\nIf perfect, say 'PERFECT'"
    ).content
    return {**state, "critique": critique}

def should_continue(state: ReflectionState) → str:
    if "PERFECT" in state["critique"] or \
        state["iteration"] ≥ state["max_iterations"]:
        return "end"
    return "refine"

def refine_node(state: ReflectionState) → ReflectionState:
    # Refine based on critique
    output = llm.invoke(
        f"Refine based on: {state['critique']}\n{state['output']}"
    )

```



```
    ).content
    return {**state, "output": output, "iteration": state["iteration"] + 1}

# Build graph
graph = StateGraph(ReflectionState)
graph.add_node("produce", produce_node)
graph.add_node("critique", critique_node)
graph.add_node("refine", refine_node)

graph.set_entry_point("produce")
graph.add_edge("produce", "critique")
graph.add_conditional_edges("critique", should_continue, {
    "end": END,
    "refine": "refine"
})
graph.add_edge("refine", "produce")
```

GOOGLE ADK

```
from google.adk.agents import LlmAgent, SequentialAgent

# Producer agent
generator = LlmAgent(
    name="DraftWriter",
    model="gemini-2.0-flash",
    instruction="Write a short paragraph about the subject.",
    output_key="draft_text"
)

# Critic agent
reviewer = LlmAgent(
    name="FactChecker",
    model="gemini-2.0-flash",
    instruction="""Review the text in 'draft_text'.
    Return JSON: {"status": "ACCURATE"|"INACCURATE", "reasoning": str}""",
    output_key="review_output"
)

# Reflection pipeline
review_pipeline = SequentialAgent(
    name="WriteAndReview",
    sub_agents=[generator, reviewer]
)
```

Key Takeaways

- **Core Concept:** Reflection enables iterative self-correction through feedback loops of generation, evaluation, and refinement.
- **Best Practice:** Use a Producer-Critic model with separate agents or roles for more objective, unbiased evaluation.

- **Common Pitfall:** Reflection increases cost and latency; set iteration limits and quality thresholds to manage this.
- **Performance Note:** Each iteration requires additional LLM calls, increasing token usage and latency, but significantly improving output quality.

Related Patterns

This pattern works well with:

- **Goal Setting and Monitoring** - Goals provide benchmarks for evaluation, monitoring tracks progress
- **Memory Management** - Conversation history enables cumulative learning from past critiques
- **Planning** - Reflection can evaluate and refine plans before execution

This pattern is often combined with:

- **Exception Handling** - Reflection can identify and correct errors before they cause failures
- **Evaluation and Monitoring** - Structured evaluation metrics guide the reflection process

References

- LangGraph Iterative Workflows: <https://langchain-ai.github.io/langgraph/how-tos/iterative/>
 - Google ADK Sequential Agents: <https://google.github.io/adk-docs/agents/sequential/>
 - Self-Consistency and Chain-of-Thought: <https://arxiv.org/abs/2203.11171>
-

PART III

Tool Use & Execution

Designing the Agent-Computer Interface

Pattern: Tool Use & Execution

Designing the Agent-Computer Interface (ACI). Best practices for tool definitions and result management.

Module ID: module-7

TOOL USE & EXECUTION

Motivation

Humans extend their capabilities through tools: a hammer for construction, a calculator for math, a smartphone for communication. Each tool has a clear purpose, specific instructions for use, and predictable results when used correctly. Just as we learn to use tools by understanding their function and boundaries, agents need well-defined tools with clear descriptions, parameters, and constraints. The Tool Use pattern creates this interface between an agent's reasoning and the external world.

Pattern Overview

What it is: Tool Use (also known as Function Calling) is the pattern that enables agents to interact with external systems, APIs, databases, and services. It bridges the gap between an LLM's reasoning capabilities and the external world, allowing agents to perform actions, retrieve real-time data, execute code, and interact with other systems.

When to use: Use this pattern whenever an agent needs to break out of the LLM's internal knowledge and interact with the outside world. This is essential for tasks requiring real-time data, accessing private or proprietary information, performing precise calculations, executing code, or triggering actions in other systems.

Why it matters: LLMs are powerful text generators, but they are fundamentally disconnected from the outside world. Their knowledge is static, limited to training data, and they lack the ability to perform actions or retrieve real-time information. The Tool Use pattern transforms a language model from a text generator into an agent capable of sensing, reasoning, and acting in the digital or physical world.

The success of tool use relies critically on the quality and robustness of the **Agent-Computer Interface (ACI)**. The ACI is the tightly controlled, isolated execution runtime where the LLM's generated commands are translated into executable, verifiable code. Just as a poor UI confuses a human user, poorly defined, ambiguous, or unreliable tools lead to agent hallucinations, costly loops, and ultimate failure.

Key Concepts

- **Function Calling:** The technical mechanism where an LLM generates structured output (often JSON) specifying which function to call and with what arguments.
- **Tool Definition:** Clear descriptions of external functions or capabilities, including purpose, parameters, types, and boundaries.
- **Agent-Computer Interface (ACI):** The standardized contract between the generative model (planner) and the code execution environment (executor).
- **Idempotency:** A critical principle where a tool call should produce the same observable side effect regardless of how many times it is executed.
- **Tool Execution:** The orchestration layer that intercepts structured tool calls, executes the actual function, and returns results to the agent.
- **Observation:** The structured result returned from tool execution that the agent uses to inform its next decision.

How It Works

The Tool Use pattern operates through a structured process:

1. **Tool Definition:** External functions or capabilities are defined and described to the LLM. This description includes the function's purpose, name, parameters, types, and what it cannot do.
2. **LLM Decision:** The LLM receives the user's request and available tool definitions. Based on its understanding, it decides if calling one or more tools is necessary to fulfill the request.
3. **Function Call Generation:** If the LLM decides to use a tool, it generates structured output (often JSON) specifying the tool name and arguments extracted from the user's request.

4. **Tool Execution:** The orchestration layer intercepts this structured output, identifies the requested tool, and executes the actual external function with the provided arguments.
5. **Observation/Result:** The output or result from tool execution is returned to the agent as an observation.
6. **LLM Processing:** The LLM receives the tool's output as context and uses it to formulate a final response or decide on the next step (which might involve calling another tool, reflecting, or providing a final answer).

While “function calling” describes invoking specific, predefined code functions, “tool calling” is a broader concept. A tool can be a traditional function, a complex API endpoint, a database request, or even an instruction directed at another specialized agent. This perspective enables sophisticated systems where agents orchestrate across diverse digital resources and intelligent entities.

When to Use This Pattern

Use this pattern when:

- **Real-time data needed:** Tasks requiring current information not in the LLM's training data (weather, stock prices, news).
- **External system interaction:** Tasks that need to interact with APIs, databases, file systems, or other services.
- **Precise calculations required:** Tasks needing deterministic computations that LLMs cannot perform reliably.
- **Code execution needed:** Tasks requiring running code snippets in a safe environment.
- **Action triggering:** Tasks that need to trigger actions in other systems (send emails, control devices, update databases).
- **Private/proprietary data access:** Tasks requiring access to user-specific or company-specific information not in public training data.
- **Dynamic information retrieval:** Tasks that need to search, query, or retrieve information from external sources.

Avoid this pattern when:

- **Pure text generation:** Tasks that only require generating text based on the LLM's training data.

- **No external dependencies:** Tasks that can be completed entirely within the LLM’s knowledge and reasoning capabilities.
- **Simple Q&A:** Basic questions that can be answered from the model’s training data without external lookup.
- **Cost-sensitive scenarios:** When the overhead of tool calls (latency, API costs) outweighs benefits.
- **Security-critical systems:** When tool execution introduces unacceptable security risks that cannot be mitigated.

Decision Guidelines

Use Tool Use when the task requires information or capabilities beyond what the LLM can provide from its training data alone. If the task needs real-time data, interaction with external systems, or the ability to perform actions, Tool Use is essential. However, if the task can be completed with the LLM’s internal knowledge and reasoning, avoid adding unnecessary complexity.

Practical Applications & Use Cases

The Tool Use pattern is applicable in virtually any scenario where an agent needs to go beyond generating text to perform an action or retrieve specific, dynamic information:

1. Information Retrieval from External Sources

Use Case: A weather agent that provides current weather conditions.

- **Tool:** A weather API that takes a location and returns current weather conditions.
- **Agent Flow:** User asks “What’s the weather in London?”, LLM identifies the need for the weather tool, calls the tool with “London”, tool returns data, LLM formats the data into a user-friendly response.

2. Interacting with Databases and APIs

Use Case: An e-commerce agent that checks inventory and order status.

- **Tools:** API calls to check product inventory, get order status, or process payments.
- **Agent Flow:** User asks “Is product X in stock?”, LLM calls the inventory API, tool returns stock count, LLM tells the user the stock status.

3. Performing Calculations and Data Analysis

Use Case: A financial agent that calculates profits and analyzes stock data.

- **Tools:** A calculator function, a stock market data API, a spreadsheet tool.
- **Agent Flow:** User asks “What’s the current price of AAPL and calculate the potential profit if I bought 100 shares at \$150?”, LLM calls stock API, gets current price, then calls calculator tool, gets result, formats response.

4. Sending Communications

Use Case: A personal assistant agent that sends emails.

- **Tool:** An email sending API.
- **Agent Flow:** User says “Send an email to John about the meeting tomorrow.”, LLM calls an email tool with the recipient, subject, and body extracted from the request.

5. Executing Code

Use Case: A coding assistant agent that runs and analyzes code.

- **Tool:** A code interpreter.
- **Agent Flow:** User provides a Python snippet and asks “What does this code do?”, LLM uses the interpreter tool to run the code and analyze its output.

6. Controlling Other Systems or Devices

Use Case: A smart home agent that controls IoT devices.

- **Tool:** An API to control smart lights.
- **Agent Flow:** User says “Turn off the living room lights.” LLM calls the smart home tool with the command and target device.

Implementation

Engineering Best Practices

1. EXPLICIT TOOL DEFINITIONS & QUALITY

Tools must have clear, distinct descriptions, strict boundaries, and required parameters. The quality of the tool description directly impacts the model's accuracy. Descriptions must detail:

- **Purpose:** What the tool solves
- **Boundaries:** What it cannot do
- **Parameter Names:** Use descriptive, obvious parameter names that make their purpose clear (e.g., `absolute_filepath` instead of `path`, `user_email_address` instead of `email`)
- **Parameter Types:** Clearly specify types and formats for all parameters
- **Negative examples:** Show the agent when not to use a tool (e.g., “Do not use `search_api` for real-time stock quotes; use `get_stock_price` instead”)
- **Example Usage:** Include example usage to illustrate how the tool should be called

Use structured data types (like Pydantic schemas in Python or TypeScript interfaces) to enforce type safety and parameter validation. The docstring for the tool should be the single source of truth for the model's understanding.

Put yourself in the model's shoes: Is it obvious how to use this tool based on the description and parameters? If you need to think carefully about it, the model will too. A good tool definition often includes example usage, edge cases, input format requirements, and clear boundaries from other tools.

1.5. TOOL FORMAT DESIGN: CHOOSING LLM-FRIENDLY FORMATS

When designing tools, there are often multiple ways to specify the same action. However, some formats are significantly more difficult for LLMs to generate correctly than others. The format you choose can dramatically impact reliability and error rates.

Key Principles for Tool Format Selection:

1. Give the model enough tokens to “think” before writing itself into a corner
 - Avoid formats that require precise counts or calculations before generation

- Example: Writing a diff requires knowing how many lines are changing in the chunk header before the new code is written—this is error-prone for LLMs

2. Keep formats close to what models see naturally in training data

- Formats that appear frequently in the model’s training corpus are easier to generate
- Example: Code in markdown code blocks is more natural than code inside JSON strings

3. Minimize formatting “overhead”

- Avoid formats that require complex escaping, counting, or precise formatting
- Example: Writing code inside JSON requires extra escaping of newlines and quotes, making it harder for the model
- Example: Requiring accurate line counts for thousands of lines of code is error-prone

Practical Examples:

Avoid: Diff-based file editing

```
# Difficult for LLMs - requires precise line counting
def edit_file(filepath: str, diff: str) → str:
    """
    Edit a file using a diff format.

    diff format: '@@ -start_line,count +start_line,count @@'
    Example: '@@ -5,3 +5,4 @@\n old line 1\n-old line 2\n+new line 2'
    """
```

Prefer: Full file replacement or append operations

```
# Easier for LLMs - no line counting required
def write_file(filepath: str, content: str) → str:
    """
    Write content to a file. If file exists, replaces it entirely.
    content: The complete file contents as a string.
    """
```

❌ Avoid: Code in JSON strings

```
# Requires escaping newlines and quotes
def execute_code(code_json: str) → str:
    """
    Execute code provided as JSON string.
    Example: {"code": "def hello():\n    print('hi')"}
    """
```

✅ Prefer: Code in markdown or plain text

```
# Natural format, no escaping needed
def execute_code(code: str) → str:
    """
    Execute Python code provided as a string.
    Code should be valid Python syntax.
    """
```

Poka-Yoke (Error-Proofing) Your Tools:

Apply the Japanese concept of “poka-yoke” (mistake-proofing) to tool design by making it harder for the model to make mistakes:

- **Use absolute paths instead of relative paths:** After an agent changes directories, relative paths become ambiguous. Requiring absolute paths eliminates this source of error.
- **Use structured types instead of free-form strings:** Instead of accepting a date as “2024-01-15” or “January 15, 2024”, require a specific ISO format.
- **Provide clear boundaries:** Explicitly state what the tool cannot do to prevent misuse.
- **Use descriptive parameter names:** Parameter names should make their purpose obvious (e.g., `absolute_filepath` instead of `path`).

Example from Production:

When building a coding agent for SWE-bench, Anthropic found that the model made mistakes with tools using relative filepaths after the agent had moved out of the root directory. By changing the tool to always require absolute filepaths, the model used the method flawlessly.

Rule of Thumb:

Think about how much effort goes into Human-Computer Interfaces (HCI), and invest similar effort in creating good Agent-Computer Interfaces (ACI). Put yourself in the model's shoes: Is it obvious how to use this tool based on the description and parameters? If you need to think carefully about it, the model will too. A good tool definition often includes example usage, edge cases, input format requirements, and clear boundaries from other tools.

2. INPUT & OUTPUT VALIDATION AND PRUNING

Add a crucial layer of security and robustness between the LLM and external systems:

- **Pre-Execution Checks:** Verify that parameters generated by the LLM (file paths, database IDs, amounts) are safe, adhere to business logic (non-negative financial values, date ranges), and prevent security risks like path traversal attacks or injection attempts.
- **Post-Execution Sanitization:** Simplify and prune complex, nested JSON or verbose API outputs into concise, token-efficient observations. Use structured querying languages (like JQ or JSONPath) to extract only the most relevant fields, preventing context window bloat.

3. HANDLING CONTEXT SWITCHING

When a tool is called, the orchestrator needs to pause the LLM's reasoning chain, execute the tool, and resume by injecting the observation. This context switch must be seamless. Prepend the observation directly to the history, ensuring it acts as the most recent, high-attention piece of data, mitigating the "Lost in the Middle" problem.

4. CONSTRAINED TOOL USE & EXECUTION GUARDRAILS

Implement safety mechanisms to prevent harmful, costly, or unproductive behavior:

- **Safety and Recursive Guardrails:** Set strict maximum number of steps in the ReAct loop, use exponential backoff for retries, and proactively block recursive calls (same tool with same input repeatedly).
- **Cost and Rate Monitoring:** Implement runtime counters and alerts for expensive tools, enforce per-session and global rate limits on external APIs.

- **Tool Sandbox Isolation:** Fully sandbox the execution environment, particularly for tools that execute arbitrary code. Use containerization technologies like Docker or gVisor to ensure agent actions are strictly constrained.

5. TOOL RESULT MANAGEMENT (RETRIEVE-THEN-READ)

For large tool outputs, use a two-step process:

- **Retrieve:** Get a pointer, list of resource IDs, or brief summary
- **Read:** Selectively pull in only specific, relevant content snippets based on reasoning

This minimizes token consumption by avoiding full dumps of large data into the context.

Code Examples

LANGCHAIN IMPLEMENTATION

```
import os
import asyncio
import nest_asyncio

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.tools import tool as langchain_tool
from langchain.agents import create_tool_calling_agent, AgentExecutor

# Initialize the language model
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)

# Define a Tool
@langchain_tool
def search_information(query: str) → str:
    """
    Provides factual information on a given topic. Use this tool to
    find answers to phrases like 'capital of France' or 'weather in London?'.
    """
    print(f"\n--Tool Called: search_information with query: '{query}' ---")

    # Simulate a search tool with predefined results
    simulated_results = {
        "weather in london": "The weather in London is currently cloudy with a temperat
        "capital of france": "The capital of France is Paris.",
        "population of earth": "The estimated population of Earth is around 8 billion p
        "tallest mountain": "Mount Everest is the tallest mountain above sea level.",
        "default": f"Simulated search result for '{query}': No specific information fou
    }

    result = simulated_results.get(query.lower(), simulated_results["default"])
    print(f"--- TOOL RESULT: {result} ---")
    return result
```

```

tools = [search_information]

# Create a Tool-Calling Agent
agent_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])

agent = create_tool_calling_agent(llm, tools, agent_prompt)
agent_executor = AgentExecutor(agent=agent, verbose=True, tools=tools)

async def run_agent_with_tool(query: str):
    """Invokes the agent executor with a query and prints the final response."""
    print(f"\n--Running Agent with Query: '{query}' ---")
    try:
        response = await agent_executor.ainvoke({"input": query})
        print("\n--Final Agent Response ---")
        print(response["output"])
    except Exception as e:
        print(f"\nAn error occurred during agent execution: {e}")

async def main():
    """Runs all agent queries concurrently."""
    tasks = [
        run_agent_with_tool("What is the capital of France?"),
        run_agent_with_tool("What's the weather like in London?"),
        run_agent_with_tool("Tell me something about dogs.")
    ]
    await asyncio.gather(*tasks)

nest_asyncio.apply()
asyncio.run(main())

```


CREWAI IMPLEMENTATION

```
import os

from crewai import Agent, Task, Crew

from crewai.tools import tool

import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s - %(message)s')

# Define a Tool
@tool("Stock Price Lookup Tool")
def get_stock_price(ticker: str) → float:
    """
    Fetches the latest simulated stock price for a given stock ticker symbol.
    Returns the price as a float. Raises a ValueError if the ticker is not found.
    """
    logging.info(f"Tool Call: get_stock_price for ticker '{ticker}'")

    simulated_prices = {
        "AAPL": 178.15,
        "GOOGL": 1750.30,
        "MSFT": 425.50,
    }

    price = simulated_prices.get(ticker.upper())

    if price is not None:
        return price
    else:
        raise ValueError(f"Simulated price for ticker '{ticker.upper()}' not found.")

# Define the Agent
financial_analyst_agent = Agent(
    role='Senior Financial Analyst',
    goal='Analyze stock data using provided tools and report key prices.',
    backstory="You are an experienced financial analyst adept at using data sources to",
    verbose=True,
```

```

        tools=[get_stock_price],
        allow_delegation=False,
    )

# Define the Task
analyze_aapl_task = Task(
    description=(
        "What is the current simulated stock price for Apple (ticker: AAPL)? "
        "Use the 'Stock Price Lookup Tool' to find it. "
        "If the ticker is not found, you must report that you were unable to retrieve t
    ),
    expected_output=(
        "A single, clear sentence stating the simulated stock price for AAPL. "
        "For example: 'The simulated stock price for AAPL is $178.15.'"
    ),
    agent=financial_analyst_agent,
)

# Formulate the Crew
financial_crew = Crew(
    agents=[financial_analyst_agent],
    tasks=[analyze_aapl_task],
    verbose=True
)

# Run the Crew
def main():
    """Main function to run the crew."""
    if not os.environ.get("OPENAI_API_KEY"):
        print("ERROR: The OPENAI_API_KEY environment variable is not set.")
        return

    print("\n## Starting the Financial Crew...")
    result = financial_crew.kickoff()
    print("\n## Crew execution finished.")
    print("\nFinal Result:\n", result)

```

```
if __name__ == "__main__":  
    main()
```

GOOGLE ADK IMPLEMENTATION

```

import asyncio

from google.adk.agents import Agent as ADKAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import google_search
from google.genai import types
import nest_asyncio

# Define variables
APP_NAME = "Google Search_agent"
USER_ID = "user1234"
SESSION_ID = "1234"

# Define Agent with access to search tool
root_agent = ADKAgent(
    name="basic_search_agent",
    model="gemini-2.0-flash-exp",
    description="Agent to answer questions using Google Search.",
    instruction="I can answer your questions by searching the internet. Just ask me any",
    tools=[google_search] # Google Search is a pre-built tool
)

# Agent Interaction
async def call_agent(query):
    """Helper function to call the agent with a query."""
    session_service = InMemorySessionService()
    session = await session_service.create_session(
        app_name=APP_NAME,
        user_id=USER_ID,
        session_id=SESSION_ID
    )

    runner = Runner(agent=root_agent, app_name=APP_NAME, session_service=session_service)
    content = types.Content(role='user', parts=[types.Part(text=query)])

```

```

events = runner.run(user_id=USER_ID, session_id=SESSION_ID, new_message=content)

for event in events:
    if event.is_final_response():
        final_response = event.content.parts[0].text
        print("Agent Response: ", final_response)

nest_asyncio.apply()
asyncio.run(call_agent("what's the latest ai news?"))

```

Key Takeaways

- **Tool Use (Function Calling)** allows agents to interact with external systems and access dynamic information beyond their training data.
- **Agent-Computer Interface (ACI)** is the critical contract between the LLM and execution environment, requiring careful design for reliability and security.
- **Idempotency** is essential: tool calls should produce the same observable side effect regardless of execution count.
- **Tool definitions** must be clear, detailed, and include boundaries and negative examples to guide proper usage.
- **Input/output validation** adds crucial security and robustness layers between the LLM and external systems.
- **Frameworks** like LangChain, CrewAI, and Google ADK provide abstractions that simplify tool integration and execution.
- **Google ADK** includes pre-built tools like Google Search, Code Execution, and Vertex AI Search that can be directly integrated.
- **Tool Use** transforms language models from text generators into agents capable of real-world action and up-to-date information retrieval.

Related Patterns

- **Planning:** Tool Use often works in conjunction with Planning, where agents create structured plans that include tool calls as steps.
- **Routing:** Routing can determine which tools are available or appropriate based on context or user permissions.
- **Reflection:** Tool results can be evaluated and refined through Reflection patterns.
- **Multi-Agent Architectures:** Different agents can have different tool sets, enabling specialization.
- **Exception Handling:** Robust error handling is critical when tools fail or return unexpected results.
- **Memory Management:** Tool results may need to be stored in external memory systems for later retrieval.

References

1. LangChain Documentation (Tools): <https://python.langchain.com/docs/integrations/tools/>
 2. Google Agent Developer Kit (ADK) Documentation (Tools): <https://google.github.io/adk-docs/tools/>
 3. OpenAI Function Calling Documentation: <https://platform.openai.com/docs/guides/function-calling>
 4. CrewAI Documentation (Tools): <https://docs.crewai.com/concepts/tools>
-

Pattern: Constrained Tool Use (Mask, Don't Remove)

Managing tool availability through programmatic constraints (logit masking) rather than dynamically modifying tool definitions, preserving KV-Cache efficiency and preventing model confusion.

Module ID: module-7a

CONSTRAINED TOOL USE (MASK, DON'T REMOVE)

Motivation

In a workshop, you might have a full toolbox but only need a few tools for a specific job. Rather than removing tools from the box, you simply focus on the ones you need, keeping others available but out of the way. Similarly, when working in a restricted environment, you adapt to what's available rather than changing your entire setup. Constrained Tool Use applies this principle: limiting tool availability through programmatic constraints while keeping the full toolset defined, preventing confusion and maintaining efficiency.

Pattern Overview

What it is: A mechanism to manage tool availability by using programmatic constraints (like logit masking) to prevent selection, rather than dynamically modifying the tool definitions in the context.

When to use: When the set of available tools must remain stable but the agent's permission or capability to use a tool must change based on the current state.

Why it matters: Dynamically altering tool definitions mid-run breaks the **KV-Cache** (Key-Value Cache) and confuses the model. Keeping tool definitions stable is critical for maintaining performance, reducing cost, and preventing the model from hallucinating tool actions.

As agents take on more capabilities, their action space naturally grows more complex—the number of tools explodes. With the popularity of MCP (Model Context Protocol) and user-configurable tools, agents can have hundreds of tools available. This complexity increases the likelihood of selecting the wrong action or taking an inefficient path. A natural reaction might be to design a dynamic action space—perhaps loading tools on demand using something RAG-like. However, experiments show a clear rule: unless absolutely necessary, avoid dynamically adding or removing tools mid-iteration.

The Constrained Tool Use pattern addresses this challenge by maintaining stable tool definitions while programmatically constraining which tools can be selected at any given moment. This approach preserves KV-Cache efficiency, prevents model confusion, and ensures reliable tool invocation.

Key Concepts

- **Logit Masking:** Programmatically preventing the LLM from generating tokens corresponding to unavailable tools during decoding, without removing tool definitions from context.
- **KV-Cache Stability:** Maintaining stable context prefixes (including tool definitions) to maximize Key-Value cache reuse, directly improving latency and reducing costs.
- **Response Prefilling:** Using inference framework capabilities to constrain output by prefilling response tokens, effectively limiting the action space without modifying tool definitions.
- **Context Engineering:** Strategic management of what appears in context to optimize performance, cost, and reliability.
- **Fixed Tool Definitions:** Keeping all tool definitions stable throughout a session, typically positioned near the front of the context after serialization.
- **State-Aware Constraints:** Using a context-aware state machine to manage tool availability based on current agent state.

How It Works: Step-by-step Explanation

1. **Define All Tools:** All possible tools and their descriptions are defined once and fixed at the start of the session. These definitions typically reside near the front of the context, before or after the system prompt.

2. **Maintain Context Stability:** The tool definitions remain in the context throughout the session to maximize the KV-cache hit rate. A single-token difference in the prompt prefix can invalidate the cache, leading to significant cost increases (up to 10× difference in uncached tokens).
3. **Apply Constraint (Masking):** When a specific tool is unavailable based on the agent's current state (e.g., the agent is not in the correct virtual environment to use a shell command), the agent framework uses programmatic constraints during the decoding step. This can be done through:
 - **Logit Masking:** Physically preventing the LLM from outputting tokens corresponding to unavailable tools
 - **Response Prefilling:** Using inference framework capabilities to constrain output by prefilling response tokens
4. **Prevent Selection:** This masking physically prevents the LLM from outputting the tokens corresponding to the unavailable tool's name, without removing the tool's definition from the context. This ensures stability while still guiding the agent's action space.

When to Use This Pattern

Use when:

- You need to enforce that the agent selects only from a certain group of tools at a given state.
- Maintaining a fixed tool set is required for KV-cache optimization.
- Previous actions and observations refer to tools that must remain defined in the context.
- You have a large, complex action space (hundreds of tools) where dynamic selection is needed.
- Tool availability must change based on agent state (e.g., environment, permissions, workflow stage).
- You need to prevent schema violations or hallucinated tool calls.

Avoid when:

- You are dealing with a task so simple that KV-cache optimization is unnecessary.
- You attempt to dynamically alter the tool set per turn by physically adding or removing tool definitions, as this breaks caching and confuses the model.
- The tool set is small and stable enough that constraints aren't needed.
- Tool availability changes are rare and don't justify the complexity of masking logic.

Decision Guidelines

Keep the tool set fixed during a single problem-solving episode. Use soft restrictions (masking) rather than physically adding/removing tool definitions mid-run. A single-token difference in the prompt prefix can invalidate the cache, leading to significant cost increases (up to 10× difference in uncached tokens). When tool definitions change, previous actions and observations still refer to tools that are no longer defined, which confuses the model and often leads to schema violations or hallucinated actions without constrained decoding.

Practical Applications & Use Cases

The Constrained Tool Use pattern is essential for managing complex action spaces while maintaining performance and reliability.

- **Prefix-Based Masking:** Manus AI uses this pattern by naming tools with consistent prefixes (e.g., all browser-related tools start with `browser_`, command-line tools with `shell_`) to easily enforce that the agent chooses only from a certain group of tools at a given state without using stateful logits processors.
- **Preventing Tool Misuse:** The pattern helps mitigate risk by constraining the agent's output to only available and authorized tools, preventing the generation of schema violations or hallucinated tool calls.
- **State-Dependent Tool Access:** When an agent must reply immediately to user input instead of taking an action, masking prevents action selection while keeping all tool definitions available for later use.
- **Environment-Based Constraints:** Tools can be masked based on the agent's current environment (e.g., no shell commands in a sandboxed environment, no database access without proper authentication).
- **Workflow Stage Constraints:** Different stages of a workflow can have different tool availability (e.g., research stage allows only search tools, execution stage allows only action tools).
- **Permission-Based Access:** Tools can be masked based on user permissions or security policies without removing them from the context.

Implementation

Prerequisites

```
# Most inference frameworks support logit masking or response prefilling  
# Check your framework's documentation for specific implementation details
```

Basic Example: Prefix Design Strategy

The implementation often involves designing tool names with consistent prefixes and then leveraging the inference framework's ability to constrain output based on prefixes or logit masking:

```

from typing import List, Dict, Set
from enum import Enum

class AgentState(Enum):
    EXPLORING = "exploring" # Can only use browser tools
    EXECUTING = "executing" # Can only use shell tools
    QUERYING = "querying" # Can only use database tools
    RESPONDING = "responding" # No tools, must respond directly

class ConstrainedToolManager:
    def __init__(self):
        # Define all tools with consistent prefixes
        self.all_tools = {
            "browser_search": "Search the web for information",
            "browser_click_link": "Click on a link in the browser",
            "browser_navigate": "Navigate to a URL",
            "shell_execute": "Execute a shell command",
            "shell_read_file": "Read a file from the filesystem",
            "query_database": "Query the database with SQL",
            "query_get_schema": "Get database schema information"
        }

        # Define tool groups by prefix
        self.tool_groups = {
            "browser_": ["browser_search", "browser_click_link", "browser_navigate"],
            "shell_": ["shell_execute", "shell_read_file"],
            "query_": ["query_database", "query_get_schema"]
        }

        self.current_state = AgentState.RESPONDING

    def get_allowed_tools(self, state: AgentState) → Set[str]:
        """Get allowed tools for a given state."""
        if state == AgentState.EXPLORING:
            return set(self.tool_groups["browser_"])

```

```

elif state == AgentState.EXECUTING:
    return set(self.tool_groups["shell_"])
elif state == AgentState.QUERYING:
    return set(self.tool_groups["query_"])
else: # RESPONDING
    return set() # No tools allowed

def get_allowed_prefixes(self, state: AgentState) → List[str]:
    """Get allowed tool name prefixes for masking."""
    if state == AgentState.EXPLORING:
        return ["browser_"]
    elif state == AgentState.EXECUTING:
        return ["shell_"]
    elif state == AgentState.QUERYING:
        return ["query_"]
    else:
        return [] # No prefixes allowed

def set_state(self, state: AgentState):
    """Update agent state and return masking configuration."""
    self.current_state = state
    allowed_prefixes = self.get_allowed_prefixes(state)
    return {
        "allowed_prefixes": allowed_prefixes,
        "mask_all_tools": len(allowed_prefixes) == 0
    }

# Usage
manager = ConstrainedToolManager()

# Agent is exploring - only browser tools allowed
masking_config = manager.set_state(AgentState.EXPLORING)

# Framework would use this to mask logits for tools not starting with "browser_"

```

```
# Agent must respond - no tools allowed  
masking_config = manager.set_state(AgentState.RESPONDING)  
# Framework would mask all tool-related tokens
```

Explanation: This example demonstrates the prefix-based strategy where tools are named with consistent prefixes. The framework can then use these prefixes to mask logits during decoding, preventing selection of unavailable tools while keeping all definitions in context.

Advanced Example: Response Prefilling with Hermes Format

This example shows how to use response prefilling to constrain the action space:

```

from typing import Optional, List
import json

class ToolConstraintManager:
    """Manages tool constraints using response prefilling."""

    def __init__(self):
        self.all_tools = {
            "browser_search": {"description": "Search the web"},
            "browser_click": {"description": "Click a link"},
            "shell_execute": {"description": "Execute command"},
            "query_db": {"description": "Query database"}
        }
        self.current_constraint = None

    def get_response_prefix(self, constraint_mode: str, allowed_tools: Optional[List[str]]):
        """
        Generate response prefix based on constraint mode.

        Hermes format examples:
        - Auto: <im_start>assistant
        - Required: <im_start>assistant<tool_call>
        - Specified: <im_start>assistant<tool_call>{"name": "browser_"}
        """
        base_prefix = "<im_start>assistant"

        if constraint_mode == "auto":
            # Model may choose to call a function or not
            return base_prefix

        elif constraint_mode == "required":
            # Model must call a function, but choice is unconstrained
            return f"{base_prefix}<tool_call>"

        elif constraint_mode == "specified":

```



```

        # Model must call a function from a specific subset
        if not allowed_tools:
            raise ValueError("allowed_tools required for 'specified' mode")

        # Prefill up to the beginning of the function name
        # This constrains to specific tools
        tool_names = [tool for tool in allowed_tools]
        # In practice, you'd prefill the JSON structure
        return f"{base_prefix}<tool_call>{{\"name\": \""

    else:
        return base_prefix

def apply_constraint(self, state: str, user_input: str) → Dict:
    """
    Apply constraint based on agent state.
    Returns configuration for response prefilling.
    """
    if state == "must_respond":
        # Agent must reply immediately, no tool calls
        return {
            "mode": "auto",
            "prefix": self.get_response_prefix("auto"),
            "mask_tools": True
        }

    elif state == "can_use_browser":
        # Only browser tools allowed
        browser_tools = [t for t in self.all_tools.keys() if t.startswith("browser_")
        return {
            "mode": "specified",
            "prefix": self.get_response_prefix("specified", browser_tools),
            "allowed_tools": browser_tools,
            "mask_tools": False
        }

```

```

elif state == "must_use_tool":
    # Must use a tool, but any tool is fine
    return {
        "mode": "required",
        "prefix": self.get_response_prefix("required"),
        "mask_tools": False
    }

else:
    # Default: auto mode
    return {
        "mode": "auto",
        "prefix": self.get_response_prefix("auto"),
        "mask_tools": False
    }

# Usage
manager = ToolConstraintManager()

# Agent must respond to user input
config = manager.apply_constraint("must_respond", "User asked a question")
# Framework uses config["prefix"] to prefill response
# config["mask_tools"] = True prevents tool selection

# Agent can use browser tools
config = manager.apply_constraint("can_use_browser", "Need to search web")
# Framework uses config["prefix"] and config["allowed_tools"] to constrain selection

```

Explanation: This advanced example demonstrates response prefilling using the Hermes format. The framework prefills response tokens to constrain the action space without modifying tool definitions. Three modes are shown: Auto (may or may not call tools), Required (must call a tool), and Specified (must call from a subset).

Framework-Specific Examples

CUSTOM LOGIT MASKING IMPLEMENTATION

```

from typing import Dict, List, Set
import torch

class LogitMasker:
    """Implements logit masking for tool constraints."""

    def __init__(self, tokenizer, tool_name_tokens: Dict[str, List[int]]):
        """
        Initialize with tokenizer and tool name token mappings.

        Args:
            tokenizer: Tokenizer to convert tool names to token IDs
            tool_name_tokens: Dict mapping tool names to their token ID sequences
        """
        self.tokenizer = tokenizer
        self.tool_name_tokens = tool_name_tokens
        self.all_tool_token_ids = set()
        for tokens in tool_name_tokens.values():
            self.all_tool_token_ids.update(tokens)

    def create_mask(self, allowed_tools: Set[str], vocab_size: int) → torch.Tensor:
        """
        Create a logit mask that allows only specified tools.

        Args:
            allowed_tools: Set of tool names that are allowed
            vocab_size: Size of vocabulary

        Returns:
            Binary mask tensor (1 = allowed, 0 = masked)
        """
        mask = torch.zeros(vocab_size, dtype=torch.bool)

```

```

        # Allow all tokens by default
        mask.fill_(True)

        # Mask all tool-related tokens
        mask[list(self.all_tool_token_ids)] = False

        # Unmask allowed tools
        for tool_name in allowed_tools:
            if tool_name in self.tool_name_tokens:
                tool_tokens = self.tool_name_tokens[tool_name]
                mask[tool_tokens] = True

        return mask

def apply_mask_to_logits(self, logits: torch.Tensor, allowed_tools: Set[str]) → torch.Tensor:
    """Apply masking to logits during decoding."""
    mask = self.create_mask(allowed_tools, logits.shape[-1])

    # Set masked logits to very negative value
    masked_logits = logits.clone()
    masked_logits[~mask] = float('-inf')

    return masked_logits

# Usage
# In practice, this would be integrated into the inference framework
# The mask would be applied during each decoding step

```

STATE MACHINE FOR TOOL AVAILABILITY

```

from typing import Dict, Set, Optional
from enum import Enum
from dataclasses import dataclass

class AgentState(Enum):
    INITIALIZING = "initializing"
    RESEARCHING = "researching"
    EXECUTING = "executing"
    RESPONDING = "responding"

@dataclass
class ToolConstraint:
    """Defines tool constraints for a state."""
    allowed_prefixes: List[str]
    blocked_tools: Set[str]
    require_tool: bool = False

class ContextAwareToolStateMachine:
    """Manages tool availability using a state machine."""

    def __init__(self):
        # Define state-specific constraints
        self.state_constraints = {
            AgentState.INITIALIZING: ToolConstraint(
                allowed_prefixes=[],
                blocked_tools=set(),
                require_tool=False
            ),
            AgentState.RESEARCHING: ToolConstraint(
                allowed_prefixes=["browser_", "search_"],
                blocked_tools={"shell_execute", "query_database"},
                require_tool=False
            ),
            AgentState.EXECUTING: ToolConstraint(

```

```

        allowed_prefixes=["shell_", "file_"],
        blocked_tools={"browser_search", "browser_click"},
        require_tool=False
    ),
    AgentState.RESPONDING: ToolConstraint(
        allowed_prefixes=[],
        blocked_tools=set(), # All tools blocked
        require_tool=False
    )
}

self.current_state = AgentState.INITIALIZING

def get_constraint_for_state(self, state: AgentState) → ToolConstraint:
    """Get tool constraint configuration for a state."""
    return self.state_constraints.get(state, self.state_constraints[AgentState.INIT])

def transition_to(self, new_state: AgentState) → ToolConstraint:
    """Transition to new state and return constraint."""
    self.current_state = new_state
    return self.get_constraint_for_state(new_state)

def get_masking_config(self, state: Optional[AgentState] = None) → Dict:
    """Get masking configuration for current or specified state."""
    target_state = state or self.current_state
    constraint = self.get_constraint_for_state(target_state)

    return {
        "allowed_prefixes": constraint.allowed_prefixes,
        "blocked_tools": constraint.blocked_tools,
        "require_tool": constraint.require_tool,
        "mask_all": len(constraint.allowed_prefixes) == 0 and len(constraint.blocked_tools) > 0
    }

# Usage
state_machine = ContextAwareToolStateMachine()

```

```
# Agent starts researching
constraint = state_machine.transition_to(AgentState.RESEARCHING)
masking_config = state_machine.get_masking_config()
# Framework uses masking_config to constrain tool selection

# Agent must respond
constraint = state_machine.transition_to(AgentState.RESPONDING)
masking_config = state_machine.get_masking_config()
# All tools masked, agent must respond directly
```

Key Takeaways

- **Stability is Crucial:** Dynamically altering the tool set during an iteration can confuse the model because previous actions and observations still refer to the old tools. This leads to schema violations or hallucinated actions without constrained decoding.
- **KV-Cache Optimization:** Since tool definitions usually live near the front of the context, stability ensures the KV-Cache remains valid, directly improving latency and cost. A single-token difference in the prompt prefix can invalidate the cache, leading to up to 10× cost increases.
- **The Solution:** Rather than physical removal, use programmatic constraints (logit masking or response prefilling) to achieve constrained tool use. This maintains context stability while still guiding the agent's action space.
- **Prefix Design:** Designing tool names with consistent prefixes (e.g., `browser_`, `shell_`) simplifies masking logic and allows easy enforcement of tool groups without stateful logits processors.
- **Response Prefilling:** Most model providers and inference frameworks support response prefilling, which allows constraining the action space without modifying tool definitions. Three modes: Auto (may or may not call), Required (must call), and Specified (must call from subset).
- **Best Practice:** Keep the tool set fixed during a single problem-solving episode. Use a context-aware state machine to manage tool availability based on current agent state, applying constraints through masking rather than definition changes.
- **Common Pitfall:** Attempting to dynamically add or remove tool definitions mid-run breaks KV-Cache and confuses the model. Always use masking instead of physical removal.

Related Patterns

This pattern works well with:

- **Stable, Append-Only Context:** This pattern is a direct engineering strategy to maintain context stability, which is the core principle of append-only context. Keeping tool definitions stable preserves KV-Cache efficiency.
- **Explicit Tool Definitions and Interfaces:** Clear tool definitions aid in implementing masking strategies, as consistently named tools (e.g., using prefixes like `browser_`) simplify the masking logic.
- **Memory Management:** Stable tool definitions align with KV-Cache optimization strategies in memory management, ensuring efficient context window usage.

This pattern is often combined with:

- **Tool Use (Function Calling):** This pattern is an advanced implementation method for reliable and safe tool invocation, ensuring tools are used correctly while maintaining performance.
- **Routing:** Routing logic can determine which tools should be masked based on context, user permissions, or workflow stage.
- **State Management:** State machines or state management systems determine when to apply constraints, transitioning between states that allow different tool groups.

References

- Manus AI: Context Engineering for AI Agents - Lessons from Building Manus
- KV-Cache Optimization: Maintaining stable context prefixes for efficient inference
- Response Prefilling: Constraining LLM outputs through token prefilling
- Hermes Format: Function calling format from NousResearch
- Model Context Protocol (MCP): Standardized protocol for tool discovery and access

PART IV

Reasoning & Planning

Enabling agents to plan and reason effectively

Reasoning Techniques

Core algorithms for agency: Chain-of-Thought, ReAct Loops, and Tree-of-Thought strategies.

Module ID: module-19

REASONING TECHNIQUES

Introduction

*Reasoning is the foundational capability that distinguishes agentic systems from simple content generators. While traditional LLMs produce text based on patterns learned during training, agentic systems must **think** before they act—simulating problem-solving internally, evaluating options, and making strategic decisions to achieve goals.*

This chapter provides an overview of reasoning techniques used in agentic systems. We'll explore how agents structure their thought processes, the different approaches available, and when each technique is most appropriate. For detailed implementation patterns, see the specific pattern modules referenced throughout this chapter.

The Importance of Reasoning in Agentic Systems

The quality of an agent's reasoning directly determines its success. A flawed thought process leads to failed actions, wasted resources, and costly error loops. Effective reasoning enables agents to:

- **Improve Accuracy:** Explicit step-by-step reasoning significantly reduces errors and hallucinations, particularly in mathematical, logical, or multi-step tasks
- **Enable Transparency:** By documenting their thought process, agents provide visibility into decision-making, essential for debugging and building user trust

- **Support Error Correction:** Structured reasoning allows agents to analyze failures, understand what went wrong, and adapt their approach
- **Enable Strategic Planning:** Advanced reasoning techniques allow agents to explore multiple paths before committing to a solution

Core Reasoning Approaches

Agentic systems employ several reasoning techniques, each suited to different problem types and complexity levels:

Chain-of-Thought (CoT)

Chain-of-Thought is the simplest and most widely adopted reasoning technique. It works by explicitly instructing the model to generate intermediate reasoning steps before producing a final answer.

How it works: The agent is prompted to “think step-by-step” or show its work. This forces the model to allocate tokens to reasoning, which significantly improves accuracy on complex problems.

When to use: CoT is ideal for tasks requiring explicit step-by-step thinking, such as mathematical problems, logical deductions, or multi-step calculations. It’s simple to implement and adds minimal overhead.

Example scenario: An agent solving “If $5x + 3 = 23$, what is x ?” would show:

Thought: I need to solve for x . First, I'll subtract 3 from both sides: $5x = 20$. Then d
Answer: $x = 4$

ReAct (Reason + Act)

ReAct is the foundational pattern for adaptive, tool-using agents. It creates a self-correcting loop that intertwines reasoning, action, and observation.

How it works: The agent cycles through three phases:

1. **Thought:** The agent reasons about the current state, available tools, and next action
2. **Action:** The agent generates structured code/JSON to call a tool

3. **Observation:** The system executes the action and returns results

4. **Loop:** The agent returns to Thought, incorporating new observations

This creates adaptive behavior where errors in tool execution are analyzed and corrected in subsequent thoughts.

When to use: ReAct is essential for any agent that needs to interact with tools, adapt to changing conditions, or correct errors autonomously. It's the core loop for most agentic systems.

Example scenario: An agent tasked with "Find the weather in Paris and London" would:

```
Thought: I need weather data for two cities. I'll use the weather API.
Action: get_weather("Paris")
Observation: Paris: 15°C, cloudy
Thought: Now I need London's weather.
Action: get_weather("London")
Observation: London: 12°C, rainy
Thought: I have both pieces of information. I can provide the answer.
Action: FINAL_ANSWER
```

Tree-of-Thoughts (ToT)

Tree-of-Thoughts is an advanced technique that explores multiple potential solution paths before committing to one. It's computationally expensive but essential for high-stakes problems.

How it works: At decision points, the agent generates multiple thought branches. Each branch is evaluated using heuristics (often another LLM call) that score likelihood of success. Lower-scoring branches are pruned, and the agent commits to the optimal path.

When to use: ToT is valuable for strategic problems where exploring multiple approaches significantly improves outcomes, such as complex planning, architecture design, or high-stakes decision-making. The cost is justified when the value of finding the optimal solution outweighs the computational expense.

Example scenario: An agent designing a system architecture might:

1. Generate three different architectural approaches
2. Evaluate each for scalability, cost, and complexity

3. Prune the lowest-scoring approach
4. Deepen reasoning on the remaining two
5. Select the optimal path

Self-Consistency

Self-Consistency improves accuracy by generating multiple independent reasoning chains and using voting to select the most frequent answer.

How it works: The agent generates several independent solutions to the same problem, then selects the answer that appears most frequently across all chains.

When to use: Self-Consistency is useful when accuracy is critical and the computational cost of multiple reasoning chains is acceptable. It's particularly effective for problems with clear right/wrong answers.

The Reasoning Process in Practice

Effective reasoning in agentic systems relies on disciplined state management. The agent must:

1. **Read Current State:** Access the latest user message, current plan, and recent observations
2. **Generate Thoughts:** Use reasoning to determine the next action based on available information
3. **Execute Actions:** Call tools or provide answers based on reasoning
4. **Update State:** Incorporate new observations for the next reasoning cycle

This process requires careful orchestration of context, memory, and tool execution. The specific patterns for implementing this—such as the ReAct pattern, Planning pattern, and Memory Management techniques—are covered in detail in their respective modules.

Choosing the Right Reasoning Technique

The choice of reasoning technique depends on several factors:

Problem Complexity: Simple problems may not need explicit reasoning, while complex multi-step problems benefit significantly from structured thinking.

Transparency Requirements: If you need to understand or debug the agent’s decision-making, explicit reasoning (CoT, ReAct) is essential.

Cost Constraints: Reasoning increases token usage and latency. Balance the benefits of explicit reasoning against performance requirements.

Error Correction Needs: If the agent must adapt to failures or changing conditions, ReAct’s observation-analysis cycle is crucial.

Strategic Importance: For high-stakes decisions where exploring multiple paths is valuable, Tree-of-Thoughts may be justified despite higher costs.

Integration with Other Capabilities

Reasoning techniques work in conjunction with other agent capabilities:

- **Tool Use:** Reasoning determines which tools to use and how to use them effectively
- **Planning:** Reasoning techniques are used to generate, evaluate, and refine plans
- **Memory Management:** Reasoning utilizes memory (context, state, external) to make informed decisions
- **Reflection:** Reasoning can be evaluated and refined through reflection patterns
- **Goal Setting:** Reasoning works towards achieving set goals, using goal information to guide decisions

Key Insights

1. **Reasoning is not optional for agents:** While simple LLMs can generate text without explicit reasoning, agents operating in dynamic environments require structured thinking to succeed.
2. **Transparency enables trust:** Users and developers need visibility into agent reasoning to trust and debug agentic systems. Explicit reasoning provides this transparency.
3. **Cost-performance trade-off:** More sophisticated reasoning (ToT, Self-Consistency) improves accuracy but increases computational cost. Choose based on problem requirements.
4. **Reasoning enables adaptation:** The ability to reason about observations allows agents to correct errors and adapt to changing conditions autonomously.

5. State management is critical: Effective reasoning requires careful management of context, observations, and plans. Poor state management undermines even sophisticated reasoning techniques.

Next Steps

This chapter provided a high-level overview of reasoning techniques. For detailed implementation guidance, see:

- **Pattern: Planning** - How agents create structured plans before execution
- **Pattern: Reflection** - How agents evaluate and refine their reasoning
- **Pattern: Tool Use & Execution** - How reasoning guides tool selection and usage
- **Memory Management** - How agents maintain context for effective reasoning

The reasoning techniques described here form the foundation for autonomous agent behavior. Understanding when and how to apply them is essential for building effective agentic systems.

Pattern: Planning

Enabling agents to create structured plans before execution, breaking down complex goals into actionable steps.

Module ID: module-8

PLANNING

Motivation

When planning a trip, you break it down into steps: choose destination, book flights, reserve hotels, create an itinerary, pack. You consider dependencies (can't pack before deciding what to bring) and adapt when obstacles arise (flight cancelled, hotel unavailable). Planning in agents works the same way: taking a high-level goal and autonomously creating a structured sequence of actions, then adapting as conditions change.

Pattern Overview

What it is: Planning is the ability for an agent to formulate a sequence of actions to move from an initial state towards a goal state, breaking down complex tasks into smaller, manageable steps.

When to use: Use planning when you need to delegate a complex goal where the “how” needs to be discovered dynamically, rather than following a predetermined workflow.

Why it matters: Planning enables agents to move beyond reactive behavior to goal-oriented, strategic problem-solving. It transforms high-level objectives into structured, executable sequences while maintaining adaptability to changing conditions and obstacles.

Intelligent behavior often involves more than just reacting to immediate input. It requires foresight, breaking down complex tasks into smaller steps, and strategizing how to achieve a desired outcome. At its core, planning allows an agent to formulate a sequence of actions to move from an initial state towards a goal state.

In the context of AI, a planning agent is like a specialist to whom you delegate a complex goal. When you ask it to “organize a team offsite,” you’re defining the what—the objective and its constraints—but not the how. The agent’s core task is to autonomously chart a course to that goal. It must first understand the initial state (e.g., budget, number of participants, desired dates) and the goal state (a successfully booked offsite), and then discover the optimal sequence of actions to connect them. The plan is not known in advance; it is created in response to the request.

A hallmark of this process is adaptability. An initial plan is merely a starting point, not a rigid script. The agent’s real power is its ability to incorporate new information and steer around obstacles. For instance, if the preferred venue becomes unavailable or a chosen caterer is fully booked, a capable agent doesn’t simply fail. It adapts. It registers the new constraint, re-evaluates its options, and formulates a new plan, perhaps by suggesting alternative venues or dates.

However, it is crucial to recognize the trade-off between flexibility and predictability. Dynamic planning is a specific tool, not a universal solution. When a problem’s solution is already well-understood and repeatable, constraining the agent to a predetermined, fixed workflow is more effective. This approach limits the agent’s autonomy to reduce uncertainty and the risk of unpredictable behavior, guaranteeing a reliable and consistent outcome. Therefore, the decision to use a planning agent versus a simple task-execution agent hinges on a single question: does the “how” need to be discovered, or is it already known?

Key Concepts

- **Goal Decomposition:** Planning breaks high-level goals into discrete, executable sub-tasks that can be sequenced logically.
- **State Space Navigation:** The agent must understand initial state, goal state, and the actions that transition between states.
- **Adaptive Replanning:** Plans are not rigid; agents must adapt when obstacles arise or new information becomes available.
- **Dependency Management:** Planning must account for task dependencies, ensuring prerequisites are completed before dependent tasks.

How It Works

Planning works through a structured process: (1) Goal Analysis—the agent understands the desired outcome and constraints, (2) State Assessment—it evaluates the current state and available resources, (3) Plan Generation—it formulates a sequence of actions to bridge the gap between current and goal states, (4)

Plan Execution—it executes the plan step by step, and (5) Monitoring and Adaptation—it monitors progress and adapts the plan when obstacles arise or conditions change.

The planning process can be explicit, where the agent generates a detailed plan document before execution, or implicit, where planning happens dynamically during execution. Explicit planning is useful for complex, multi-step tasks where the full sequence needs to be understood upfront. Implicit planning is more reactive, generating the next step based on current state without a full plan document.

Frameworks support planning through various mechanisms. CrewAI enables agents to create plans as part of their task execution. Google DeepResearch demonstrates planning through multi-step research plans that are generated, reviewed, and executed iteratively. LangGraph and LangChain support planning through state management and conditional workflows that can represent planned sequences.

When to Use This Pattern

✅ Use this pattern when:

- **Complex, multi-step goals:** The task requires a sequence of interdependent actions that must be discovered and coordinated.
- **Dynamic environments:** Conditions change during execution, requiring plan adaptation.
- **Goal discovery needed:** The “how” to achieve the goal is not predetermined and must be discovered.
- **Long-horizon tasks:** The task spans multiple steps where intermediate planning improves outcomes.
- **Resource coordination:** The task requires coordinating multiple resources, tools, or agents in a specific sequence.

❌ Avoid this pattern when:

- **Fixed workflows suffice:** The solution path is well-understood and can be hardcoded as a workflow.
- **Simple, single-step tasks:** The task can be completed in one or two steps without needing a plan.
- **Predictability is critical:** You need guaranteed, consistent behavior that fixed workflows provide.
- **Real-time constraints:** The overhead of planning adds unacceptable latency for time-sensitive tasks.

Decision Guidelines

Use planning when the benefits of adaptability and goal discovery outweigh the costs of increased complexity and potential unpredictability. Consider: the complexity of the goal (more complex = more benefit from planning), the variability of the environment (more variable = more need for adaptive planning), and whether the solution path is known (unknown = use planning, known = use workflow). Be aware that planning adds latency and cost, and can introduce unpredictability. For critical systems requiring guaranteed behavior, consider hybrid approaches that combine planning with fixed workflow fallbacks.

Practical Applications & Use Cases

Planning is a core computational process in autonomous systems, enabling agents to synthesize sequences of actions to achieve specified goals.

- **Business Process Automation:** Decompose complex workflows like employee onboarding into sequenced sub-tasks with dependencies.
- **Robotics and Navigation:** Generate paths or action sequences to transition from initial to goal state while optimizing for metrics.
- **Content Generation:** Formulate plans for complex outputs like research reports with distinct phases for gathering, summarizing, and structuring.
- **Customer Support:** Create systematic plans for multi-step problem resolution including diagnosis, solution implementation, and escalation.
- **Project Management:** Break down high-level projects into task sequences with dependencies and resource allocation.

Implementation

Prerequisites

```
pip install crewai langchain langchain-openai  
# or  
pip install google-adk
```

Basic Example

```
from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4-turbo")

# Planning agent
planner_agent = Agent(
    role='Project Planner',
    goal='Create and execute plans for complex tasks',
    backstory='You are an expert at breaking down complex goals into actionable plans.'
    llm=llm,
    verbose=True
)

# Task with planning requirement
task = Task(
    description="""1. Create a detailed plan for organizing a team offsite.
    2. Execute the plan step by step.""",
    expected_output="A plan and execution report",
    agent=planner_agent
)

# Execute
crew = Crew(
    agents=[planner_agent],
    tasks=[task],
    process=Process.sequential
)

result = crew.kickoff()
print(result)
```

Explanation: This example demonstrates planning using CrewAI. The agent is instructed to first create a plan, then execute it. The planning happens as part of the agent's task execution, where it breaks down the complex goal into manageable steps before proceeding.

Advanced Example

```

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from typing import List, Dict
import json

llm = ChatOpenAI(model="gpt-4o", temperature=0)

class PlanningAgent:
    def __init__(self):
        self.llm = llm
        self.plan_history = []

    def generate_plan(self, goal: str, constraints: Dict) → Dict:
        """Generate an initial plan for a goal."""
        prompt = ChatPromptTemplate.from_messages([
            ("system", """You are a planning expert. Create a detailed plan.
            Return JSON with:
            - "steps": list of {"id": int, "action": str, "dependencies": [int]}
            - "estimated_duration": str
            - "resources_needed": list"""),
            ("user", f"Goal: {goal}\nConstraints: {constraints}")
        ])

        response = self.llm.invoke(prompt.format_messages())
        plan = json.loads(response.content)
        self.plan_history.append({"goal": goal, "plan": plan})
        return plan

    def adapt_plan(self, current_plan: Dict, obstacle: str) → Dict:
        """Adapt plan when obstacles arise."""
        prompt = ChatPromptTemplate.from_messages([
            ("system", """Adapt the plan to handle obstacles.
            Return updated plan JSON."""),

```

```

        ("user", f"Current plan: {json.dumps(current_plan)}\nObstacle: {obstacle}")
    ])

    response = self.llm.invoke(prompt.format_messages())
    adapted_plan = json.loads(response.content)
    return adapted_plan

def execute_step(self, step: Dict, context: Dict) → Dict:
    """Execute a single plan step."""
    # Simulate step execution
    return {
        "step_id": step["id"],
        "status": "completed",
        "result": f"Executed: {step['action']}",
        "context": context
    }

# Usage
agent = PlanningAgent()
plan = agent.generate_plan(
    goal="Organize team offsite for 30 people",
    constraints={"budget": 10000, "location": "Lisbon", "dates": "Q2 2025"}
)

print("Generated Plan:")
print(json.dumps(plan, indent=2))

# Adapt if obstacle arises
if obstacle_detected:
    plan = agent.adapt_plan(plan, "Preferred venue unavailable")

```

Explanation: This advanced example implements a `PlanningAgent` class with plan generation, adaptation, and execution capabilities. It demonstrates structured plan representation with dependencies, obstacle handling through adaptive replanning, and plan history tracking. This shows production-ready planning with JSON-structured plans and adaptation mechanisms.

Framework-Specific Examples

CREWAI

```
from crewai import Agent, Task, Crew

planner = Agent(
    role='Strategic Planner',
    goal='Create comprehensive plans',
    backstory='Expert in strategic planning',
    verbose=True
)

planning_task = Task(
    description="""Create a detailed plan for: {goal}
    Then execute the plan.""",
    agent=planner
)

crew = Crew(agents=[planner], tasks=[planning_task])
result = crew.kickoff(inputs={"goal": "Launch new product"})
```

GOOGLE DEEPRESEARCH

```
# DeepResearch demonstrates planning through multi-step research plans
# The system:
# 1. Deconstructs user prompt into research plan
# 2. Presents plan for user review/modification
# 3. Executes iterative search-and-analysis loop
# 4. Dynamically formulates queries based on gathered information
# 5. Consolidates findings into structured summary

# Planning is implicit in the research pipeline structure
```


Key Takeaways

- **Core Concept:** Planning enables agents to formulate sequences of actions to achieve goals, breaking complex tasks into manageable steps.
- **Best Practice:** Use explicit planning for complex, long-horizon tasks; use implicit planning for reactive, adaptive scenarios.
- **Common Pitfall:** Over-planning simple tasks adds unnecessary complexity; use fixed workflows when the solution path is known.
- **Performance Note:** Planning adds latency and cost but improves outcomes for complex, multi-step tasks requiring coordination.

Related Patterns

This pattern works well with:

- **Goal Setting and Monitoring** - Goals provide the target for planning, monitoring tracks plan execution
- **Reflection** - Plans can be evaluated and refined through reflection
- **Routing** - Planning can determine which routes to take in a workflow

This pattern is often combined with:

- **Tool Use** - Plans specify which tools to use and in what sequence
- **Multi-Agent** - Planning can coordinate multiple agents working on different plan steps

References

- CrewAI Documentation: <https://docs.crewai.com/>
- Google DeepResearch: <https://deepresearch.google/>
- Planning in AI Systems: https://en.wikipedia.org/wiki/Automated_planning_and_scheduling

Pattern: Prioritization

Enabling agents to assess and rank tasks, objectives, or actions based on significance, urgency, dependencies, and criteria.

Module ID: module-22

PRIORITIZATION

Motivation

Facing a long to-do list, you assess what's urgent, what's important, and what depends on other tasks. You might tackle urgent deadlines first, then important but less urgent tasks, while considering what blocks other work. Doctors triage patients by severity. Project managers prioritize features by impact. Prioritization in agents mirrors this: evaluating tasks against criteria like significance, urgency, and dependencies to determine the optimal order of execution.

Pattern Overview

What it is: Prioritization is a pattern that enables agents to assess and rank tasks, objectives, or actions based on their significance, urgency, dependencies, and established criteria. This ensures agents concentrate efforts on the most critical tasks, resulting in enhanced effectiveness and goal alignment.

When to use: Use the Prioritization pattern when an agentic system must autonomously manage multiple, often conflicting, tasks or goals under resource constraints to operate effectively in a dynamic environment.

Why it matters: In complex, dynamic environments, agents frequently encounter numerous potential actions, conflicting goals, and limited resources. Without a defined process for determining the subsequent action, agents may experience reduced efficiency, operational delays, or failures to achieve key objectives. Prioritization addresses this by enabling informed decision-making when addressing multiple demands, prioritizing vital or urgent activities over less critical ones.

The fundamental aspects of agent prioritization typically involve several elements. First, criteria definition establishes the rules or metrics for task evaluation, including urgency, importance, dependencies, resource availability, cost/benefit analysis, and user preferences. Second, task evaluation involves assessing each potential task against these defined criteria, utilizing methods ranging from simple rules to complex scoring or reasoning by LLMs. Third, scheduling or selection logic refers to the algorithm that, based on the evaluations, selects the optimal next action or task sequence. Finally, dynamic re-prioritization allows the agent to modify priorities as circumstances change.

Key Concepts

- **Criteria Definition:** Establish rules or metrics for task evaluation (urgency, importance, dependencies, resource availability, cost/benefit, user preferences).
- **Task Evaluation:** Assess each potential task against defined criteria using methods from simple rules to complex LLM-based reasoning.
- **Scheduling Logic:** Algorithm that selects the optimal next action or task sequence based on evaluations, potentially utilizing queues or planning components.
- **Dynamic Re-prioritization:** Modify priorities as circumstances change (new critical events, approaching deadlines), ensuring adaptability and responsiveness.
- **Multi-Level Prioritization:** Prioritization can occur at various levels—selecting overarching objectives (high-level goal prioritization), ordering steps within a plan (sub-task prioritization), or choosing the next immediate action (action selection).

How It Works

Prioritization operates through a systematic process. First, the agent defines evaluation criteria relevant to its domain and objectives. These criteria might include urgency (time sensitivity), importance (impact on primary objective), dependencies (prerequisites for other tasks), resource availability (readiness of necessary tools or information), cost/benefit analysis (effort versus expected outcome), and user preferences for personalized agents.

Second, the agent evaluates each potential task against these criteria. This evaluation can be rule-based (simple if-else logic), scoring-based (assigning numerical scores to each criterion and aggregating), or LLM-based (using language models to reason about task priority).

Third, based on the evaluations, a scheduling algorithm selects the optimal next action. This might involve sorting tasks by priority score, using a priority queue, or integrating with a planning component that considers task dependencies and resource constraints.

Finally, the system implements dynamic re-prioritization, continuously monitoring the environment and adjusting priorities when new information arrives, deadlines approach, or circumstances change.

When to Use This Pattern

✓ Use this pattern when:

- **Multiple competing tasks:** Agents face numerous potential actions with limited resources to execute them all.
- **Conflicting goals:** Multiple objectives exist that cannot all be pursued simultaneously.
- **Resource constraints:** Time, computational resources, or other constraints require selective task execution.
- **Dynamic environments:** Circumstances change frequently, requiring adaptive prioritization.
- **User preferences matter:** Personalized agents need to consider user-defined importance and preferences.
- **Dependency management:** Tasks have dependencies that affect their execution order.

✗ Avoid this pattern when:

- **Single task execution:** Only one task or action is available at a time.
- **Fixed sequence:** Tasks must be executed in a predetermined, unchangeable order.
- **Simple scenarios:** The order of task execution is trivial or doesn't impact outcomes.
- **Real-time constraints:** Extremely low-latency requirements where prioritization overhead is prohibitive.

Decision Guidelines

Choose this pattern when the benefits of intelligent task ordering outweigh the added complexity. Consider the prioritization method: rule-based prioritization is fast and deterministic but less flexible; scoring-based prioritization offers good balance but requires careful weight tuning; LLM-based prioritization is most flexible but adds latency and cost. The choice depends on your accuracy requirements, latency constraints, and the complexity of the prioritization decision.

Practical Applications & Use Cases

Prioritization is essential for building efficient, goal-aligned agentic systems that can manage complexity and resource constraints. Common applications include task management, resource allocation, and dynamic decision-making.

- **Automated Customer Support:** Agents prioritize urgent requests (like system outage reports) over routine matters (such as password resets), and may give preferential treatment to high-value customers.
- **Cloud Computing:** AI manages and schedules resources by prioritizing allocation to critical applications during peak demand, while relegating less urgent batch jobs to off-peak hours to optimize costs.
- **Autonomous Driving Systems:** Continuously prioritize actions to ensure safety and efficiency. For example, braking to avoid a collision takes precedence over maintaining lane discipline or optimizing fuel efficiency.
- **Financial Trading:** Bots prioritize trades by analyzing factors like market conditions, risk tolerance, profit margins, and real-time news, enabling prompt execution of high-priority transactions.
- **Project Management:** AI agents prioritize tasks on a project board based on deadlines, dependencies, team availability, and strategic importance.
- **Cybersecurity:** Agents monitoring network traffic prioritize alerts by assessing threat severity, potential impact, and asset criticality, ensuring immediate responses to the most dangerous threats.
- **Personal Assistant AIs:** Utilize prioritization to manage daily lives, organizing calendar events, reminders, and notifications according to user-defined importance, upcoming deadlines, and current context.

Implementation

Prerequisites

```
pip install langchain langchain-openai python-dotenv pydantic
```

Basic Example

```

from typing import List, Dict
from enum import Enum

class Priority(Enum):
    P0 = 0 # Critical/Urgent
    P1 = 1 # High
    P2 = 2 # Medium
    P3 = 3 # Low

class Task:
    def __init__(self, id: str, description: str, urgency: int = 0,
                  importance: int = 0, deadline: float = None):
        self.id = id
        self.description = description
        self.urgency = urgency # 0-10 scale
        self.importance = importance # 0-10 scale
        self.deadline = deadline # Unix timestamp
        self.priority_score = self._calculate_priority()

    def _calculate_priority(self) → float:
        """Calculate priority score based on urgency and importance."""
        # Weighted combination
        base_score = (self.urgency * 0.6) + (self.importance * 0.4)

        # Boost for approaching deadlines
        if self.deadline:
            import time
            time_until_deadline = self.deadline - time.time()
            if time_until_deadline < 3600: # Less than 1 hour
                base_score += 5.0
            elif time_until_deadline < 86400: # Less than 1 day
                base_score += 2.0

```

```

        return base_score

def get_priority_level(self) → Priority:
    """Convert score to priority level."""
    if self.priority_score ≥ 8:
        return Priority.P0
    elif self.priority_score ≥ 5:
        return Priority.P1
    elif self.priority_score ≥ 2:
        return Priority.P2
    else:
        return Priority.P3

class TaskPrioritizer:
    """Simple task prioritization system."""

    def __init__(self):
        self.tasks: List[Task] = []

    def add_task(self, task: Task):
        """Add a task to the prioritizer."""
        self.tasks.append(task)

    def get_prioritized_tasks(self) → List[Task]:
        """Get tasks sorted by priority (highest first)."""
        return sorted(self.tasks, key=lambda t: t.priority_score, reverse=True)

    def get_next_task(self) → Task:
        """Get the highest priority task."""
        prioritized = self.get_prioritized_tasks()
        return prioritized[0] if prioritized else None

# Example usage
prioritizer = TaskPrioritizer()

# Add tasks with different priorities

```



```
prioritizer.add_task(Task("T1", "Fix critical security bug", urgency=10, importance=10))
prioritizer.add_task(Task("T2", "Update documentation", urgency=2, importance=3))
prioritizer.add_task(Task("T3", "Review pull request", urgency=5, importance=4))

# Get prioritized list
for task in prioritizer.get_prioritized_tasks():
    print(f"{task.id}: {task.description} - Priority: {task.get_priority_level().name}")

# Get next task to work on
next_task = prioritizer.get_next_task()
print(f"\nNext task: {next_task.description}")
```

Explanation: This basic example demonstrates a simple prioritization system that calculates priority scores based on urgency and importance, with additional boosts for approaching deadlines. Tasks are sorted by their priority scores, and the system can return the highest priority task for execution.

Advanced Example: LLM-Based Prioritization

```
import os

from typing import List, Optional, Dict
from dotenv import load_dotenv
from pydantic import BaseModel, Field
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.tools import Tool
from langchain_openai import ChatOpenAI
from langchain.agents import AgentExecutor, create_react_agent
from langchain.memory import ConversationBufferMemory

load_dotenv()

llm = ChatOpenAI(temperature=0.5, model="gpt-4o-mini")

class Task(BaseModel):
    """Represents a single task in the system."""
    id: str
    description: str
    priority: Optional[str] = None # P0, P1, P2
    assigned_to: Optional[str] = None

class TaskManager:
    """In-memory task manager."""
    def __init__(self):
        self.tasks: Dict[str, Task] = {}
        self.next_task_id = 1

    def create_task(self, description: str) → Task:
        """Creates and stores a new task."""
        task_id = f"TASK-{self.next_task_id:03d}"
        new_task = Task(id=task_id, description=description)
        self.tasks[task_id] = new_task
        self.next_task_id += 1
        return new_task
```

```

def update_task(self, task_id: str, **kwargs) → Optional[Task]:
    """Updates a task."""
    task = self.tasks.get(task_id)
    if task:
        update_data = {k: v for k, v in kwargs.items() if v is not None}
        updated_task = task.model_copy(update=update_data)
        self.tasks[task_id] = updated_task
        return updated_task
    return None

def list_all_tasks(self) → str:
    """Lists all tasks currently in the system."""
    if not self.tasks:
        return "No tasks in the system."
    task_strings = []
    for task in self.tasks.values():
        task_strings.append(
            f"ID: {task.id}, Desc: '{task.description}', "
            f"Priority: {task.priority or 'N/A'},"
            f"Assigned To: {task.assigned_to or 'N/A'}"
        )
    return "Current Tasks:\n" + "\n".join(task_strings)

task_manager = TaskManager()

# Tools for the Project Manager Agent
class CreateTaskArgs(BaseModel):
    description: str = Field(description="A detailed description of the task.")

class PriorityArgs(BaseModel):
    task_id: str = Field(description="The ID of the task to update, e.g., 'TASK-001'.")
    priority: str = Field(description="The priority to set. Must be one of: 'P0', 'P1',")

def create_new_task_tool(description: str) → str:
    """Creates a new project task with the given description."""

```

```

    task = task_manager.create_task(description)
    return f"Created task {task.id}: '{task.description}'."

def assign_priority_to_task_tool(task_id: str, priority: str) → str:
    """Assigns a priority (P0, P1, P2) to a given task ID."""
    if priority not in ["P0", "P1", "P2"]:
        return "Invalid priority. Must be P0, P1, or P2."
    task = task_manager.update_task(task_id, priority=priority)
    return f"Assigned priority {priority} to task {task.id}." if task else f"Task {task

pm_tools = [
    Tool(
        name="create_new_task",
        func=create_new_task_tool,
        description="Use this first to create a new task and get its ID.",
        args_schema=CreateTaskArgs
    ),
    Tool(
        name="assign_priority_to_task",
        func=assign_priority_to_task_tool,
        description="Use this to assign a priority to a task after it has been created.",
        args_schema=PriorityArgs
    ),
    Tool(
        name="list_all_tasks",
        func=task_manager.list_all_tasks,
        description="Use this to list all current tasks and their status."
    ),
]

# Project Manager Agent
pm_prompt_template = ChatPromptTemplate.from_messages([
    ("system", """You are a focused Project Manager LLM agent. Your goal is to manage p
When you receive a new task request, follow these steps:
1. First, create the task with the given description using the `create_new_task` tool.
2. Next, analyze the user's request to determine priority:

```

```

- If urgent/critical/ASAP mentioned → P0
- If important but not urgent → P1
- Otherwise → P2

3. Assign the priority using `assign_priority_to_task`.
4. Use `list_all_tasks` to show the final state.

Priority levels: P0 (highest), P1 (medium), P2 (lowest)
"""),
    ("placeholder", "{chat_history}"),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

pm_agent = create_react_agent(llm, pm_tools, pm_prompt_template)
pm_agent_executor = AgentExecutor(
    agent=pm_agent,
    tools=pm_tools,
    verbose=True,
    handle_parsing_errors=True,
    memory=ConversationBufferMemory(memory_key="chat_history", return_messages=True)
)

# Example usage
result = pm_agent_executor.invoke({
    "input": "Create a task to implement a new login system. It's urgent and critical."
})

```

Explanation: This advanced example demonstrates LLM-based prioritization where an agent analyzes task descriptions and automatically assigns priorities based on urgency and importance cues. The agent uses tools to create tasks, assign priorities, and manage the task list, making intelligent prioritization decisions based on natural language input.

Key Takeaways

- **Core Concept:** Prioritization enables agents to intelligently rank and select tasks, ensuring efficient resource utilization and goal alignment.
- **Best Practice:** Combine multiple criteria (urgency, importance, dependencies) rather than relying on a single factor for robust prioritization.
- **Common Pitfall:** Over-prioritizing can lead to analysis paralysis; balance thoroughness with decision speed.
- **Performance Note:** Prioritization adds computational overhead; use efficient algorithms and consider caching for repeated similar decisions.

Related Patterns

This pattern works well with:

- **Planning** - Prioritization often informs planning by determining task order
- **Routing** - Prioritization can determine which route or handler to use
- **Resource-Aware Optimization** - Prioritization considers resource constraints when ranking tasks

This pattern is often combined with:

- **Multi-Agent** - Different agents may have different priorities; coordination is needed
- **Goal Setting and Monitoring** - Priorities are often tied to goal achievement
- **Dynamic Re-prioritization** - Priorities change as circumstances evolve

References

- LangChain Agents: <https://python.langchain.com/docs/modules/agents/>
- Task Prioritization in AI Systems: Research on multi-criteria decision making for agents
- Project Management with AI: Best practices for automated task prioritization

PART V

Memory & Context Management

Managing context windows and externalizing memory

Memory Management

How to manage the context window, externalize memory, and use recitation patterns to keep agents on track.

Module ID: module-10

MEMORY MANAGEMENT

Introduction

Memory is what transforms agents from stateless responders into intelligent systems capable of learning, adapting, and maintaining context across interactions. Without memory, agents cannot remember past conversations, learn from experience, or build upon previous work. Effective memory management is one of the most critical aspects of building production-ready agentic systems.

This chapter provides an overview of memory management strategies for agentic systems. We'll explore the different types of memory, the challenges of managing finite context windows, and the techniques used to extend memory beyond immediate context. For specific implementation patterns, see the pattern modules referenced throughout this chapter.

The Two Types of Agent Memory

Agent memory can be broadly categorized into two types, each serving different purposes:

Short-Term Memory (Contextual Memory)

Short-term memory exists within the LLM’s context window—the immediate working memory that contains recent messages, agent replies, tool usage results, and agent reflections from the current interaction.

Characteristics:

- **Limited Capacity:** Context windows have hard limits (typically 32K to 1M+ tokens depending on the model)
- **High Attention:** Information in the context window receives full model attention
- **Ephemeral:** Lost once the session concludes unless explicitly saved
- **Costly:** Processing large contexts consumes tokens and increases latency

Challenges:

- **Token Limits:** Exceeding context limits causes errors or truncation
- **Attention Decay:** Information at the beginning of long contexts may receive less attention (“Lost in the Middle” problem)
- **Cost:** Large contexts are expensive to process repeatedly
- **KV-Cache Efficiency:** Changing prompt prefixes invalidates Key-Value caches, dramatically increasing latency

Long-Term Memory (Persistent Memory)

Long-term memory acts as a repository for information agents need to retain across interactions, tasks, or extended periods. Data is stored outside the agent’s immediate processing environment, typically in databases, knowledge graphs, or vector databases.

Characteristics:

- **Unlimited Capacity:** Can store vast amounts of information
- **Persistent:** Survives across sessions and interactions
- **Query-Based:** Information is retrieved on-demand rather than always present
- **Specialized Storage:** Different storage types optimized for different access patterns

Challenges:

- **Retrieval Overhead:** Querying external memory adds latency

- **Relevance:** Must retrieve the right information at the right time
- **Consistency:** Managing updates and ensuring data consistency
- **Integration:** Seamlessly integrating retrieved information into context

Key Memory Management Challenges

The Context Window Problem

The most fundamental challenge in memory management is the finite context window. As agents operate over longer periods or handle larger tasks, they must:

1. **Manage Token Limits:** Prevent exceeding context window capacity
2. **Maintain Relevance:** Keep the most important information accessible
3. **Preserve Context:** Retain critical information even as context fills
4. **Optimize Performance:** Maximize KV-Cache efficiency to reduce latency

The “Lost in the Middle” Problem

Research shows that LLMs have reduced attention to information in the middle of long contexts. Important information placed at the beginning or end receives more attention than information in the middle. This creates challenges for:

- **Long Conversations:** Critical early context may be “lost” as conversations extend
- **Large Documents:** Key information in the middle of documents may be overlooked
- **Complex Plans:** High-level goals defined early may be forgotten during execution

Cost and Performance Optimization

Memory management directly impacts cost and performance:

- **KV-Cache Efficiency:** Stable prompt prefixes enable cache reuse, reducing latency by up to 10×
- **Token Consumption:** Large contexts consume more tokens, increasing costs
- **Retrieval Overhead:** External memory queries add latency but reduce context size

Memory Management Strategies

Context Window Management

Effective context window management involves:

Stable Prefixes: Keep system prompts and tool definitions stable to maximize KV-Cache efficiency. A single token change in the prefix can invalidate the entire cache.

Append-Only History: Use append-only message structures rather than modifying previous messages. This maintains cache efficiency while allowing context to grow.

Context Compression: When approaching token limits, summarize old messages into compact blocks, maintaining the gist while preserving space for active reasoning.

Selective Inclusion: Only include the most relevant information in context, using external memory for less critical data.

External Memory Systems

For data too large for context windows, external memory systems use an **Offload/Query protocol**:

1. **Offload:** Save raw content to disk or database
2. **Pointer:** Place only a reference in context (e.g., “Content saved to /data/doc1.txt”)
3. **Read on Demand:** Agent queries external memory via specialized tools when needed

This pattern, detailed in the **Pattern: Leverage External Memory (Filesystem as Context)** module, enables agents to handle datasets far exceeding context limits.

The Recitation Pattern

The Recitation Pattern addresses the “Lost in the Middle” problem by maintaining persistent plan files (like `todo.md`) that agents read at every step. This brings high-level goals from the distant past to the immediate present, ensuring agents remain focused on macro-objectives while executing micro-tasks.

This pattern is covered in detail in the **Pattern: Persistent Task List (Recitation)** module.

Context Compression Techniques

When context approaches limits, several compression techniques are available:

- **Summarization:** Condense old messages into summaries
- **Pruning:** Remove less relevant information
- **Chunking:** Break large content into manageable pieces
- **Attention Manipulation:** Use techniques to improve attention to critical information

These techniques are explored in the **Context Compression: Managing the Finite Window** module.

Memory in Different Frameworks

Different frameworks provide different memory management capabilities:

Google ADK:

- **Session Service:** Manages conversation history and temporary state
- **Memory Service:** Provides long-term, searchable knowledge storage
- **State Management:** Structured ways to maintain conversation history

LangChain:

- **Memory Classes:** Various memory types (Buffer, Summary, etc.)
- **Conversation Chains:** Built-in memory management for conversational agents
- **Vector Stores:** Integration with RAG systems for long-term memory

LangGraph:

- **State Management:** Typed state objects that persist across steps
- **Message History:** Append-only message structures optimized for caching

Choosing Memory Strategies

The choice of memory strategy depends on several factors:

Data Volume: Small data fits in context; large data requires external memory

Persistence Requirements: Session-only data uses State management; cross-session data requires MemoryService or databases

Query Patterns: Exact matches work with filesystem storage; semantic queries require vector databases (RAG)

Performance Needs: High-performance systems must optimize KV-Cache efficiency through stable prefixes

Cost Constraints: Large contexts are expensive; external memory with selective retrieval can reduce costs

Integration with Other Capabilities

Memory management integrates with other agent capabilities:

- **Reasoning Techniques:** Memory provides context for reasoning and planning
- **Tool Use:** External memory is accessed via specialized tools
- **Knowledge Retrieval (RAG):** RAG systems provide long-term memory through vector databases
- **Goal Setting and Monitoring:** Memory stores goals and tracks progress across sessions
- **Planning:** Memory maintains plans and tracks execution progress

Key Insights

1. **Memory is not optional:** Agents operating over time or handling complex tasks require sophisticated memory management. Without it, they cannot learn, adapt, or maintain context.
2. **KV-Cache optimization is critical:** Stable prompt prefixes and append-only history can reduce latency by up to 10×. This is one of the most impactful performance optimizations.
3. **Context compression is essential:** Long-running agents must compress context to manage costs and prevent token limit errors. Summarization and pruning are critical techniques.
4. **External memory enables scale:** The Offload/Query pattern allows agents to handle datasets far exceeding context limits, essential for production systems.
5. **The Recitation Pattern prevents goal drift:** Maintaining persistent plans that are read at every step ensures agents stay focused on high-level objectives in long-horizon tasks.

Next Steps

This chapter provided an overview of memory management concepts. For detailed implementation guidance, see:

- **Pattern: Persistent Task List (Recitation)** - Maintaining persistent plans to prevent goal drift
- **Pattern: Leverage External Memory (Filesystem as Context)** - Offloading and retrieving large data
- **Context Compression: Managing the Finite Window** - Techniques for fitting information into finite contexts
- **Pattern: Knowledge Retrieval (RAG)** - Using vector databases for semantic long-term memory

Effective memory management is essential for building production-ready agentic systems. Understanding these concepts and patterns will enable you to build agents that can operate effectively over time and handle complex, long-horizon tasks.

Pattern: Persistent Task List (Recitation)

A context engineering strategy where agents maintain a running plan and continuously append it to context to maintain goal alignment in long-horizon tasks.

Module ID: module-10a

PERSISTENT TASK LIST (RECITATION)

Motivation

When working on a long project, you keep a task list visible—on your desk, whiteboard, or screen—to stay focused on your goals. As you complete tasks, you update the list, but the main objectives remain visible. This prevents you from losing sight of the big picture amid daily details. The Persistent Task List pattern does the same for agents: maintaining a running plan in context to prevent goal drift during long, complex tasks.

Pattern Overview

What it is: A mechanism where the agent maintains a running plan (like a [todo.md](#) file) and continuously appends the updated plan to the end of the context.

When to use: When designing long-horizon AI agents that need to sustain goals and avoid forgetting high-level objectives over multiple steps.

Why it matters: This deliberate “recitation” manipulates the model’s attention, pushing the global plan into the most recent context and mitigating the “lost in the middle” problem that occurs in long contexts.

The Persistent Task List (Recitation) pattern addresses a fundamental challenge in long-horizon agent design: maintaining focus on high-level objectives as the agent executes many intermediate steps. As agents process complex, multi-step tasks, the context window fills with detailed execution history, tool results, and intermediate reasoning. This accumulation can cause the agent’s original goals and plan to become “lost in the middle” of the context, leading to goal drift and reduced effectiveness.

Recitation is a context engineering strategy that actively manages the LLM’s finite attention window. By continuously appending the updated plan to the end of the context, the pattern ensures that high-level objectives remain within the model’s recent attention span. This manipulation of attention is deliberate and strategic—the plan is not just stored but actively recited into the context at each iteration, biasing the model’s focus toward global objectives.

The pattern is particularly effective because LLMs exhibit recency bias, paying more attention to information that appears later in the context. By keeping the plan recent, the agent maintains alignment with its original goals even as it navigates complex, branching workflows with hundreds of steps.

Key Concepts

- **Persistent Task List:** A running plan or to-do list that tracks the agent’s high-level objectives and progress, typically stored in a persistent file or state.
- **Recitation:** The deliberate act of appending the updated plan to the end of the context at each iteration to manipulate attention.
- **Context Engineering:** Strategic manipulation of the context window to bias the model’s attention toward important information.
- **Long-Horizon Tasks:** Complex tasks that span many steps, requiring the agent to maintain focus on objectives over extended execution.
- **Goal Alignment:** Ensuring the agent’s actions remain aligned with its original high-level goals throughout execution.

How It Works: Step-by-step Explanation

1. **Maintain the Plan:** The agent is instructed to maintain a running to-do list or plan, often externalized to a persistent file (e.g., [todo.md](#)) or stored in the agent’s state.
2. **Update Progress:** At each step in the agent’s iterative loop, it updates the task list by checking off completed subtasks.

3. **Recite into Context:** The updated plan is then appended to the end of the context for the next iteration. This constant overwriting or appending ensures the objectives are always within the model’s recent attention span.
4. **Bias Attention:** By keeping the plan recent, the model’s focus is biased toward the global objectives, reducing goal misalignment and avoiding the agent drifting off-topic.

When to Use This Pattern

Use when:

- Building long-horizon agents that must sustain goals over many steps.
- Mitigating the risk of the agent forgetting its high-level goals or getting “lost in the middle.”
- The agent needs to dynamically pull information back into context to remind itself what it is supposed to be doing.
- Working on open-ended problems where the agent generates and manages its own steps.

Avoid when:

- The task is simple or single-turn, as the overhead of managing and reciting the task list is unnecessary and increases token cost.
- The agent is purely reactive and does not require explicit step-by-step planning.
- The context window is short and cannot accommodate the additional plan recitation overhead.

Decision Guidelines

Use this pattern as a critical context engineering strategy to ensure the agent remains focused on its high-level goals throughout a complex or long-running workflow. It is especially effective in open-ended problems where the agent generates and manages its own steps. Consider the trade-off: recitation adds token cost but prevents costly goal drift and rework. For tasks requiring sustained focus over 10+ steps, the benefits typically outweigh the costs.

Practical Applications & Use Cases

The Persistent Task List (Recitation) pattern is essential for maintaining goal alignment in complex, multi-step agent workflows.

- **Autonomous Research:** An agent tasked with writing a comprehensive research report uses a plan to track phases like information gathering, synthesis, and revision over hundreds of steps.
- **Code Generation:** Systems like Claude Code use a no-op Todo list tool as a context engineering strategy to keep the agent on track during multi-file, multi-step coding tasks.
- **Workflow Persistence:** Manus AI utilizes this pattern by making its agent create and continuously update a [todo.md](#) file as subtasks are completed, ensuring goal persistence.
- **Filesystem Integration:** Writing a plan to the filesystem allows the agent to pull this information back into the context window later on, enabling goal persistence across sessions.
- **Multi-Agent Orchestration:** Orchestrator agents maintain and recite plans as they delegate tasks to worker agents, ensuring the overall objective remains clear despite distributed execution.

Implementation

Prerequisites

```
pip install deepagents  
  
# or  
  
pip install langchain langchain-openai  
  
# or  
  
pip install google-adk
```

Basic Example: Planning Tool (Conceptual)

The deepagents package incorporates a built-in Write to-dos tool, which is a key component of its deep agent architecture. Although conceptually acting as a planning tool, it is often a no-op function, meaning its primary purpose is manipulating context and attention rather than performing an external action. Its detailed description provided to the LLM guides the agent on how to manage tasks, prioritize them, and update their status in real time (e.g., pending, in progress, completed).

```

# Conceptual Tool Implementation (based on deepagents)

def write_to_dos_tool(new_todos: str) → str:
    """
    Updates the agent's internal state with a new to-do list, which is
    then included in the context for the next iteration (Recitation).

    Args:
        new_todos: The full text of the updated to-do list.
    """
    # In a real system, this updates the agent's internal state dictionary
    # (e.g., state['to_dos'] = new_todos) and returns an Observation.

    # The agent reads the description and knows to output an updated list
    # via this tool call when its plan changes.

    print(f"DEBUG: Reciting updated task list to context:\n{new_todos}")
    return "Task list updated in memory/state."

# The Recitation pattern is realized when the agent framework
# ensures the content of 'new_todos' is included in the prompt
# for every subsequent LLM call.

```

Advanced Example: Filesystem-Based Recitation

```

from typing import Dict, List
from pathlib import Path
import json

class RecitationAgent:
    def __init__(self, todo_file: str = "todo.md"):
        self.todo_file = Path(todo_file)
        self.todo_file.touch(exist_ok=True)
        self.context_history = []

    def read_todo(self) → str:
        """Read the persistent task list from filesystem."""
        try:
            return self.todo_file.read_text()
        except FileNotFoundError:
            return "# Task List\n\nNo active tasks."

    def update_todo(self, updated_todos: str) → str:
        """Update the task list and return confirmation."""
        self.todo_file.write_text(updated_todos)
        return f"Task list updated. Current plan:\n\n{updated_todos}"

    def build_context(self, user_message: str) → List[Dict[str, str]]:
        """Build context with recitation pattern."""
        # Read the current plan
        current_plan = self.read_todo()

        # Build context: history + new message + recited plan
        messages = self.context_history.copy()
        messages.append({"role": "user", "content": user_message})

        # Recitation: Append plan to end of context
        messages.append({

```

```

        "role": "system",
        "content": f"## Current Plan (Recited)\n\n{current_plan}\n\nRemember: This
    })

    return messages

def process_step(self, user_message: str, llm_response: str):
    """Process a step and update context history."""
    # Add to history (append-only for KV-Cache optimization)
    self.context_history.append({"role": "user", "content": user_message})
    self.context_history.append({"role": "assistant", "content": llm_response})

    # Check if response contains updated todos
    if "write_to_dos" in llm_response or "update_todo" in llm_response.lower():
        # Extract and update todos (simplified - in production, use structured extr
        # This would typically be handled by a tool call parser
        pass

# Usage
agent = RecitationAgent("agent_todo.md")

# Initial planning
initial_plan = """# Research Project Plan

- [ ] Gather sources on topic X
- [ ] Analyze key findings
- [ ] Draft report sections
- [ ] Review and refine
"""
agent.update_todo(initial_plan)

# Each step recites the plan
context = agent.build_context("I've gathered 5 sources. What's next?")
# The plan is automatically appended to context, keeping goals in focus

```

Explanation: This example demonstrates the Recitation pattern with filesystem persistence. The agent maintains a [todo.md](#) file that is read at each step and appended to the context. This ensures the plan remains in the model's recent attention span, preventing goal drift during long-horizon tasks.

Framework-Specific Examples

LANGGRAPH: RECITATION NODE

```

from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    todo_list: str
    scratchpad: str

def recite_plan_node(state: AgentState) → AgentState:
    """Node that recites the plan into context."""
    # Read current plan from state
    plan = state.get("todo_list", "# No active plan")

    # Append plan to messages (recitation)
    state["messages"].append({
        "role": "system",
        "content": f"## Current Plan\n\n{plan}\n\nKeep this plan in mind as you work."
    })

    return state

def reasoning_node(state: AgentState) → AgentState:
    """Main reasoning node that processes tasks."""
    # LLM processes with plan in recent context
    # ... LLM call with state["messages"] ...
    return state

# Build graph with recitation
workflow = StateGraph(AgentState)
workflow.add_node("recite_plan", recite_plan_node)
workflow.add_node("reasoning", reasoning_node)

```

```
workflow.add_edge("recite_plan", "reasoning")  
workflow.add_edge("reasoning", "recite_plan") # Loop back to re-recite  
workflow.set_entry_point("recite_plan")
```


GOOGLE ADK: STATE-BASED RECITATION

```

from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

def get_plan_from_state(state: dict) → str:
    """Retrieve plan from agent state."""
    return state.get("todo_list", "# No active plan")

# Agent with recitation in instruction
agent = LlmAgent(
    name="RecitationAgent",
    model="gemini-2.0-flash",
    instruction="""You are a task-oriented agent.

    At the start of each turn, you will see your current plan (todo list)
    recited in the context. This plan represents your high-level objectives.

    As you complete tasks:
    1. Update the plan by marking tasks as complete
    2. Add new subtasks as needed
    3. The updated plan will be recited in the next turn

    Always keep the plan updated and use it to guide your actions.""",
    output_key="last_response"
)

# Runner manages state and can inject plan into context
runner = Runner(
    agent=agent,
    app_name="recitation_app",
    session_service=InMemorySessionService()
)

# Custom context builder that recites plan

```

```
def build_context_with_recitation(session_state: dict, user_input: str):  
    plan = session_state.get("todo_list", "# No active plan")  
    return f"{user_input}\n\n## Current Plan (Recited)\n\n{plan}"
```

MANUS AI PATTERN: FILESYSTEM [TODO.MD](#)

```

from pathlib import Path
import re

class ManusStyleRecitation:
    def __init__(self, workspace_dir: str = "./workspace"):
        self.workspace = Path(workspace_dir)
        self.todo_file = self.workspace / "todo.md"
        self.workspace.mkdir(exist_ok=True)

    def ensure_todo_exists(self):
        """Ensure todo.md exists, create if not."""
        if not self.todo_file.exists():
            self.todo_file.write_text("# Task List\n\n## Active Tasks\n\n- [ ] Initial")

    def read_and_recite(self) → str:
        """Read todo.md and format for recitation."""
        self.ensure_todo_exists()
        content = self.todo_file.read_text()
        return f"## Current Task Plan (from todo.md)\n\n{content}\n\n---\n\nThis plan i

    def update_todo(self, new_content: str):
        """Update todo.md with new plan."""
        self.todo_file.write_text(new_content)
        return f"Updated todo.md. Plan now contains {len(new_content.split(chr(10)))} l

    def extract_todo_update(self, agent_response: str) → str:
        """Extract todo update from agent response (simplified)."""
        # In production, use structured tool calls
        # This is a simplified regex-based extraction
        pattern = r"```markdown\s*(#.*?)\s*```"
        match = re.search(pattern, agent_response, re.DOTALL)
        if match:
            return match.group(1)
        return None

```

```
# Usage
recitation = ManusStyleRecitation()

# At each agent step:
current_plan = recitation.read_and_recite()
# Append current_plan to end of context before LLM call

# After LLM responds:
# Check if response contains todo update, then:
# recitation.update_todo(updated_plan)
```

Key Takeaways

- **Core Concept:** Recitation is a context engineering strategy used to actively manage the LLM’s finite context window by biasing its attention toward high-level goals.
- **Best Practice:** The plan should be frequently overwritten or appended to the end of the prompt to maximize the effect of recency bias.
- **Common Pitfall:** Avoid using this pattern for simple, single-turn tasks where the overhead outweighs the benefits. Also, ensure the plan doesn’t become too verbose, as it adds to token costs.
- **Performance Note:** This pattern is essential for avoiding the performance degradation associated with the “lost in the middle” problem in very long contexts. The tool used for managing the to-do list can be a logical no-op, as its function is simply to update the conversational context or state.
- **Implementation Note:** The recitation mechanism works best when combined with stable, append-only context structures to maximize KV-Cache efficiency.

Related Patterns

This pattern works well with:

- **Leverage External Memory (Filesystem as Context):** The task list itself is often stored in an external persistent store (like a filesystem) so it can be reliably referenced and updated across many steps.

- **Stable, Append-Only Context:** Recitation aligns with the need to keep the prompt prefix stable; the plan is appended to the context history rather than modifying the core instructions.
- **Memory Management:** Recitation is a specific memory management technique for maintaining goal alignment in long-horizon tasks.

This pattern is often combined with:

- **Planning:** The Recitation pattern is the execution mechanism for ensuring the plan generated by the Planning pattern is followed throughout a long task.
- **Orchestrator (Coordinator) Pattern:** The central Orchestrator agent typically maintains and recites the plan as it delegates and synthesizes results from worker agents.
- **Goal Setting and Monitoring:** Recitation ensures that goals set at the beginning remain visible and actionable throughout execution.

References

- Agentic AI System Design Patterns
 - Context Engineering for AI Agents: Lessons from Building Manus
 - Implementing deepagents: a technical walkthrough
 - Deep Agents
 - How agents can use filesystems for context engineering
 - Agentic Design Patterns Engineering Playbook
-

Pattern: Leverage External Memory (Filesystem as Context)

Treating external persistent storage as an unlimited extension of the agent's working memory, enabling

restorable compression and just-in-time retrieval of large data.

Module ID: module-10b

LEVERAGE EXTERNAL MEMORY (FILESYSTEM AS CONTEXT)

Motivation

Humans don't store everything in working memory. We use notebooks, calendars, reference books, and digital files as external memory. When needed, we retrieve relevant information. We can't remember every detail, but we know where to find it. The External Memory pattern gives agents this capability: treating persistent storage as unlimited memory, retrieving information just-in-time rather than trying to keep everything in the limited context window.

Pattern Overview

What it is: A mechanism to treat an external persistent memory (like a filesystem or database) as an unlimited extension of the agent's working memory.

When to use: When the agent needs to handle large data or unstructured observations (like web pages or PDF text) that would exceed context length limits. Also used for long-horizon tasks that require persistent, restorable information retention.

Why it matters: It helps mitigate the fundamental constraint of the LLM’s finite context window. It reduces token costs and latency by allowing the agent to retrieve only the relevant information when needed.

The Leverage External Memory pattern addresses one of the most fundamental constraints in LLM-based agent systems: the finite context window. As agents process complex tasks, they encounter large data sources—web search results, PDF documents, codebases, or extensive research findings—that would quickly exhaust the available context tokens. Simply truncating or summarizing this data risks losing critical information, while including everything inflates costs and can degrade performance due to attention dilution.

This pattern treats external storage (filesystem, databases, or other persistent stores) as an unlimited extension of the agent’s working memory. Instead of keeping all data in the immediate context, the agent offloads large content to external storage and retains only lightweight references (file paths, URLs, or database keys). When specific information is needed, the agent performs targeted retrieval, pulling only the relevant portions back into context.

The pattern is particularly powerful because it enables restorable compression: the agent can drop large content from context while maintaining the ability to retrieve it precisely when needed. This creates a “just-in-time” information architecture where context remains focused and efficient, while the agent retains access to an unlimited knowledge base.

Key Concepts

- **Externalized Memory:** Large data stored outside the context window in persistent storage (filesystem, database, etc.).
- **Scratchpad Memory Pattern:** Using temporary files or workspaces as intermediate storage for agent computations and observations.
- **Restorable Compression:** Dropping large content from context while retaining a reference (path, URL, key) that allows precise retrieval later.
- **Just-in-Time Retrieval:** Selectively reading only relevant portions of stored data when needed, rather than loading everything upfront.
- **Context Engineering:** Strategic management of what information appears in the context window to optimize performance and cost.

How It Works: Step-by-step Explanation

1. **Offload Large Data:** The agent writes intermediate results, notes, or large data (such as a 10K token web search result) to an external persistent file (a scratch file) instead of keeping it in the immediate conversation context.
2. **Retain Reference:** The compression strategy is always *restorable*—the agent drops the large content but retains a lightweight reference, such as the file path or URL, so it can be reloaded on demand.
3. **Just-in-Time Retrieval:** When necessary for a subsequent step, the agent selectively reads only the relevant portions of the stored data. This can be done using dedicated tools like `grep` or `read_file` to pull in only what is needed.
4. **Inject Context:** The small, relevant snippets are injected into the agent's next prompt, ensuring the context is focused and concise, thereby preventing the context window from being flooded.

When to Use This Pattern

✅ Use when:

- Building long-horizon agents that must sustain goals without losing information.
- Dealing with large inputs (web search, PDFs) that would exceed context limits.
- Implementing complex systems where large amounts of necessary context must be available on demand (e.g., codebases or documentation).
- The agent needs to process multiple large documents or datasets that cannot fit simultaneously in context.
- Working with dynamic data that changes over time and needs to be stored and retrieved across sessions.

❌ Avoid when:

- The task is single-turn or simple, as the overhead of file management is unnecessary.
- The application requires only a small, static set of knowledge accessible via standard RAG techniques.
- All necessary information fits comfortably within the context window without performance degradation.
- The retrieval overhead (file I/O, database queries) would introduce unacceptable latency for real-time applications.

Decision Guidelines

This pattern is a crucial context engineering strategy, particularly for multi-step agentic systems, as it makes the retrieved context as small a subset of the needed information as possible. The filesystem effectively acts as a single, flexible interface for the agent to store, retrieve, and update an infinite amount of context. Consider: the size of your data (large = external memory), the retrieval pattern (targeted = filesystem tools, semantic = RAG), and the persistence requirement (session = temporary files, cross-session = persistent storage). Always ensure references are maintained so data remains restorable.

Practical Applications & Use Cases

The Leverage External Memory pattern is essential for agents that work with large datasets, complex codebases, or extensive research materials.

- **Manus AI:** Uses the filesystem as “structured, externalized memory” to store and retrieve information across agent steps.
- **Planning Persistence:** The agent writes its long-horizon plan (e.g., a `todo.md` file) to the filesystem, allowing it to recite this plan back into the context later to remind itself of its objectives. Anthropic’s research agent saves its plan to memory before spawning subagents.
- **Subagent Collaboration:** Subagents write their research findings and knowledge directly to the filesystem, and only pass lightweight references to the main coordinator, minimizing token overhead and avoiding the “game of telephone.”
- **Code Agents:** Agent systems, such as those in Deep Agents, are equipped with filesystem tools like `read_file`, `write_file`, `ls`, and `edit_file` to navigate and manipulate codebases that exceed context limits.
- **Skill Management:** Instructions and skills can be stored as files, which the agent can dynamically read as needed, rather than stuffing all instructions into the system prompt.
- **Research Agents:** Agents conducting literature reviews or web research offload search results and articles to files, then retrieve specific sections when synthesizing findings.
- **Document Processing:** Agents processing large PDFs or documents save extracted content to files, then query specific sections when answering questions.

Implementation

Prerequisites

```
pip install langchain langchain-openai  
# or  
pip install google-adk  
# or  
pip install deepagents
```

Basic Example: Offloading and Retrieval

This example demonstrates the scratchpad memory pattern, where a large web search observation is offloaded to a file, and only relevant parts are retrieved later:

```

from pathlib import Path
from typing import Dict, List
import json

class ExternalMemoryAgent:
    def __init__(self, workspace_dir: str = "./workspace"):
        self.workspace = Path(workspace_dir)
        self.workspace.mkdir(exist_ok=True)
        self.memory_index = {} # Maps keys to file paths

    def offload_to_memory(self, content: str, key: str, metadata: Dict = None) → str:
        """Offload large content to external memory and return reference."""
        filepath = self.workspace / f"{key}.txt"
        filepath.write_text(content)

        # Store metadata for retrieval
        self.memory_index[key] = {
            "path": str(filepath),
            "metadata": metadata or {},
            "size": len(content)
        }

        # Return lightweight reference
        return f"Content stored in memory: {key} ({len(content)} chars). Use retrieve_m

    def retrieve_memory(self, key: str, query: str = None, max_lines: int = 50) → str:
        """Retrieve from external memory, optionally with targeted search."""
        if key not in self.memory_index:
            return f"Memory key '{key}' not found."

        filepath = Path(self.memory_index[key]["path"])
        if not filepath.exists():
            return f"Memory file for '{key}' not found."

        content = filepath.read_text()

```

```

        # If query provided, search for relevant lines
        if query:
            lines = content.split('\n')
            relevant_lines = [
                line for line in lines
                if query.lower() in line.lower()
           ][:max_lines]
            return '\n'.join(relevant_lines)

        # Return first portion if no query
        return content[:2000] # First 2000 chars

    def list_memory(self) → List[str]:
        """List all available memory keys."""
        return list(self.memory_index.keys())

# Usage
agent = ExternalMemoryAgent()

# Offload large web search result
large_result = "..." # 10K token web search result
reference = agent.offload_to_memory(
    large_result,
    key="web_search_agentic_patterns",
    metadata={"source": "web_search", "query": "agentic AI design patterns"}
)

# Later, retrieve only relevant parts
relevant = agent.retrieve_memory(
    "web_search_agentic_patterns",
    query="recitation pattern",
    max_lines=20
)

# Inject 'relevant' into next prompt instead of full 10K token result

```

Explanation: This example demonstrates the core pattern: offloading large content to external storage, maintaining a reference, and performing targeted retrieval. The agent can store unlimited data externally while keeping context focused and efficient.

Advanced Example: Filesystem Tools with Targeted Reading

```

from pathlib import Path
from typing import Optional, List
import re

class FilesystemMemoryTools:
    def __init__(self, workspace: Path):
        self.workspace = workspace
        self.workspace.mkdir(exist_ok=True)

    def write_file(self, filepath: str, content: str) → str:
        """Write content to file in workspace."""
        full_path = self.workspace / filepath
        full_path.parent.mkdir(parents=True, exist_ok=True)
        full_path.write_text(content)
        return f"Wrote {len(content)} characters to {filepath}"

    def read_file(self, filepath: str, start_line: int = 1, end_line: Optional[int] = None) → str:
        """Read file with optional line range for targeted retrieval."""
        full_path = self.workspace / filepath
        if not full_path.exists():
            return f"File {filepath} not found."

        lines = full_path.read_text().split('\n')

        # Adjust for 0-based indexing
        start_idx = max(0, start_line - 1)
        end_idx = end_line if end_line else len(lines)

        selected_lines = lines[start_idx:end_idx]
        return '\n'.join(selected_lines)

    def grep_file(self, filepath: str, pattern: str, max_matches: int = 10) → str:
        """Search file for pattern and return matching lines with context."""

```

```

    full_path = self.workspace / filepath
    if not full_path.exists():
        return f"File {filepath} not found."

    content = full_path.read_text()
    lines = content.split('\n')

    matches = []
    for i, line in enumerate(lines):
        if re.search(pattern, line, re.IGNORECASE):
            # Include line number and context
            context_start = max(0, i - 1)
            context_end = min(len(lines), i + 2)
            context = '\n'.join(lines[context_start:context_end])
            matches.append(f"Line {i+1}:\n{context}")
            if len(matches) ≥ max_matches:
                break

    return '\n'.join(matches) if matches else f"No matches found for pattern: {patt

def list_files(self, directory: str = ".") → str:
    """List files in directory."""
    dir_path = self.workspace / directory
    if not dir_path.exists():
        return f"Directory {directory} not found."

    files = [f.name for f in dir_path.iterdir() if f.is_file()]
    dirs = [f.name + "/" for f in dir_path.iterdir() if f.is_dir()]
    return '\n'.join(sorted(dirs + files))

# Usage with agent
workspace = Path("./agent_workspace")
fs_tools = FilesystemMemoryTools(workspace)

# Agent offloads large PDF text
pdf_content = "..." # Large extracted PDF text

```



```
fs_tools.write_file("research_paper.pdf.txt", pdf_content)

# Later, agent searches for specific information
relevant_section = fs_tools.grep_file(
    "research_paper.pdf.txt",
    pattern="recitation|external memory",
    max_matches=5
)

# Agent uses only the relevant section in context
```

Explanation: This advanced example provides filesystem tools with targeted reading capabilities. The `read_file` tool supports line ranges, and `grep_file` enables semantic search within stored files. This allows agents to retrieve precisely what they need without loading entire files into context.

Framework-Specific Examples

DEEP AGENTS: BUILT-IN FILESYSTEM TOOLS

```
# Deep Agents includes built-in filesystem tools with detailed specifications

def read_file_tool(filepath: str, start_line: int = 1, num_lines: int = 2000) → str:
    """
    Read a file from the workspace.

    Args:
        filepath: Path to file relative to workspace
        start_line: Line number to start reading from (1-indexed)
        num_lines: Maximum number of lines to read (default: 2000)

    Returns:
        File content within specified line range
    """
    # Implementation reads up to 2000 lines by default
    # but allows specifying line offsets and limits
    pass

def write_file_tool(filepath: str, content: str) → str:
    """Write content to file in workspace."""
    pass

def list_files_tool(directory: str = ".") → str:
    """List files and directories in workspace."""
    pass

# Agent uses these tools to manage external memory
# Large observations are written to files, then read selectively
```

LANGGRAPH: FILESYSTEM STATE MANAGEMENT

```

from langgraph.graph import StateGraph
from typing import TypedDict, Annotated
import operator
from pathlib import Path

class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    scratchpad_files: dict # Maps keys to file paths
    workspace: str

def offload_to_scratchpad(state: AgentState, content: str, key: str) → AgentState:
    """Offload content to scratchpad file."""
    workspace = Path(state["workspace"])
    workspace.mkdir(exist_ok=True)

    filepath = workspace / f"{key}.txt"
    filepath.write_text(content)

    state["scratchpad_files"][key] = str(filepath)
    return state

def retrieve_from_scratchpad(state: AgentState, key: str, query: str = None) → AgentSt
    """Retrieve from scratchpad with optional filtering."""
    if key not in state["scratchpad_files"]:
        return state

    filepath = Path(state["scratchpad_files"][key])
    content = filepath.read_text()

    # If query, filter content
    if query:
        lines = [line for line in content.split('\n') if query.lower() in line.lower()]
        content = '\n'.join(lines[:50]) # Top 50 matches

```

```
# Inject into messages

state["messages"].append({
    "role": "system",
    "content": f"Retrieved from {key}:\n\n{content[:2000]}"
})

return state


# Graph with external memory management
workflow = StateGraph(AgentState)
workflow.add_node("offload", offload_to_scratchpad)
workflow.add_node("retrieve", retrieve_from_scratchpad)
# ... rest of workflow
```

GOOGLE ADK: EXTERNAL STORAGE INTEGRATION

```

from google.adk.agents import LlmAgent
from google.adk.tools import Tool
from pathlib import Path

def read_file_tool(filepath: str, start_line: int = 1, end_line: int = None) → str:
    """Read file with line range support."""
    path = Path(filepath)
    if not path.exists():
        return f"File {filepath} not found."

    lines = path.read_text().split('\n')
    start_idx = max(0, start_line - 1)
    end_idx = end_line if end_line else len(lines)

    return '\n'.join(lines[start_idx:end_idx])

def write_file_tool(filepath: str, content: str) → str:
    """Write content to file."""
    path = Path(filepath)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(content)
    return f"Wrote {len(content)} characters to {filepath}"

# Agent with filesystem tools
agent = LlmAgent(
    name="ExternalMemoryAgent",
    model="gemini-2.0-flash",
    instruction="""You have access to filesystem tools for managing external memory.

    When you receive large data (web search results, documents, etc.):
    1. Use write_file to save it to external storage
    2. Keep only a reference in your response
    3. Use read_file with line ranges to retrieve specific parts when needed
  """
)

```

```
This keeps your context focused and efficient.""",
tools=[read_file_tool, write_file_tool]
)
```

Key Takeaways

- **Core Function:** External memory provides persistent, unlimited storage, preventing context window limits from being hit by offloading large data.
- **Efficiency Principle:** Retrieval should be focused (Just-in-Time Retrieval), ensuring the agent injects only necessary snippets into the prompt to reduce token costs and latency.
- **Persistence Requirement:** The agent must maintain a reference (like a file path or URL) to ensure the dropped information remains restorable.
- **Context Quality:** Context engineering, which includes leveraging external memory, is essential because an agent's ability to reason is entirely dependent on the quality of its context.
- **Best Practice:** Design tools with targeted retrieval capabilities (line ranges, search functions) to enable precise information extraction without loading entire files.
- **Common Pitfall:** Failing to maintain references to offloaded data makes it irretrievable, defeating the purpose of external memory. Always ensure references are preserved in agent state or context.

Related Patterns

This pattern works well with:

- **Persistent Task List (Recitation):** The persistent plan is often stored in the external filesystem (`todo.md`) to enable its continuous recitation into the context.
- **Stable, Append-Only Context:** Offloading large data helps maintain a stable context prefix, which is crucial for maximizing KV-cache reuse and reducing cost.
- **Memory Management:** External memory is a key component of comprehensive memory management strategies, complementing context window management and compression.

This pattern is often combined with:

- **Tool Result Management (Retrieve-then-Read):** This structure is implemented by using filesystem tools that allow targeted reading (e.g., specifying a line range) after the initial large data has been stored.
- **Knowledge Retrieval (RAG):** External memory can store retrieved documents, while RAG provides semantic search capabilities over the stored content.
- **Multi-Agent Architectures:** Subagents write findings to shared external memory, and the orchestrator retrieves only relevant portions when synthesizing results.

References

- Agentic AI System Design Patterns
 - Context Engineering for AI Agents: Lessons from Building Manus
 - Deep Agents: Filesystem Tools Documentation
 - LangGraph State Management: <https://langchain-ai.github.io/langgraph/>
 - Google ADK Tools: <https://google.github.io/adk-docs/tools/>
 - How agents can use filesystems for context engineering
-

Context Compression: Managing the Finite Window

Comprehensive techniques for fitting necessary information into the LLM's finite context window through externalization, summarization, pruning, and attention manipulation.

Module ID: module-10c

CONTEXT COMPRESSION: MANAGING THE FINITE WINDOW

Motivation

When summarizing a long meeting, you extract key decisions and action items, not every word spoken. When reading a research paper, you focus on the abstract and conclusions. Humans naturally compress information, keeping essential details while discarding the rest. Context Compression applies this principle: fitting necessary information into the finite context window through summarization, pruning, and selective retrieval, just as we distill complex information into manageable insights.

Pattern Overview

What it is: Context compression encompasses techniques used to fit the necessary information for a task into the Large Language Model's (LLM) finite context window, thereby sustaining performance and reducing costs.

When to use: When building agents that process large amounts of information, maintain long conversation histories, or work with extensive datasets that would exceed context window limits or degrade performance.

Why it matters: The finite context window is a fundamental constraint of LLM-based agents. Without effective compression strategies, agents hit hard limits, suffer performance degradation, and incur excessive costs. Context compression enables agents to work with unlimited information while maintaining efficiency and performance.

The context window represents the maximum number of tokens an LLM can process in a single interaction. This finite boundary creates a fundamental challenge: agents must balance the need for comprehensive information against the constraints of token limits, processing costs, and performance degradation. As agents tackle complex, multi-step tasks, they accumulate conversation history, tool results, and intermediate reasoning that can quickly exhaust available context.

Context compression is not a single technique but a comprehensive strategy combining multiple approaches. The most powerful method is externalizing memory—offloading large or long-term information to persistent storage. For information that must remain in context, techniques like summarization, truncation, and attention manipulation ensure the agent maintains focus on what matters most.

Effective context compression is essential for production agent systems. It directly impacts cost (fewer tokens = lower API costs), latency (shorter contexts = faster processing), and performance (focused contexts = better reasoning). Without compression, agents cannot scale to handle real-world complexity.

Key Concepts

- **Finite Context Window:** The maximum number of tokens an LLM can process at once, creating a hard limit on information capacity.
- **Externalized Memory:** Offloading large data to persistent storage (filesystem, database) to extend working memory beyond context limits.
- **Restorable Compression:** Dropping content from context while maintaining references (paths, URLs) for on-demand retrieval.
- **Contextual Pruning:** Removing or summarizing less relevant information to preserve space for critical content.
- **Summarization:** Condensing conversation history or documents into compact representations that preserve essential information.

- **Chunking:** Breaking large documents into smaller, manageable pieces for processing and retrieval.
- **Attention Manipulation:** Strategically positioning important information (like plans) to bias model attention.
- **Lost-in-the-Middle Problem:** Performance degradation when critical information appears in the middle of very long contexts.

How It Works: Step-by-step Explanation

Context compression operates through multiple complementary strategies:

1. **Externalize Large Data:** The primary compression strategy is to offload large or long-term information to external persistent storage. The agent writes intermediate results, tool outputs, or large observations to files or databases, keeping only lightweight references in context.
2. **Maintain Restorable References:** Compression must be restorable. The agent drops large content from the prompt but retains references (file paths, URLs, database keys) that enable precise retrieval when needed.
3. **Just-in-Time Retrieval:** When specific information is required, the agent retrieves only relevant snippets using targeted tools (grep, line-range reads, semantic search) rather than loading entire files.
4. **Summarize Context History:** For information that must remain in context, older conversation segments are summarized into compact blocks, preserving essential information while freeing space for active reasoning.
5. **Prune and Prioritize:** Less critical information is truncated or removed, ensuring the most relevant content remains accessible within context limits.
6. **Manipulate Attention:** Important information (like high-level plans) is strategically positioned (e.g., at the end of context) to leverage recency bias and maintain focus.

When to Use This Pattern

Use when:

- Building agents that process large documents, datasets, or extensive research materials.
- Maintaining long conversation histories across multiple turns.
- Working with unstructured data (web pages, PDFs) that exceeds context limits.
- Implementing multi-step agents that accumulate intermediate results and reasoning.

- Cost and latency optimization are critical requirements.
- Performance degradation is observed with long contexts.

✗ Avoid when:

- All necessary information fits comfortably within context limits without performance issues.
- The task is simple and single-turn, making compression overhead unnecessary.
- Real-time retrieval latency from external storage is unacceptable.
- The compression strategy would lose critical information that cannot be restored.

Decision Guidelines

Context compression is essential for any production agent system handling real-world complexity. The strategy should be layered: externalize large data first (most effective), then summarize/prune what remains in context, and finally use attention manipulation for critical information. Consider: data size (large = externalize), access pattern (frequent = keep in context, rare = externalize), and criticality (essential = keep recent, supplementary = summarize or externalize). Always maintain restorable references for externalized data.

Practical Applications & Use Cases

Context compression is fundamental to building scalable, efficient agent systems across diverse applications.

- **Research Agents:** Agents conducting literature reviews offload search results and papers to external storage, retrieving specific sections when synthesizing findings.
- **Code Generation Agents:** Systems like Claude Code use external memory to store codebase context, reading specific files and functions on demand rather than loading entire repositories.
- **Long-Running Conversations:** Chatbots and assistants compress old conversation history through summarization, maintaining recent context while preserving essential context from earlier exchanges.
- **Document Processing:** Agents processing large PDFs or documents save extracted content externally, then query specific sections when answering questions.
- **Multi-Agent Systems:** Orchestrator agents compress subagent outputs, storing detailed results externally and keeping only summaries and references in context.

- **RAG Systems:** Knowledge bases are chunked and indexed, with agents retrieving only relevant chunks rather than entire documents.
- **Planning Agents:** Agents maintain persistent plans externally ([todo.md](#)) and recite them into context, ensuring goals remain visible without consuming context space.

Implementation

Prerequisites

```
pip install langchain langchain-openai
# or
pip install google-adk
# or
pip install tiktoken # For token counting
```

Basic Example: Context Compression Manager

This example demonstrates a comprehensive context compression system combining externalization, summarization, and pruning:

```

from typing import List, Dict
from pathlib import Path
import tiktoken

class ContextCompressionManager:
    def __init__(self, workspace_dir: str = "./workspace", max_tokens: int = 100000):
        self.workspace = Path(workspace_dir)
        self.workspace.mkdir(exist_ok=True)
        self.max_tokens = max_tokens
        self.encoding = tiktoken.encoding_for_model("gpt-4")
        self.context_history = []
        self.external_memory = {}

    def count_tokens(self, text: str) → int:
        """Count tokens in text."""
        return len(self.encoding.encode(text))

    def externalize(self, content: str, key: str) → str:
        """Offload large content to external storage."""
        filepath = self.workspace / f"{key}.txt"
        filepath.write_text(content)
        self.external_memory[key] = str(filepath)

        # Return lightweight reference
        size = len(content)
        return f"[External Memory: {key} ({size} chars stored)]"

    def should_compress(self) → bool:
        """Check if context needs compression."""
        total_tokens = sum(self.count_tokens(msg["content"]) for msg in self.context_history)
        return total_tokens > self.max_tokens * 0.9

    def summarize_old_messages(self, messages: List[Dict], llm) → str:
        """Summarize old conversation messages."""
        if not messages:

```

```

        return ""

    # Prepare messages for summarization
    conversation_text = "\n".join([
        f"{msg['role']}: {msg['content'][:500]}" # Truncate for summary
        for msg in messages
    ])

    summary_prompt = f"""Summarize this conversation history concisely,
    preserving key decisions, user preferences, and important context:

    {conversation_text}

    Summary: """

    summary = llm.invoke(summary_prompt).content
    return summary

def compress_context(self, llm) → List[Dict]:
    """Compress context when approaching limits."""
    if not self.should_compress():
        return self.context_history

    # Keep recent messages (last 10)
    recent_messages = self.context_history[-10:]
    old_messages = self.context_history[:-10]

    # Summarize old messages
    if old_messages:
        summary = self.summarize_old_messages(old_messages, llm)
        summary_message = {
            "role": "system",
            "content": f"Previous conversation summary: {summary}"
        }

        # Replace old messages with summary
        self.context_history = [summary_message] + recent_messages

```

```

        return self.context_history

def add_message(self, role: str, content: str, llm=None):
    """Add message with automatic compression."""
    # Check if content is too large
    if self.count_tokens(content) > 5000:
        # Externalize large content
        key = f"message_{len(self.context_history)}"
        reference = self.externalize(content, key)
        self.context_history.append({
            "role": role,
            "content": f"{reference}\n\n[Content stored externally. Use retrieve_ex
        })
    else:
        self.context_history.append({"role": role, "content": content})

    # Compress if needed
    if llm and self.should_compress():
        self.compress_context(llm)

def retrieve_external(self, key: str, query: str = None) → str:
    """Retrieve from external memory with optional filtering."""
    if key not in self.external_memory:
        return f"Key '{key}' not found in external memory."

    filepath = Path(self.external_memory[key])
    content = filepath.read_text()

    # If query provided, filter content
    if query:
        lines = [line for line in content.split('\n') if query.lower() in line.lower]
        return '\n'.join(lines[:50]) # Top 50 matches

    return content[:2000] # Return first portion

```

```
# Usage

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o")
compressor = ContextCompressionManager(max_tokens=100000)

# Add messages (large content automatically externalized)
compressor.add_message("user", "Large web search result...", llm)
compressor.add_message("assistant", "Processing...", llm)

# Context automatically compressed when approaching limits
compressed_context = compressor.compress_context(llm)
```

Explanation: This example demonstrates a comprehensive compression manager that automatically externalizes large content, summarizes old messages, and maintains context within token limits. It combines multiple compression strategies in a unified system.

Advanced Example: Intelligent Context Pruning

```

from typing import List, Dict, Tuple
import re
from collections import defaultdict

class IntelligentContextPruner:
    def __init__(self, max_tokens: int = 100000, keep_recent: int = 20):
        self.max_tokens = max_tokens
        self.keep_recent = keep_recent
        self.encoding = tiktoken.encoding_for_model("gpt-4")
        self.importance_scores = {}

    def calculate_importance(self, message: Dict, context: List[Dict]) → float:
        """Calculate importance score for a message."""
        score = 0.0
        content = message.get("content", "")

        # Recent messages are more important
        position = context.index(message) if message in context else len(context)
        recency_score = 1.0 / (position + 1)
        score += recency_score * 0.3

        # System messages are important
        if message.get("role") == "system":
            score += 0.4

        # Messages with tool results might be important
        if "tool" in content.lower() or "result" in content.lower():
            score += 0.2

        # Messages with user queries are important
        if message.get("role") == "user":
            score += 0.3

```

```

# Check for key indicators
key_phrases = ["error", "important", "critical", "decision", "plan"]
if any(phrase in content.lower() for phrase in key_phrases):
    score += 0.2

return score

def prune_context(self, context: List[Dict], target_tokens: int) → List[Dict]:
    """Intelligently prune context to fit within token budget."""
    # Calculate importance for each message
    importance_scores = {
        i: self.calculate_importance(msg, context)
        for i, msg in enumerate(context)
    }

    # Always keep most recent messages
    recent_indices = set(range(max(0, len(context) - self.keep_recent), len(context)

    # Sort by importance (descending)
    sorted_indices = sorted(
        importance_scores.items(),
        key=lambda x: x[1],
        reverse=True
    )

    # Select messages to keep
    kept_indices = set(recent_indices)
    current_tokens = sum(
        self.count_tokens(context[i]["content"])
        for i in recent_indices
    )

    for idx, score in sorted_indices:
        if idx in kept_indices:
            continue

```

```

        msg_tokens = self.count_tokens(context[idx]["content"])
        if current_tokens + msg_tokens ≤ target_tokens:
            kept_indices.add(idx)
            current_tokens += msg_tokens
        else:
            break

    # Reconstruct context in original order
    pruned_context = [
        context[i] for i in sorted(kept_indices)
        if i < len(context)
    ]

    return pruned_context

def count_tokens(self, text: str) → int:
    """Count tokens in text."""
    return len(self.encoding.encode(text))

def compress_with_summarization(self, context: List[Dict], llm, max_summary_tokens:
    """Compress by summarizing less important messages."""
    # Identify messages to summarize (not in recent set)
    recent_count = min(self.keep_recent, len(context))
    to_summarize = context[:-recent_count] if recent_count < len(context) else []
    to_keep = context[-recent_count:] if recent_count < len(context) else context

    if not to_summarize:
        return context

    # Create summary of old messages
    summary_text = "\n".join([
        f"{msg['role']}: {msg['content'][:200]}"
        for msg in to_summarize
    ])

    summary_prompt = f"""Create a concise summary of this conversation history,

```

```

        preserving key information, decisions, and context:

        {summary_text}

        Summary (max {max_summary_tokens} tokens):"""

        summary = llm.invoke(summary_prompt).content

        # Combine summary with recent messages
        compressed = [
            {"role": "system", "content": f"Previous conversation summary: {summary}"}
        ] + to_keep

        return compressed

# Usage
pruner = IntelligentContextPruner(max_tokens=100000, keep_recent=20)

# Prune context intelligently
compressed = pruner.prune_context(long_context, target_tokens=80000)

# Or use summarization
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o")
compressed = pruner.compress_with_summarization(long_context, llm)

```

Explanation: This advanced example implements intelligent context pruning that considers message importance, recency, and content type. It prioritizes critical information while removing less important messages, and can also use summarization for more aggressive compression.

Framework-Specific Examples

LANGCHAIN: CONVERSATION SUMMARY MEMORY

```
from langchain_openai import ChatOpenAI
from langchain.memory import ConversationSummaryMemory
from langchain.chains import ConversationChain

llm = ChatOpenAI(model="gpt-4o")

# Summary memory automatically compresses old messages
memory = ConversationSummaryMemory(
    llm=llm,
    max_token_limit=1000, # Target summary size
    return_messages=True
)

chain = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

# Long conversation automatically compressed
response1 = chain.predict(input="Tell me about AI agents")
# ... many turns later ...
response10 = chain.predict(input="What did we discuss earlier?")
# Old messages are summarized, recent ones preserved
```

GOOGLE ADK: SESSION STATE WITH COMPRESSION

```

from google.adk.sessions import InMemorySessionService
from google.adk.agents import LlmAgent
from google.adk.runners import Runner

def compress_session_state(state: dict, max_size: int = 10000) → dict:
    """Compress session state when it exceeds size limit."""
    state_str = str(state)
    if len(state_str) > max_size:
        # Summarize or externalize large state
        # Keep only essential keys
        essential_keys = ["user_id", "current_task", "recent_messages"]
        compressed = {k: state[k] for k in essential_keys if k in state}
        # Externalize rest
        # ... externalization logic ...
        return compressed
    return state

session_service = InMemorySessionService()
session_service.add_compression_hook(compress_session_state)

agent = LlmAgent(
    name="CompressedAgent",
    model="gemini-2.0-flash",
    instruction="Work efficiently within context limits."
)

runner = Runner(
    agent=agent,
    app_name="compressed_app",
    session_service=session_service
)

```

CUSTOM CHUNKING FOR RAG

```

from typing import List
import tiktoken

class DocumentChunker:
    def __init__(self, chunk_size: int = 1000, overlap: int = 200):
        self.chunk_size = chunk_size
        self.overlap = overlap
        self.encoding = tiktoken.encoding_for_model("gpt-4")

    def chunk_text(self, text: str) → List[str]:
        """Split text into overlapping chunks."""
        tokens = self.encoding.encode(text)
        chunks = []

        for i in range(0, len(tokens), self.chunk_size - self.overlap):
            chunk_tokens = tokens[i:i + self.chunk_size]
            chunk_text = self.encoding.decode(chunk_tokens)
            chunks.append(chunk_text)

        return chunks

    def chunk_by_sentences(self, text: str) → List[str]:
        """Chunk by sentences for better semantic coherence."""
        sentences = re.split(r'(?<[.!?])\s+', text)
        chunks = []
        current_chunk = []
        current_size = 0

        for sentence in sentences:
            sent_tokens = len(self.encoding.encode(sentence))
            if current_size + sent_tokens > self.chunk_size and current_chunk:
                chunks.append(' '.join(current_chunk))
                current_chunk = [sentence]
                current_size = sent_tokens

```

```

        else:
            current_chunk.append(sentence)
            current_size += sent_tokens

    if current_chunk:
        chunks.append(' '.join(current_chunk))

    return chunks

# Usage
chunker = DocumentChunker(chunk_size=1000, overlap=200)
chunks = chunker.chunk_by_sentences(large_document)

# Store chunks in vector database
# Retrieve only relevant chunks for context

```

Key Takeaways

- **Core Strategy:** Context compression is essential for managing the finite context window through externalization, summarization, pruning, and attention manipulation.
- **Primary Method:** Externalizing memory (offloading to filesystem/database) is the most powerful compression technique, enabling unlimited information storage with restorable references.
- **Layered Approach:** Combine multiple strategies: externalize large data first, then summarize/prune what remains, and use attention manipulation for critical information.
- **Restorable Compression:** Always maintain references (paths, URLs, keys) when externalizing data to enable precise retrieval when needed.
- **Performance Impact:** Effective compression directly improves cost, latency, and reasoning quality by keeping contexts focused and within optimal token ranges.
- **Common Pitfall:** Aggressive compression that loses critical information or fails to maintain restorable references defeats the purpose. Always preserve essential context and references.
- **Best Practice:** Monitor context token usage and implement automatic compression when approaching limits (e.g., 90% of max tokens) to prevent hard failures.

Related Patterns

This pattern works well with:

- **Leverage External Memory (Filesystem as Context):** Externalization is the primary compression strategy, offloading large data to persistent storage.
- **Persistent Task List (Recitation):** Attention manipulation through recitation keeps important plans visible without consuming context space.
- **Memory Management:** Context compression is a key component of comprehensive memory management, complementing context window optimization and external memory systems.
- **Knowledge Retrieval (RAG):** Chunking enables RAG systems to retrieve only relevant document sections rather than entire documents.

This pattern is often combined with:

- **Stable, Append-Only Context:** Compression helps maintain stable context prefixes for KV-Cache optimization.
- **Tool Result Management:** Large tool results are externalized, with only summaries or references kept in context.
- **Multi-Agent Architectures:** Orchestrators compress subagent outputs, storing details externally and keeping summaries in context.

References

- Agentic AI System Design Patterns
- Context Engineering for AI Agents: Lessons from Building Manus
- LangChain Memory Management: <https://python.langchain.com/docs/modules/memory/>
- Google ADK Sessions: <https://google.github.io/adk-docs/sessions/>
- Context Compression Techniques: <https://arxiv.org/abs/2309.00071>
- Lost in the Middle: How Language Models Use Long Contexts: <https://arxiv.org/abs/2307.03172>

PART VI

Multi-Agent Systems

Scaling up with multiple agents working together

Multi-Agent Architectures

Scaling up: Orchestrator-Workers, Evaluator-Optimizers, and Swarm patterns.

Module ID: module-9

MULTI-AGENT ARCHITECTURES

Introduction

As agentic systems tackle increasingly complex problems, a single agent often reaches its limits. Context overload, lack of specialization, and the complexity of multifaceted objectives can cause even sophisticated agents to fail. Multi-agent architectures address these limitations by enabling specialized agents to collaborate, each focusing on their domain of expertise.

This chapter provides an overview of multi-agent architectures, exploring when and why to use multiple agents, the benefits they provide, and the organizational patterns that enable effective collaboration. For detailed implementation patterns, see the specific pattern modules referenced throughout this chapter.

The Case for Multi-Agent Systems

Multi-agent architectures represent a paradigm shift from pursuing a single, all-powerful super-agent toward sophisticated, collaborative systems. The collective strength lies in division of labor and synergy created through coordinated effort.

Why Single Agents Fail

Even highly capable agents face fundamental limitations:

- **Context Overload:** Complex tasks require more information than fits in a single context window

- **Lack of Specialization:** Generalist agents cannot match the performance of specialized agents in their domains
- **Sequential Bottlenecks:** Tasks that could be parallelized are forced into sequential execution
- **Cognitive Load:** Managing multiple concerns simultaneously reduces performance on each

The Multi-Agent Advantage

Multi-agent systems address these limitations through:

Specialization: Each agent can be optimized for specific domains (research, writing, coding, analysis), resulting in higher quality outputs than a single generalist agent. Specialized agents with domain-specific tools and prompts produce better results than filtering everything through a general coordinator.

Parallelization: Multiple agents can work simultaneously on independent tasks, dramatically reducing execution time. Parallel execution of subagents can reduce research time by up to 90% for complex queries.

Scalability: Multi-agent systems effectively scale token usage and capacity for tasks that exceed single-agent limits. By distributing work across agents with separate context windows, systems add capacity for parallel reasoning.

Information Compression: Subagents facilitate compression by operating in parallel with their own context windows, exploring different aspects simultaneously before condensing the most important information for the lead agent.

Key Organizational Patterns

Multi-agent systems employ several organizational patterns, each suited to different problem types:

Orchestrator-Worker Pattern

A central orchestrator (lead agent/coordinator) breaks high-level goals into sub-tasks and delegates them to specialized worker agents. The orchestrator:

- Analyzes queries and develops strategies
- Dynamically creates specialized subagents with clear objectives
- Coordinates parallel execution
- Synthesizes results from multiple workers

This pattern is detailed in the **Pattern: Orchestrator-Worker (Coordinator)** module.

Example: A research system where a LeadResearcher agent coordinates multiple Subagents exploring different aspects of a question in parallel, then synthesizes their findings.

Evaluator-Optimizer Pattern

An iterative loop where one agent generates solutions and another critiques them, enabling quality improvement through feedback. The generator creates initial outputs, the evaluator provides critique, and the generator refines based on feedback.

This pattern is similar to the **Pattern: Reflection** but involves separate specialized agents rather than a single agent reflecting on itself.

Example: A coding system where a Coder agent writes code, a Reviewer agent critiques it, and the Coder refines based on feedback in an iterative loop.

Swarm Architectures

Autonomous agents that interact to solve problems through consensus, division of labor, or emergent coordination. Agents may compete, collaborate, or divide tasks organically based on their capabilities and current workload.

Example: Multiple research agents that independently explore a problem space, share findings, and converge on solutions through consensus.

Benefits of Multi-Agent Systems

Compression and Information Distillation

The essence of search and research is compression—distilling insights from vast information. Subagents facilitate compression by operating in parallel with their own context windows, exploring different aspects simultaneously before condensing the most important information for the lead agent.

Each subagent provides separation of concerns with distinct tools, prompts, and exploration trajectories, reducing path dependency and enabling thorough, independent investigations.

Parallelization and Speed

Multi-agent systems excel at breadth-first queries involving multiple independent directions. Parallel execution of subagents can reduce research time by up to 90% for complex queries, allowing systems to accomplish in minutes what would take hours sequentially.

This parallelization is critical for tasks that require exploring many sources simultaneously or handling multiple independent sub-tasks.

Scaling Performance

Once intelligence reaches a threshold, multi-agent systems become vital for scaling performance. Even generally-intelligent agents face limits when operating as individuals; groups of agents can accomplish far more.

Internal evaluations show multi-agent systems can outperform single-agent systems by 90%+ on complex research tasks, particularly when tasks require decomposing into parallel subtasks.

Token Usage and Capacity

Multi-agent systems effectively scale token usage for tasks that exceed single-agent limits. Analysis shows that token usage explains 80% of performance variance in complex browsing tasks. By distributing work across agents with separate context windows, systems add capacity for parallel reasoning.

Important Trade-off: Multi-agent systems typically use 15× more tokens than simple chat interactions, requiring tasks with sufficient value to justify the expense.

When to Use Multi-Agent Systems

Multi-agent architectures are most valuable when:

- **Tasks are too complex for a single agent:** Require diverse expertise or multiple distinct capabilities
- **Specialization is needed:** Different aspects require specialized knowledge or skills
- **Quality assurance is critical:** Iterative review and refinement are essential
- **Parallel processing is possible:** Multiple independent sub-tasks can be executed concurrently
- **Context window limitations:** Full context exceeds a single agent's capacity
- **Open-ended problems:** Required steps cannot be predicted in advance

Multi-agent systems are **not** ideal when:

- **Simple tasks:** Can be effectively handled by a single, well-configured agent
- **Tight coupling:** Sub-tasks are so tightly coupled that coordination overhead exceeds benefits
- **Low-latency requirements:** Coordination overhead is prohibitive
- **Resource constraints:** Computational or cost constraints make multiple agents impractical
- **Fixed workflows:** Tasks follow rigid, predetermined sequences

Key Design Considerations

Context Isolation

Each agent operates with its own context window, preventing information overload and enabling parallel exploration. This isolation is critical for managing complexity and enabling true parallelization.

Dynamic Task Decomposition

The orchestrator determines subtasks at runtime based on the input, rather than using fixed workflows. This enables adaptation to unpredictable requirements and open-ended problems.

Coordination Mechanisms

Agents must communicate, share state, and coordinate their actions toward common goals. This requires:

- Clear communication protocols
- Shared state management
- Task delegation mechanisms
- Result synthesis strategies

Specialization Strategy

Effective multi-agent systems require careful design of agent specializations:

- **Domain Expertise:** Each agent focuses on a specific domain (research, writing, coding)
- **Tool Specialization:** Agents have access to domain-specific tools
- **Prompt Optimization:** Specialized prompts tuned for each agent's role

- **Clear Boundaries:** Well-defined responsibilities to avoid duplication and gaps

Integration with Other Capabilities

Multi-agent systems integrate with other agent capabilities:

- **Pattern: Routing** - Orchestrators use routing to delegate tasks to appropriate workers
- **Pattern: Parallelization** - Multiple agents can work concurrently on independent tasks
- **Pattern: Planning** - Orchestrators create plans that guide multi-agent workflows
- **Memory Management** - Shared state and context enable agent coordination
- **Pattern: Inter-Agent Communication (A2A)** - Standardized protocols for agent communication
- **Pattern: Exception Handling** - Robust error handling is critical when multiple agents interact

Key Insights

1. **Multi-agent systems are not always better:** They add significant complexity and cost (15× more tokens). Use them when the benefits of specialization and parallelization justify the overhead.
2. **Specialization is key:** Specialized agents with domain-specific tools and prompts outperform generalist agents filtering everything through a coordinator.
3. **Parallelization drives performance:** The ability to execute multiple agents in parallel can reduce execution time by up to 90% for suitable tasks.
4. **Context isolation enables scale:** Each agent's separate context window adds capacity for parallel reasoning, essential for complex tasks.
5. **Coordination complexity grows rapidly:** Effective multi-agent systems require careful design of communication, state sharing, and task delegation mechanisms.

Next Steps

This chapter provided an overview of multi-agent architectures. For detailed implementation guidance, see:

- **Pattern: Orchestrator-Worker (Coordinator)** - Detailed implementation of the orchestrator-worker pattern

- **Pattern: Reflection** - How evaluator-optimizer patterns work
- **Pattern: Inter-Agent Communication (A2A)** - Protocols for agent communication
- **Pattern: Parallelization** - Techniques for parallel agent execution

Multi-agent architectures enable agents to tackle problems that exceed single-agent capabilities. Understanding when and how to use them is essential for building sophisticated agentic systems that can handle complex, real-world challenges.

Pattern: Orchestrator-Worker (Coordinator)

A central orchestrator dynamically breaks down complex goals into subtasks, delegates to specialized workers, and synthesizes results. The most common pattern for complex multi-agent tasks.

Module ID: module-9a

ORCHESTRATOR- WORKER (COORDINATOR) PATTERN

Motivation

A conductor coordinates an orchestra, assigning parts to different sections while maintaining the overall vision. A project manager breaks down a complex project, delegates specialized tasks to team members, and synthesizes their contributions into a cohesive result. The Orchestrator-Worker pattern mirrors this: a central coordinator breaks down complex goals, delegates to specialized workers, and integrates their outputs into a unified solution.

Pattern Overview

What it is: A central agent, often called the Coordinator or Lead Agent, dynamically breaks down a complex goal into smaller subtasks, delegates them to specialized worker agents, and synthesizes the workers' outputs to produce the final result.

When to use: For complex tasks that cannot be handled by a single agent, especially when the required subtasks are unpredictable and dynamic, rather than fixed. It's the most common pattern for complex tasks.

Why it matters: It enables **specialization** (assigning tasks to dedicated agents with specific skills), **parallelization** (often running subtasks concurrently for speed), and **resilience** (isolating failures to individual agents).

The Orchestrator-Worker pattern, also known as the Coordinator pattern, represents one of the most fundamental and widely-used multi-agent architectures. Unlike rigid, predefined workflows, this pattern enables dynamic task decomposition where the orchestrator agent analyzes the high-level goal and determines the necessary subtasks at runtime. This flexibility makes it particularly powerful for handling complex, unpredictable tasks that require diverse expertise.

The pattern's strength lies in its ability to leverage specialization. Each worker agent can be optimized for a specific domain—research, writing, coding, analysis, or review—resulting in higher quality outputs than a single generalist agent could produce. The orchestrator acts as a strategic coordinator, managing the overall workflow, handling dependencies between subtasks, and synthesizing results into a coherent final output.

This pattern is especially valuable for long-horizon tasks where the orchestrator can maintain high-level context and goals while workers focus on specific execution details. The separation of concerns also enables better context management, as the orchestrator can isolate context for specific agents, preventing information overload and improving efficiency.

Key Concepts

- **Orchestrator (Coordinator/Lead Agent):** The central agent that receives high-level goals, decomposes them into subtasks, delegates to workers, and synthesizes results.
- **Worker Agents:** Specialized agents that execute specific subtasks using domain expertise and specialized tools.
- **Dynamic Task Decomposition:** The orchestrator determines subtasks at runtime based on the input, rather than using fixed workflows.
- **Specialization:** Each worker agent focuses on a specific domain or capability, improving overall system effectiveness.
- **Parallelization:** Independent subtasks can be executed concurrently by different workers, reducing overall latency.

- **Hierarchical Organization:** Workers can themselves become orchestrators for sub-subtasks, creating nested multi-agent structures.
- **Context Isolation:** The orchestrator can manage and isolate context for specific agents, improving efficiency and preventing information overload.

How It Works: Step-by-step Explanation

1. **Receive and Decompose:** The Orchestrator receives a high-level user request. It uses an AI model for reasoning to analyze and dynamically break the request into smaller, manageable pieces (subtasks).
2. **Delegate:** The Orchestrator dispatches each subtask to the most appropriate specialized worker agent. The Orchestrator must provide clear, non-overlapping objectives to the subagents to avoid duplication of work or gaps in coverage.
3. **Execute and Return:** Worker agents execute their specific task, often using specialized tools (e.g., querying a database or calling an API). They return their findings to the Orchestrator.
4. **Synthesize:** The Orchestrator integrates the outputs from all worker agents to compile and return the final, coherent response to the user.

When to Use This Pattern

Use when:

- **Complex, multifaceted tasks:** Tasks that require diverse expertise or multiple distinct capabilities that no single agent can handle effectively.
- **Dynamic task requirements:** When subtasks cannot be predetermined and must be determined at runtime based on the input.
- **Specialization needed:** Different aspects of the task require specialized knowledge or skills (e.g., research, writing, coding, review).
- **Parallel processing possible:** Multiple independent sub-tasks can be executed concurrently by different agents.
- **Long-horizon tasks:** Tasks that span many steps where maintaining high-level context is essential.
- **Context window limitations:** Tasks where the full context exceeds a single agent's context window capacity.
- **Resilience requirements:** When isolating failures to individual agents is important for system reliability.

✗ Avoid when:

- **Simple single-agent tasks:** Tasks that can be effectively handled by a single, well-configured agent.
- **Fixed workflows:** When tasks follow a rigid, predetermined sequence that doesn't benefit from dynamic decomposition.
- **Tight coupling required:** Tasks where sub-tasks are so tightly coupled that coordination overhead exceeds benefits.
- **Low-latency requirements:** When the overhead of multi-agent coordination and communication is prohibitive.
- **Resource constraints:** When computational or cost constraints (increased model calls) make multiple agents impractical.
- **Minimal complexity:** When the added complexity of multi-agent coordination doesn't provide sufficient benefit.

Decision Guidelines

Use the Orchestrator-Worker pattern when the benefits of specialization, parallelization, and dynamic task decomposition outweigh the added complexity and coordination overhead. This pattern is ideal for complex tasks where subtasks are unpredictable and require diverse expertise. Consider: task complexity (complex = orchestrator-worker), specialization needs (diverse expertise = multiple workers), and dynamic requirements (unpredictable = dynamic decomposition). However, be aware of trade-offs: this pattern increases model calls, which raises latency, token throughput, and operational costs compared to a single-agent system. For simple or tightly-coupled tasks, a single agent or simpler workflow may be more efficient.

Practical Applications & Use Cases

The Orchestrator-Worker pattern is the most common pattern for complex agentic tasks, enabling sophisticated systems that can handle multifaceted problems.

- **Anthropic's Research System:** The LeadResearcher agent (Orchestrator) analyzes a complex query, develops a strategy, and spawns multiple specialized Subagents (Workers) in parallel to investigate different aspects. The orchestrator saves its plan to memory before spawning subagents, enabling better context management.

- **Customer Service:** A coordinator agent analyzes a customer's request (e.g., order status, refund, technical support) and routes the task to the appropriate specialized agent (billing specialist, technical support agent, product information agent).
- **Code Generation:** Useful for products that involve complex changes across multiple files, where the Orchestrator determines which files need modification and delegates to specialized coding agents.
- **Research and Report Generation:** An orchestrator breaks down research into sub-topics, delegates to specialized researcher agents who work in parallel, then a writer agent synthesizes findings into a comprehensive report.
- **Content Creation Workflows:** A planner agent creates an outline, writer agents draft sections in parallel, and an editor agent reviews and refines the content for quality and consistency.
- **Scientific Research:** Multiple specialized agents collaborate on hypothesis generation, experimental design, data analysis, and paper writing, with an orchestrator coordinating the overall research process.
- **Multi-file Software Projects:** An orchestrator analyzes requirements, identifies affected files, and delegates changes to specialized agents (frontend, backend, database, testing).

Implementation

Prerequisites

```
pip install langchain langchain-openai langgraph
# or
pip install google-adk
# or
pip install crewai # Multi-agent orchestration framework
```

Basic Example: Orchestrator-Worker Pattern

This example demonstrates a basic orchestrator-worker system where the orchestrator dynamically decomposes tasks and delegates to specialized workers:

```

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Dict
import json

llm = ChatOpenAI(model="gpt-4o", temperature=0)

class OrchestratorState(TypedDict):
    goal: str
    plan: str
    subtasks: List[Dict]
    worker_results: Dict[str, str]
    final_output: str

def orchestrator_decompose(state: OrchestratorState) → OrchestratorState:
    """Orchestrator receives goal and decomposes into subtasks."""
    goal = state["goal"]

    # Use LLM to dynamically create a plan and break down into subtasks
    decomposition_prompt = f"""You are an orchestrator agent. Analyze the following goal and decompose it into subtasks.

Goal: {goal}

For each subtask, determine:
1. The subtask description
2. The type of worker needed (research, write, code, analyze, review)
3. What information is needed as input
4. What output is expected

Return a JSON list of subtasks with keys: description, worker_type, input_needed, expected_output"""

    response = llm.invoke(decomposition_prompt)

    # Parse response (in production, use structured output)
    try:

```

```

        subtasks = json.loads(response.content)
    except:
        # Fallback parsing
        subtasks = [
            {"description": "Research the topic", "worker_type": "research", "input_nee
            {"description": "Write summary", "worker_type": "write", "input_needed": "r
        ]

    # Save plan to state (for recitation pattern)
    plan = f"Goal: {goal}\nSubtasks: {len(subtasks)}\n" + "\n".join([f"- {t['descriptio

    return {
        **state,
        "plan": plan,
        "subtasks": subtasks,
        "worker_results": {}
    }

def research_worker(state: OrchestratorState) → OrchestratorState:
    """Specialized research worker agent."""
    # Get current subtask
    current_subtask = state["subtasks"][0] if state["subtasks"] else None
    if not current_subtask or current_subtask["worker_type"] ≠ "research":
        return state

    research_prompt = f"""You are a research specialist. Conduct research on the follow

{current_subtask['input_needed']]

Provide comprehensive findings with key points, sources, and relevant information."""

    result = llm.invoke(research_prompt)

    worker_results = state.get("worker_results", {})
    worker_results["research"] = result.content

```



```

        # Remove completed subtask
        remaining_subtasks = state["subtasks"][1:]

    return {
        **state,
        "worker_results": worker_results,
        "subtasks": remaining_subtasks
    }

def write_worker(state: OrchestratorState) → OrchestratorState:
    """Specialized writing worker agent."""
    current_subtask = state["subtasks"][0] if state["subtasks"] else None
    if not current_subtask or current_subtask["worker_type"] ≠ "write":
        return state

    # Get research results
    research_results = state["worker_results"].get("research", "")

    write_prompt = f"""You are a writing specialist. Based on the following research, w

Research Findings:
{research_results}

Write a clear, well-structured summary that synthesizes the key information."""

    result = llm.invoke(write_prompt)

    worker_results = state["worker_results"]
    worker_results["write"] = result.content

    remaining_subtasks = state["subtasks"][1:]

    return {
        **state,
        "worker_results": worker_results,
        "subtasks": remaining_subtasks,
    }

```

```

        "final_output": result.content
    }

def orchestrator_synthesize(state: OrchestratorState) → OrchestratorState:
    """Orchestrator synthesizes all worker results into final output."""
    goal = state["goal"]
    worker_results = state["worker_results"]
    plan = state.get("plan", "")

    synthesis_prompt = f"""You are an orchestrator agent. Synthesize the following work

Original Goal: {goal}

Plan:
{plan}

Worker Results:
{json.dumps(worker_results, indent=2)}

Create a comprehensive final output that integrates all worker contributions and direct

    result = llm.invoke(synthesis_prompt)

    return {
        **state,
        "final_output": result.content
    }

def route_to_worker(state: OrchestratorState) → str:
    """Route to appropriate worker based on current subtask."""
    if not state["subtasks"]:
        return "synthesize"

    worker_type = state["subtasks"][0]["worker_type"]
    if worker_type == "research":
        return "research_worker"

```

```

        elif worker_type == "write":
            return "write_worker"
        else:
            return "synthesize"

# Build graph
graph = StateGraph(OrchestratorState)
graph.add_node("orchestrator", orchestrator_decompose)
graph.add_node("research_worker", research_worker)
graph.add_node("write_worker", write_worker)
graph.add_node("synthesize", orchestrator_synthesize)

graph.set_entry_point("orchestrator")
graph.add_conditional_edges("orchestrator", route_to_worker)
graph.add_edge("research_worker", "write_worker")
graph.add_conditional_edges("write_worker", route_to_worker)
graph.add_edge("synthesize", END)

# Execute
result = graph.invoke({"goal": "Create a comprehensive report on renewable energy trend
print(result["final_output"])

```

Explanation: This example demonstrates the core orchestrator-worker pattern: the orchestrator dynamically decomposes the goal into subtasks, delegates to specialized workers (research, writing), and synthesizes the results. The orchestrator maintains the plan and coordinates the workflow, while workers focus on their specialized domains.

Advanced Example: Hierarchical Orchestrator with Context Management

This advanced example shows nested orchestrators and context management using external memory:

```

from pathlib import Path
from typing import Dict, List
import json

class HierarchicalOrchestrator:
    def __init__(self, workspace_dir: str = "./workspace"):
        self.workspace = Path(workspace_dir)
        self.workspace.mkdir(exist_ok=True)
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0)
        self.plan_file = self.workspace / "orchestrator_plan.md"

    def save_plan_to_memory(self, goal: str, plan: str):
        """Save plan to external memory before spawning subagents (context management).
        self.plan_file.write_text(f"# Orchestrator Plan\n\nGoal: {goal}\n\n{plan}")
        return f"Plan saved to {self.plan_file}. Use read_plan() to retrieve."

    def read_plan(self) → str:
        """Read plan from external memory (recitation pattern)."""
        if self.plan_file.exists():
            return self.plan_file.read_text()
        return "No plan found."

    def decompose_with_planning(self, goal: str) → Dict:
        """Orchestrator creates plan and decomposes goal."""
        # Create comprehensive plan
        plan_prompt = f"""You are a lead orchestrator agent. Analyze this goal and crea

Goal: {goal}

Create a detailed plan that includes:
1. High-level strategy
2. Required subtasks
3. Dependencies between subtasks
4. Required worker types
5. Expected outcomes

```

```

Return as structured plan."""

    plan_response = self.llm.invoke(plan_prompt)
    plan = plan_response.content

    # Save plan to memory (context management)
    self.save_plan_to_memory(goal, plan)

    # Decompose into subtasks
    decomposition_prompt = f"""Based on this plan, break down into specific subtask

Plan:
{plan}

Create a list of subtasks with clear, non-overlapping objectives."""

    decomposition_response = self.llm.invoke(decomposition_prompt)

    # Parse subtasks (simplified)
    subtasks = self._parse_subtasks(decomposition_response.content)

    return {
        "goal": goal,
        "plan": plan,
        "subtasks": subtasks
    }

def delegate_to_worker(self, subtask: Dict, context: Dict) → str:
    """Delegate subtask to appropriate worker with isolated context."""
    worker_type = subtask.get("worker_type", "general")

    # Isolate context for this worker (only relevant information)
    worker_context = {
        "subtask": subtask,
        "relevant_info": context.get("relevant_info", ""),
    }

```

```

        "goal": context.get("goal", "")
    }

    # Route to specialized worker
    if worker_type == "research":
        return self._research_worker(worker_context)
    elif worker_type == "write":
        return self._write_worker(worker_context)
    elif worker_type == "code":
        return self._code_worker(worker_context)
    else:
        return self._general_worker(worker_context)

def _research_worker(self, context: Dict) → str:
    """Specialized research worker."""
    prompt = f"""You are a research specialist. Conduct research on:

{context['subtask']['description']}

Goal context: {context['goal']}

Provide comprehensive research findings."""

    result = self.llm.invoke(prompt)
    return result.content

def _write_worker(self, context: Dict) → str:
    """Specialized writing worker."""
    prompt = f"""You are a writing specialist. Write:

{context['subtask']['description']}

Based on: {context.get('relevant_info', '')}

Create well-structured, clear content."""

```

```

        result = self.llm.invoke(prompt)
        return result.content

    def _code_worker(self, context: Dict) → str:
        """Specialized coding worker."""
        prompt = f"""You are a coding specialist. Implement:

{context['subtask']['description']}

Requirements: {context.get('relevant_info', '')}

Provide complete, working code with comments."""

        result = self.llm.invoke(prompt)
        return result.content

    def _general_worker(self, context: Dict) → str:
        """General worker for unspecified tasks."""
        prompt = f"""Execute this task:

{context['subtask']['description']}

Context: {context.get('relevant_info', '')}"""

        result = self.llm.invoke(prompt)
        return result.content

    def synthesize_results(self, goal: str, worker_results: Dict[str, str]) → str:
        """Orchestrator synthesizes all worker results."""
        # Read plan from memory (recitation)
        plan = self.read_plan()

        synthesis_prompt = f"""You are an orchestrator agent. Synthesize worker results

Original Goal: {goal}

```

```

Plan (from memory):
{plan}

Worker Results:
{json.dumps(worker_results, indent=2)}

Create a comprehensive final output that:
1. Directly addresses the original goal
2. Integrates all worker contributions
3. Maintains coherence and quality
4. Follows the strategic plan"""

    result = self.llm.invoke(synthesis_prompt)
    return result.content

def _parse_subtasks(self, content: str) → List[Dict]:
    """Parse subtasks from LLM response (simplified)."""
    # In production, use structured output or better parsing
    lines = content.split('\n')
    subtasks = []
    for line in lines:
        if line.strip() and ('-' in line or line[0].isdigit()):
            subtasks.append({
                "description": line.strip().lstrip('- ').lstrip('0123456789. '),
                "worker_type": "general" # Would be determined by LLM
            })
    return subtasks[:5] # Limit for example

# Usage
orchestrator = HierarchicalOrchestrator()

# Orchestrator decomposes goal
decomposition = orchestrator.decompose_with_planning(
    "Create a comprehensive analysis of AI agent architectures"
)

```



```
# Execute subtasks (can be parallelized)
worker_results = {}
for subtask in decomposition["subtasks"]:
    result = orchestrator.delegate_to_worker(
        subtask,
        {"goal": decomposition["goal"], "relevant_info": ""}
    )
    worker_results[subtask["description"]] = result

# Orchestrator synthesizes
final_output = orchestrator.synthesize_results(
    decomposition["goal"],
    worker_results
)
```

Explanation: This advanced example demonstrates hierarchical orchestrators with context management. The orchestrator saves its plan to external memory before spawning workers (enabling better context management), delegates with isolated context for each worker, and synthesizes results. Workers can themselves become orchestrators for complex subtasks, creating nested structures.

Framework-Specific Examples

GOOGLE ADK: ORCHESTRATOR WITH SUB-AGENTS

```
from google.adk.agents import Agent
from google.adk.runners import Runner

# Define specialized worker agents
researcher = Agent(
    name="Researcher",
    model="gemini-2.0-flash",
    instruction="You are a research specialist. Conduct thorough research on assigned t
    tools=[search_tool, web_scraper_tool]
)

writer = Agent(
    name="Writer",
    model="gemini-2.0-flash",
    instruction="You are a writing specialist. Create clear, well-structured content.",
    tools=[writing_tool, formatting_tool]
)

coder = Agent(
    name="Coder",
    model="gemini-2.0-flash",
    instruction="You are a coding specialist. Write clean, functional code.",
    tools=[code_editor_tool, test_runner_tool]
)

# Create orchestrator (coordinator)
orchestrator = Agent(
    name="Orchestrator",
    model="gemini-2.0-flash",
    instruction="""You are an orchestrator agent that coordinates complex tasks.

Your responsibilities:
```

```

1. Analyze high-level goals and break them into subtasks
2. Delegate subtasks to appropriate worker agents (Researcher, Writer, Coder)
3. Provide clear, non-overlapping objectives to workers
4. Synthesize worker results into final output

Available workers:
- Researcher: For research and information gathering tasks
- Writer: For content creation and writing tasks
- Coder: For coding and software development tasks

Always maintain the high-level goal and ensure worker outputs align with it."""
    sub_agents=[researcher, writer, coder]
)

# Runner executes orchestrator
runner = Runner(
    agent=orchestrator,
    app_name="orchestrator_app"
)

# Orchestrator dynamically decomposes and delegates
result = runner.run("Create a comprehensive guide on agentic AI patterns")

```

LANGGRAPH: DYNAMIC ORCHESTRATION

```

from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Dict
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", temperature=0)

class OrchestratorState(TypedDict):
    goal: str
    plan: str
    subtasks: List[Dict]
    worker_results: Dict[str, str]
    current_worker: str
    final_output: str

def orchestrator_node(state: OrchestratorState) → OrchestratorState:
    """Orchestrator decomposes goal into subtasks."""
    goal = state["goal"]

    # Dynamic decomposition
    prompt = f"Break down this goal into subtasks: {goal}"
    response = llm.invoke(prompt)

    # Parse and create subtasks
    subtasks = parse_subtasks(response.content)

    return {
        **state,
        "plan": response.content,
        "subtasks": subtasks
    }

def worker_node(state: OrchestratorState) → OrchestratorState:
    """Generic worker node that routes to specialized workers."""
    if not state["subtasks"]:

```

```

        return {**state, "current_worker": "done"}

    current_subtask = state["subtasks"][0]
    worker_type = current_subtask.get("type", "general")

    # Execute with specialized prompt
    prompt = create_worker_prompt(worker_type, current_subtask, state["goal"])
    result = llm.invoke(prompt)

    # Store result
    worker_results = state.get("worker_results", {})
    worker_results[current_subtask["id"]] = result.content

    # Remove completed subtask
    remaining = state["subtasks"][1:]

    return {
        **state,
        "worker_results": worker_results,
        "subtasks": remaining,
        "current_worker": worker_type
    }

def synthesize_node(state: OrchestratorState) → OrchestratorState:
    """Orchestrator synthesizes all results."""
    prompt = f"""Synthesize these worker results for goal: {state['goal']}

Results: {json.dumps(state['worker_results'], indent=2)}"""

    result = llm.invoke(prompt)

    return {
        **state,
        "final_output": result.content
    }

```

```
def should_continue(state: OrchestratorState) → str:
    """Determine next step."""
    if state["subtasks"]:
        return "worker"
    elif state.get("final_output"):
        return "end"
    else:
        return "synthesize"

# Build graph
graph = StateGraph(OrchestratorState)
graph.add_node("orchestrator", orchestrator_node)
graph.add_node("worker", worker_node)
graph.add_node("synthesize", synthesize_node)

graph.set_entry_point("orchestrator")
graph.add_edge("orchestrator", "worker")
graph.add_conditional_edges("worker", should_continue)
graph.add_edge("synthesize", END)
```

CREWAI: MULTI-AGENT ORCHESTRATION

```
from crewai import Agent, Task, Crew
from crewai.tools import tool

# Define specialized agents (workers)
researcher = Agent(
    role='Research Specialist',
    goal='Conduct thorough research on assigned topics',
    backstory='You are an expert researcher with deep knowledge in multiple domains.',
    verbose=True
)

writer = Agent(
    role='Writing Specialist',
    goal='Create clear, well-structured content',
    backstory='You are an expert writer skilled at synthesizing information into compelling narratives',
    verbose=True
)

# Define tasks
research_task = Task(
    description='Research the topic: AI agent architectures',
    agent=researcher
)

writing_task = Task(
    description='Write a comprehensive guide based on research findings',
    agent=writer,
    context=[research_task] # Depends on research task
)

# Create crew (orchestrator coordinates)
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
```

```

        verbose=True
    )

    # Execute - orchestrator manages workflow
    result = crew.kickoff()

```

Key Takeaways

- **Core Concept:** The Orchestrator-Worker pattern enables dynamic task decomposition where a central orchestrator breaks down goals into subtasks and delegates to specialized workers.
- **Key Benefits:** Specialization, parallelization, and resilience are the primary advantages, enabling complex tasks that exceed single-agent capabilities.
- **Dynamic Flexibility:** Unlike fixed workflows, the orchestrator determines subtasks at runtime, making it adaptable to unpredictable task requirements.
- **Context Management:** The orchestrator can save plans to memory before spawning workers, enabling better context isolation and management.
- **Trade-offs:** This pattern increases model calls, latency, token throughput, and operational costs compared to single-agent systems. Use when benefits outweigh costs.
- **Best Practice:** Provide clear, non-overlapping objectives to workers to avoid duplication and ensure complete coverage. Maintain the high-level goal throughout execution.
- **Common Pitfall:** Over-coordination can add unnecessary overhead. Ensure workers have clear roles and minimal coupling. Avoid using this pattern for simple tasks that a single agent can handle.
- **Hierarchical Potential:** Workers can themselves become orchestrators for complex subtasks, creating nested multi-agent structures for very complex problems.

Related Patterns

This pattern works well with:

- **Planning:** Orchestrators create plans that guide task decomposition and worker coordination.
- **Persistent Task List (Recitation):** Orchestrators maintain and recite plans to keep high-level goals visible while workers execute subtasks.

- **Leverage External Memory:** Orchestrators save plans to external memory before spawning workers, and workers can store results externally.
- **Context Compression:** Orchestrators compress worker outputs, storing details externally and keeping summaries in context.
- **Routing:** Orchestrators use routing logic to delegate tasks to appropriate workers based on task type and worker capabilities.
- **Parallelization:** Independent subtasks can be executed concurrently by different workers, reducing overall latency.

This pattern is often combined with:

- **Multi-Agent Architectures:** This is the most common pattern within multi-agent systems.
- **Tool Use:** Each worker may have specialized tools for their domain (research tools, coding tools, writing tools).
- **Memory Management:** Shared state and context enable orchestrator-worker coordination.
- **Inter-Agent Communication:** Workers need mechanisms to communicate results back to the orchestrator.

References

- Agentic AI System Design Patterns
 - Anthropic's Research System: LeadResearcher and Subagents Architecture
 - LangGraph Multi-Agent: <https://langchain-ai.github.io/langgraph/how-tos/multi-agent/>
 - Google ADK Agents: <https://google.github.io/adk-docs/agents/>
 - CrewAI Framework: <https://docs.crewai.com/> (Multi-agent orchestration framework)
 - Multi-Agent Systems Research: Academic literature on agent coordination and collaboration
-

PART VII

Advanced Capabilities

Learning, protocols, goal management, and human interaction

Learning and Adaptation

Enabling agents to improve their performance over time through experience, feedback, and adaptive mechanisms.

Module ID: module-11

LEARNING AND ADAPTATION

Introduction

Static agents that operate with fixed parameters and strategies eventually reach their limits. As environments change, new situations arise, and user needs evolve, agents must adapt to remain effective. Learning and adaptation transform static agents into dynamic, evolving systems capable of improving autonomously through experience.

This chapter provides an overview of learning and adaptation approaches for agentic systems. We'll explore different learning paradigms, adaptation mechanisms, and when these capabilities are most valuable. For specific implementation patterns, see the pattern modules referenced throughout this chapter.

The Need for Learning and Adaptation

Agents operating in real-world environments face constant change:

- **Dynamic Environments:** Conditions, requirements, and constraints evolve over time
- **Novel Situations:** Agents encounter scenarios not anticipated during initial design
- **User Preferences:** Individual users have different needs and preferences that change over time
- **Performance Optimization:** Agents can improve their strategies, accuracy, and efficiency through experience

Without learning and adaptation, agents remain rigid, unable to optimize strategies or personalize interactions over time. They cannot handle novel situations or improve from experience.

Learning Paradigms

Reinforcement Learning

Agents learn optimal behaviors through trial and error, receiving rewards for positive outcomes and penalties for negative ones. The agent explores the action space, learns which actions lead to better outcomes, and adjusts its policy accordingly.

Characteristics:

- Requires reward signals to guide learning
- Learns through interaction with the environment
- Can discover optimal strategies through exploration
- Well-suited for sequential decision-making problems

Challenges:

- Requires careful reward design
- Can be sample-inefficient
- May require significant exploration before finding good strategies

Memory-Based Learning

Agents recall past experiences to adjust current actions in similar situations. This enhances context awareness and enables agents to apply lessons learned from previous interactions.

Characteristics:

- Leverages historical interactions
- Enables personalization based on past behavior
- Relatively simple to implement
- Effective for improving user experience over time

Online Learning

Agents continuously update knowledge with new data as it arrives, essential for real-time reactions in dynamic environments. This enables agents to adapt quickly to changing conditions.

Characteristics:

- Adapts in real-time as new information arrives
- No separate training phase required
- Enables rapid response to environmental changes
- Well-suited for streaming data scenarios

Self-Modification

Advanced agents can modify their own code or strategies based on performance feedback, enabling autonomous improvement. Systems like the Self-Improving Coding Agent (SICA) demonstrate this capability.

Characteristics:

- Agents can improve their own implementation
- Enables autonomous capability enhancement
- Requires robust testing and validation
- High potential but also high risk

Evolutionary Algorithms

Systems use evolutionary frameworks to generate, evaluate, and select improved solutions iteratively. LLM-based systems like AlphaEvolve use this approach to discover new and more efficient solutions.

Characteristics:

- Explores solution space through variation and selection
- Can discover novel approaches
- Computationally expensive
- Effective for optimization problems

Adaptation Mechanisms

Adaptation is the visible change in an agent's behavior or knowledge that comes from learning. Agents adapt by:

Changing Strategy: Adjusting their approach based on what works and what doesn't

Updating Understanding: Incorporating new information and correcting misconceptions

Modifying Goals: Adjusting objectives based on changing requirements or constraints

Personalizing Behavior: Tailoring interactions based on individual user preferences and history

When Learning and Adaptation Are Valuable

Learning and adaptation are most valuable when:

- **Dynamic Environments:** The agent operates in unpredictable, changing conditions
- **Personalization Needed:** The agent must tailor interactions to individual users
- **Performance Optimization:** The agent needs to improve over time
- **Novel Situation Handling:** The agent encounters unanticipated scenarios
- **Long-Term Operation:** The agent operates over extended periods where learning accumulates

Learning and adaptation are **not** ideal when:

- **Static, Well-Defined Tasks:** The task is fixed and doesn't benefit from learning
- **Deterministic Requirements:** The system requires guaranteed, consistent behavior
- **Limited Data:** Insufficient data or feedback prevents meaningful learning
- **Security-Critical Systems:** Learning introduces unpredictability that may violate security guarantees
- **Simple, One-Shot Tasks:** The task completes in a single interaction

Key Design Considerations

Feedback Mechanisms

Effective learning requires quality feedback:

- **Reward Signals:** Clear indicators of success and failure
- **User Feedback:** Explicit or implicit signals from users
- **Performance Metrics:** Objective measures of agent effectiveness
- **Error Signals:** Information about what went wrong and why

Evaluation and Safety

Learning systems require robust evaluation and safety mechanisms:

- **Performance Monitoring:** Track whether learning is improving or degrading performance
- **Safety Constraints:** Prevent harmful adaptations
- **Validation:** Test adaptations before deploying them
- **Rollback Mechanisms:** Ability to revert to previous versions if needed

Balance with Stability

Learning must balance improvement with stability:

- **Gradual Updates:** Small, careful changes rather than dramatic shifts
- **Trust Regions:** Constrain updates to maintain reliable behavior
- **Hybrid Approaches:** Combine learning with fixed, reliable fallbacks
- **Conservative Strategies:** Prefer proven approaches over experimental ones in critical scenarios

Integration with Other Capabilities

Learning and adaptation integrate with other agent capabilities:

- **Evaluation and Monitoring:** Learning requires evaluation to determine what to learn
- **Memory Management:** Learning relies on memory to store experiences and knowledge

- **Reflection:** Agents can learn by reflecting on their own performance
- **Goal Setting and Monitoring:** Learning helps agents achieve goals more effectively over time
- **Human-in-the-Loop:** Human feedback can guide learning processes

Key Insights

1. **Learning is not always necessary:** Many agents operate effectively with fixed strategies. Only add learning when the benefits justify the complexity.
2. **Feedback quality determines learning success:** Poor feedback leads to poor learning. Invest in high-quality feedback mechanisms.
3. **Safety is critical:** Learning can lead to harmful adaptations. Implement robust safety mechanisms and validation.
4. **Balance improvement with stability:** Agents must improve while maintaining reliable behavior. Gradual, validated updates are safer than dramatic changes.
5. **Evaluation is essential:** Without evaluation, you cannot determine if learning is helping or hurting. Continuous monitoring is critical.

Next Steps

This chapter provided an overview of learning and adaptation concepts. For detailed implementation guidance, see:

- **Evaluation and Monitoring** - How to evaluate agent performance and guide learning
- **Memory Management** - How to store experiences and knowledge for learning
- **Pattern: Reflection** - How agents can learn by reflecting on their performance
- **Pattern: Goal Setting and Monitoring** - How learning helps achieve goals

Learning and adaptation enable agents to improve over time and adapt to changing conditions. Understanding when and how to implement these capabilities is essential for building agents that remain effective as environments evolve.

Model Context Protocol (MCP)

A standardized protocol for agents to discover, access, and interact with external tools and data sources.

Module ID: module-12

MODEL CONTEXT PROTOCOL (MCP)

Introduction

To enable LLMs to function effectively as agents, their capabilities must extend beyond text generation. Interaction with the external environment is necessary, including access to current data, utilization of external software, and execution of specific operational tasks. The Model Context Protocol (MCP) provides a standardized interface for LLMs to discover, communicate with, and utilize external resources, tools, and data sources.

This chapter provides an overview of MCP, exploring how it enables standardized integration between LLMs and external systems. We'll discuss the protocol's architecture, key concepts, and when it's most valuable. For specific implementation patterns, see the pattern modules referenced throughout this chapter.

What is MCP?

Model Context Protocol (MCP) is an open standard that provides a standardized interface for LLMs to discover, communicate with, and utilize external resources, tools, and data sources through a client-server architecture.

Key Characteristics:

- **Open Standard:** Promotes interoperability across different LLM providers and tools

- **Client-Server Architecture:** LLM applications (clients) connect to MCP servers that expose capabilities
- **Standardized Interface:** Universal adapter that allows any LLM to plug into any external system
- **Dynamic Discovery:** Clients can query servers to learn available capabilities without redeployment

Why MCP Matters

MCP addresses the need for standardized integration between LLMs and external systems:

Before MCP: Each integration required custom code for each LLM provider and each external system, creating a combinatorial explosion of integration complexity.

With MCP: A single standardized protocol enables any compliant LLM to access any compliant tool or resource, dramatically reducing integration complexity and promoting an ecosystem of reusable components.

MCP Architecture

MCP operates on a client-server architecture:

MCP Servers

Servers expose capabilities to LLMs:

- **Tools:** Executable functions that perform actions (e.g., `send_email`, `query_database`)
- **Resources:** Static data that can be read (e.g., PDF files, database records)
- **Prompts:** Templates that guide LLM interaction

MCP Clients

Clients consume server capabilities:

- **LLM Host Applications:** Applications that host LLM interactions
- **AI Agents:** Agents themselves can act as MCP clients
- **Discovery:** Clients query servers to learn available capabilities

Transport Mechanisms

MCP supports multiple transport mechanisms:

- **JSON-RPC over STDIO:** For local interactions (fast, secure)
- **Streamable HTTP/SSE:** For remote connections (scalable, distributed)

Key Concepts

Discovery

MCP clients can dynamically query servers to learn what capabilities they offer. This enables “just-in-time” discovery without redeployment. Agents can discover new tools and resources as they become available.

Standardization

MCP provides an open, standardized protocol promoting interoperability. Any compliant tool can be accessed by any compliant LLM, creating an ecosystem of reusable components.

Resources vs. Tools vs. Prompts

MCP defines three main component types:

- **Resources:** Static data (e.g., PDF files, database records) that can be read
- **Tools:** Executable functions (e.g., `send_email`, `query_API`) that perform actions
- **Prompts:** Templates that guide LLM interaction and provide structured input formats

Agent-Friendly API Design

MCP’s effectiveness depends heavily on the design of the underlying APIs it exposes. Developers must consider not just the connection, but the nature of the data being exchanged:

- **Text-Based Returns:** APIs should return text (Markdown, JSON) rather than binary formats (PDFs, images) that agents cannot parse
- **Filtering and Sorting:** APIs should support filtering and sorting to enable efficient agent queries
- **Structured Responses:** Well-structured responses enable agents to process information effectively

How MCP Works

MCP operates through a structured interaction flow:

1. **Discovery:** The MCP client queries a server to learn available capabilities
2. **Request Formulation:** The LLM determines it needs a tool/resource and formulates a request
3. **Client Communication:** The MCP client sends a standardized call to the appropriate server
4. **Server Execution:** The server authenticates, validates, and executes the action by interfacing with underlying software
5. **Response and Context Update:** The server sends a standardized response back, updating the LLM's context

When to Use MCP

MCP is most valuable when:

- **Multiple External Integrations:** You need to connect LLMs to various external systems, databases, or APIs
- **Interoperability Required:** You want tools and resources to work across different LLM providers and applications
- **Dynamic Capability Discovery:** Your agent needs to discover and use new tools without redeployment
- **Composability:** You want to combine multiple tools and services into complex workflows
- **Reusability:** You want to create tools that can be used by any compliant LLM application

MCP may be less valuable when:

- **Simple, Single Integration:** You only need to connect to one external system and don't need standardization
- **Proprietary Requirements:** Your use case requires vendor-specific features not supported by MCP
- **Performance-Critical:** The protocol overhead adds unacceptable latency for real-time applications
- **Minimal External Needs:** Your agent doesn't need to interact with external systems beyond basic function calling

Key Design Principles

Agent-Friendly APIs

When designing APIs for MCP, consider agent needs:

- **Text-Based Formats:** Return text (Markdown, JSON) rather than binary formats
- **Structured Responses:** Well-structured data enables effective agent processing
- **Filtering and Sorting:** Support efficient querying and data selection
- **Clear Documentation:** Comprehensive descriptions help agents use tools effectively

Standardization Benefits

MCP's value comes from standardization:

- **Reduced Integration Complexity:** One protocol instead of many custom integrations
- **Ecosystem Development:** Reusable tools and resources that work across systems
- **Interoperability:** Tools work with any compliant LLM
- **Composability:** Combine multiple tools into complex workflows

Integration with Other Capabilities

MCP integrates with other agent capabilities:

- **Pattern: Tool Use & Execution** - MCP provides a standardized way to expose tools
- **Pattern: Knowledge Retrieval (RAG)** - MCP can expose RAG systems as resources
- **Pattern: Inter-Agent Communication (A2A)** - MCP can facilitate agent-to-agent communication
- **Multi-Agent Architectures** - MCP enables agents to discover and use specialized tools

Key Insights

1. **MCP enables an ecosystem:** Standardization creates reusable tools and resources that work across different LLMs and applications.

2. **API design matters:** Wrapping legacy APIs without modification may be suboptimal. Design APIs with agent needs in mind.
3. **Discovery enables flexibility:** Dynamic discovery allows agents to adapt to new capabilities without redeployment.
4. **Text-based formats are essential:** Agents cannot parse binary formats. APIs should return text (Markdown, JSON) for effective agent use.
5. **Standardization reduces complexity:** One protocol eliminates the need for custom integrations for each LLM-tool combination.

Next Steps

This chapter provided an overview of MCP concepts. For detailed implementation guidance, see:

- **Pattern: Tool Use & Execution** - How agents use tools, including MCP-exposed tools
- **Pattern: Knowledge Retrieval (RAG)** - How MCP can expose knowledge bases as resources
- **Pattern: Inter-Agent Communication (A2A)** - How MCP facilitates agent communication

MCP provides a standardized foundation for connecting LLMs to external systems. Understanding this protocol enables you to build agents that can seamlessly integrate with diverse tools and resources in a standardized, interoperable way.

Goal Setting and Monitoring

Establishing clear objectives and tracking progress toward goals, enabling agents to measure success and adapt strategies.

Module ID: module-13

GOAL SETTING AND MONITORING

Introduction

For AI agents to be truly effective and purposeful, they need more than just the ability to process information or use tools—they need a clear sense of direction and a way to know if they’re actually succeeding. Goal setting and monitoring transform simple reactive agents into proactive, goal-oriented systems capable of autonomous and reliable operation.

This chapter provides an overview of goal setting and monitoring approaches for agentic systems. We’ll explore how to define effective goals, monitor progress, and enable agents to adapt when goals aren’t being met. For specific implementation patterns, see the pattern modules referenced throughout this chapter.

The Challenge of Goal-Oriented Behavior

AI agents often lack a clear direction, preventing them from acting with purpose beyond simple, reactive tasks. Without defined objectives, they cannot:

- Independently tackle complex, multi-step problems
- Orchestrate sophisticated workflows

- Determine if their actions are leading to successful outcomes
- Adapt when conditions change or obstacles arise

This limits their autonomy and prevents them from being truly effective in dynamic, real-world scenarios.

Effective Goal Setting

SMART Goals

Goals should be **Specific, Measurable, Achievable, Relevant, and Time-bound**:

- **Specific:** Clear, unambiguous objectives rather than vague aspirations
- **Measurable:** Quantifiable criteria for success
- **Achievable:** Realistic given agent capabilities and constraints
- **Relevant:** Aligned with overall system objectives
- **Time-bound:** Clear deadlines or timeframes

Goal Hierarchy

Complex systems often require goal hierarchies:

- **High-Level Goals:** Strategic objectives (e.g., “Improve customer satisfaction”)
- **Mid-Level Goals:** Tactical objectives (e.g., “Reduce response time to under 2 minutes”)
- **Low-Level Goals:** Operational objectives (e.g., “Answer this specific customer query”)

Agents work from high-level goals down to specific actions, maintaining alignment with overall objectives.

Success Criteria

Clearly defined metrics and thresholds determine when goals are met:

- **Quantitative Metrics:** Numerical measures (accuracy, latency, cost)
- **Qualitative Criteria:** Subjective measures (user satisfaction, content quality)
- **Thresholds:** Specific values that indicate success or failure
- **Multi-Criteria:** Goals often have multiple success criteria that must all be met

Monitoring Progress

Continuous Observation

Monitoring involves continuously observing:

- **Agent Actions:** What the agent is doing and why
- **Environmental States:** Current conditions and context
- **Tool Outputs:** Results from tool executions
- **Progress Metrics:** Measurable indicators of goal progress

Progress Tracking

Effective monitoring tracks progress against goals:

- **Current State:** Where the agent is now
- **Target State:** Where the agent needs to be
- **Gap Analysis:** The difference between current and target
- **Trend Analysis:** Whether progress is improving or degrading

Feedback Loops

Monitoring creates feedback loops that enable adaptation:

- **Assessment:** Evaluate whether goals are being met
- **Detection:** Identify when progress deviates from expectations
- **Adaptation:** Adjust plans or strategies when needed
- **Verification:** Confirm that adaptations are working

Adaptive Behavior

When monitoring indicates goals aren't being met, agents must adapt:

Plan Revision

Agents revise their plans when current approaches aren't working:

- **Identify Issues:** Understand why progress is off-track
- **Generate Alternatives:** Develop new approaches
- **Select Best Option:** Choose the most promising alternative
- **Update Plan:** Modify the execution plan accordingly

Strategy Adjustment

Agents adjust their strategies based on monitoring feedback:

- **Change Approach:** Try different methods or techniques
- **Reallocate Resources:** Shift focus to higher-priority areas
- **Modify Constraints:** Adjust limitations or boundaries
- **Escalate Issues:** Request human intervention when needed

Goal Refinement

Sometimes goals themselves need adjustment:

- **Clarify Ambiguity:** Make vague goals more specific
- **Update Priorities:** Shift focus based on new information
- **Revise Deadlines:** Adjust timeframes when necessary
- **Split Complex Goals:** Break large goals into smaller, manageable ones

When Goal Setting and Monitoring Are Valuable

Goal setting and monitoring are most valuable when:

- **Multi-Step Tasks:** The agent must execute complex, coordinated tasks
- **Autonomous Operation:** The agent needs to operate independently
- **Dynamic Environments:** Conditions change and the agent must adapt
- **Reliability Requirements:** The agent must reliably achieve specific outcomes

- **Progress Visibility:** You need visibility into agent progress and goal achievement

They are **not** ideal when:

- **Simple, Single-Step Tasks:** The task completes in one action
- **Reactive-Only Systems:** The agent only responds to immediate inputs
- **Fixed Workflows:** The solution path is predetermined
- **No Success Criteria:** There are no clear metrics to determine achievement

Implementation Approaches

Framework-Based Approaches

Many frameworks provide built-in support for goals:

Google ADK: Goals are often conveyed through agent instructions, with monitoring accomplished through state management and tool interactions.

LangChain/LangGraph: Goals can be embedded in agent prompts and state, with monitoring through state observation and callback mechanisms.

Custom Implementation

Custom implementations provide more control:

- **Goal Representation:** Structured data structures for goals and success criteria
- **Monitoring Systems:** Custom tracking and evaluation mechanisms
- **Adaptation Logic:** Agent-specific strategies for responding to monitoring feedback

Integration with Other Capabilities

Goal setting and monitoring integrate with other agent capabilities:

- **Pattern: Planning** - Goals drive plan generation and execution
- **Pattern: Reflection** - Monitoring enables agents to reflect on progress
- **Evaluation and Monitoring** - Goal achievement is a key evaluation metric

- **Memory Management** - Goals and progress are stored in memory
- **Pattern: Exception Handling** - Monitoring detects when goals can't be met and triggers error handling

Key Insights

1. **Clear goals enable autonomy:** Well-defined goals allow agents to operate independently while maintaining direction.
2. **Monitoring enables adaptation:** Without monitoring, agents cannot know if they're succeeding or need to adjust.
3. **SMART goals are essential:** Vague or unmeasurable goals prevent effective monitoring and adaptation.
4. **Feedback loops are critical:** Monitoring must create feedback that drives adaptation, not just observation.
5. **Balance autonomy with oversight:** Agents need enough autonomy to pursue goals effectively, but enough oversight to ensure they stay on track.

Next Steps

This chapter provided an overview of goal setting and monitoring concepts. For detailed implementation guidance, see:

- **Pattern: Planning** - How agents create plans to achieve goals
- **Pattern: Reflection** - How agents evaluate progress toward goals
- **Evaluation and Monitoring** - How to measure goal achievement
- **Memory Management** - How to store and track goals over time

Goal setting and monitoring are essential for building autonomous, goal-oriented agentic systems. Understanding these concepts enables you to build agents that can operate independently while reliably achieving their objectives.

Pattern: Exception Handling and Recovery

Robust error handling and recovery mechanisms that enable agents to gracefully handle failures and continue operation.

Module ID: module-14

EXCEPTION HANDLING AND RECOVERY

Motivation

When a plan fails—a restaurant is closed, a flight is delayed, or a tool breaks—humans adapt. We find alternatives, adjust expectations, and continue toward our goal. We build resilience by having backup plans and learning from mistakes. Exception Handling gives agents this same resilience: gracefully handling failures, recovering from errors, and adapting strategies when things go wrong, just as humans do in everyday life.

Pattern Overview

What it is: Exception Handling and Recovery is a pattern that equips AI agents with the capability to anticipate, detect, manage, and recover from operational failures, ensuring robust and resilient operation in unpredictable environments.

When to use: Use this pattern for any AI agent deployed in a dynamic, real-world environment where system failures, tool errors, network issues, or unpredictable inputs are possible and operational reliability is a key requirement.

Why it matters: For AI agents to operate reliably in diverse real-world environments, they must be able to manage unforeseen situations, errors, and malfunctions. Just as humans adapt to unexpected obstacles, intelligent agents need robust systems to detect problems, initiate recovery procedures, or at least ensure controlled failure. This essential requirement ensures agents are not only intelligent but also stable and reliable.

AI agents operating in real-world environments inevitably encounter unforeseen situations, errors, and system malfunctions. These disruptions can range from tool failures and network issues to invalid data, threatening the agent's ability to complete its tasks. Without a structured way to manage these problems, agents can be fragile, unreliable, and prone to complete failure when faced with unexpected hurdles. This unreliability makes it difficult to deploy them in critical or complex applications where consistent performance is essential.

The Exception Handling and Recovery pattern provides a standardized solution for building robust and resilient AI agents. It equips them with the capability to anticipate, manage, and recover from operational failures. The pattern involves proactive error detection, such as monitoring tool outputs and API responses, and reactive handling strategies like logging for diagnostics, retrying transient failures, or using fallback mechanisms. For more severe issues, it defines recovery protocols, including reverting to a stable state, self-correction by adjusting its plan, or escalating the problem to a human operator.

This pattern may sometimes be used with reflection. For example, if an initial attempt fails and raises an exception, a reflective process can analyze the failure and reattempt the task with a refined approach, such as an improved prompt, to resolve the error.

Key Concepts

- **Error Detection:** Meticulously identifying operational issues as they arise, including invalid tool outputs, API errors, timeouts, or incoherent responses.
- **Error Handling:** Response plans including logging, retries, fallbacks, graceful degradation, and notifications.
- **Recovery:** Restoring the agent to stable operation through state rollback, diagnosis, self-correction, or escalation.
- **Proactive Preparation:** Anticipating potential issues and developing strategies to mitigate them before they occur.
- **Reactive Strategies:** Responding to errors as they occur with appropriate handling mechanisms.

How It Works

Exception Handling and Recovery operates through a three-stage process: (1) Error Detection—the system identifies operational issues through validation, monitoring, or anomaly detection, (2) Error Handling—once detected, errors are handled through logging, retries, fallbacks, graceful degradation, or notifications, (3) Recovery—the system restores stable operation through state rollback, diagnosis, self-correction, replanning, or escalation to human operators.

Error detection involves validating tool outputs, checking API error codes, monitoring response times, and identifying incoherent responses. Monitoring by other agents or specialized monitoring systems enables proactive anomaly detection. Error handling strategies include logging for debugging, retrying with adjusted parameters for transient errors, using alternative strategies (fallbacks), maintaining partial functionality (graceful degradation), and alerting human operators (notifications). Recovery mechanisms include state rollback to undo error effects, diagnosis to investigate causes, self-correction through plan adjustment, and escalation for complex or severe cases.

When to Use This Pattern

Use this pattern when:

- **Real-world deployment:** The agent operates in environments where perfect conditions cannot be guaranteed.
- **External dependencies:** The agent relies on external services, APIs, or tools that may fail.
- **Critical operations:** Failures could have significant consequences requiring robust error handling.
- **Unpredictable inputs:** The agent receives inputs that may be invalid, malformed, or unexpected.
- **Network operations:** The agent performs network operations subject to connectivity issues or timeouts.

Avoid this pattern when:

- **Controlled environments:** The agent operates in highly controlled, predictable environments with guaranteed reliability.
- **Simple, stateless operations:** The agent performs simple operations without external dependencies or state.
- **Prototype/testing:** Early prototypes where error handling adds unnecessary complexity.

- **Deterministic workflows:** Fixed workflows with guaranteed success paths don't need exception handling.

Decision Guidelines

Use Exception Handling and Recovery when the benefits of robustness and reliability outweigh the implementation complexity. Consider: the criticality of operations (more critical = more need for handling), the reliability of dependencies (less reliable = more need for handling), and the cost of failures (higher cost = more need for handling). Be aware that exception handling adds complexity and overhead, but is essential for production systems. Implement comprehensive error detection, logging, and recovery mechanisms to ensure reliable operation.

Practical Applications & Use Cases

Exception Handling and Recovery is critical for any agent deployed in a real-world scenario where perfect conditions cannot be guaranteed.

- **Customer Service Chatbots:** Handle database downtime by detecting API errors, informing users, and escalating to human agents.
- **Automated Financial Trading:** Manage “insufficient funds” or “market closed” errors by logging, avoiding repeated invalid trades, and notifying users.
- **Smart Home Automation:** Detect device failures, retry operations, and notify users when manual intervention is needed.
- **Data Processing Agents:** Skip corrupted files, log errors, continue processing, and report skipped files rather than halting entirely.
- **Web Scraping Agents:** Handle CAPTCHAs, changed website structures, or server errors by pausing, using proxies, or reporting failures.
- **Robotics and Manufacturing:** Detect sensor failures, attempt readjustment, retry operations, and alert human operators when persistent.

Implementation

Prerequisites

```
pip install langchain langchain-openai  
# or  
pip install google-adk
```

Basic Example

```

from langchain_openai import ChatOpenAI
from typing import Dict, Optional
import time
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class RobustAgent:
    def __init__(self, max_retries: int = 3):
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0)
        self.max_retries = max_retries

    def execute_with_retry(self, tool_call: str, params: Dict) → Optional[Dict]:
        """Execute tool call with retry logic."""
        for attempt in range(self.max_retries):
            try:
                result = self._execute_tool(tool_call, params)
                if self._validate_result(result):
                    return result
            except Exception as e:
                logger.error(f"Error on attempt {attempt + 1}: {e}")
                if attempt < self.max_retries - 1:
                    time.sleep(2 ** attempt) # Exponential backoff
                else:
                    return self._fallback_operation(tool_call, params)
        return None

    def _execute_tool(self, tool_call: str, params: Dict) → Dict:
        """Execute tool (simulated)."""
        # In production, actual tool execution

```

```
        if "error" in params.get("simulate", ""):
            raise Exception("Simulated error")
        return {"status": "success", "data": "result"}

    def _validate_result(self, result: Dict) → bool:
        """Validate tool result."""
        return result.get("status") == "success" and "data" in result

    def _fallback_operation(self, tool_call: str, params: Dict) → Dict:
        """Fallback operation when primary fails."""
        logger.info(f"Using fallback for {tool_call}")
        return {"status": "fallback", "message": "Used alternative method"}

# Usage
agent = RobustAgent()
result = agent.execute_with_retry("database_query", {"query": "SELECT * FROM users"})
```

Explanation: This example demonstrates basic exception handling with retry logic, validation, and fallback mechanisms. The agent retries failed operations with exponential backoff, validates results, and falls back to alternative operations when primary methods fail. This ensures robust operation despite transient failures.

Advanced Example

```

from langchain_openai import ChatOpenAI
from typing import Dict, List, Optional, Callable
import json
import logging
from enum import Enum

class ErrorSeverity(Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"

class ExceptionHandler:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0)
        self.error_log = []
        self.recovery_strategies = {}

    def handle_exception(self, error: Exception, context: Dict) → Dict:
        """Comprehensive exception handling."""
        # Log error
        error_entry = {
            "error": str(error),
            "type": type(error).__name__,
            "context": context,
            "timestamp": time.time(),
            "severity": self._assess_severity(error, context)
        }
        self.error_log.append(error_entry)
        logger.error(f"Exception: {error_entry}")

        # Determine recovery strategy
        strategy = self._determine_recovery_strategy(error, context)

```

```

        # Execute recovery
        recovery_result = self._execute_recovery(strategy, error, context)

    return {
        "error": error_entry,
        "strategy": strategy,
        "recovery": recovery_result
    }

def _assess_severity(self, error: Exception, context: Dict) → ErrorSeverity:
    """Assess error severity."""
    error_str = str(error).lower()

    if "critical" in error_str or "fatal" in error_str:
        return ErrorSeverity.CRITICAL
    elif "timeout" in error_str or "connection" in error_str:
        return ErrorSeverity.HIGH
    elif "validation" in error_str or "format" in error_str:
        return ErrorSeverity.MEDIUM
    else:
        return ErrorSeverity.LOW

def _determine_recovery_strategy(self, error: Exception, context: Dict) → str:
    """Determine appropriate recovery strategy."""
    error_type = type(error).__name__
    error_str = str(error).lower()

    # Retry for transient errors
    if "timeout" in error_str or "connection" in error_str:
        return "retry"

    # Fallback for service errors
    if "service" in error_str or "api" in error_str:
        return "fallback"

    # Self-correction for logic errors

```

```

        if "validation" in error_str or "invalid" in error_str:
            return "self_correct"

        # Escalation for critical errors
        if self._assess_severity(error, context) == ErrorSeverity.CRITICAL:
            return "escalate"

        return "log_and_continue"

def _execute_recovery(self, strategy: str, error: Exception, context: Dict) → Dict:
    """Execute recovery strategy."""
    if strategy == "retry":
        return self._retry_operation(context)
    elif strategy == "fallback":
        return self._fallback_operation(context)
    elif strategy == "self_correct":
        return self._self_correct(context)
    elif strategy == "escalate":
        return self._escalate_to_human(error, context)
    else:
        return {"status": "logged", "action": "continue"}

def _retry_operation(self, context: Dict) → Dict:
    """Retry failed operation."""
    max_retries = context.get("max_retries", 3)
    for i in range(max_retries):
        try:
            # Retry logic
            return {"status": "retried", "attempt": i + 1}
        except Exception as e:
            if i == max_retries - 1:
                return {"status": "retry_failed", "error": str(e)}
            time.sleep(2 ** i)
    return {"status": "retry_exhausted"}

def _fallback_operation(self, context: Dict) → Dict:

```



```

        """Use fallback operation."""
        fallback = context.get("fallback")
        if fallback:
            return {"status": "fallback_used", "method": fallback}
        return {"status": "no_fallback_available"}

def _self_correct(self, context: Dict) → Dict:
    """Self-correct based on error analysis."""
    prompt = f"""Analyze this error and suggest correction:
    Error: {context.get('error')}
    Context: {json.dumps(context)}
    Provide corrected approach."""

    response = self.llm.invoke(prompt)
    return {"status": "self_corrected", "correction": response.content}

def _escalate_to_human(self, error: Exception, context: Dict) → Dict:
    """Escalate to human operator."""
    # In production, send to human queue
    return {
        "status": "escalated",
        "message": f"Critical error escalated: {error}",
        "context": context
    }

# Usage
handler = ExceptionHandler()
try:
    # Operation that might fail
    result = risky_operation()
except Exception as e:
    recovery = handler.handle_exception(e, {"operation": "risky_operation"})
    print(f"Recovery: {recovery}")
    
```

Explanation: This advanced example implements comprehensive exception handling with severity assessment, strategy determination, and multiple recovery mechanisms. It demonstrates production-ready error handling with logging, retry logic, fallbacks, self-correction, and escalation capabilities.

Framework-Specific Examples

GOOGLE ADK SEQUENTIAL AGENT WITH FALLBACK

```
from google.adk.agents import Agent, SequentialAgent

# Primary handler
primary_handler = Agent(
    name="primary_handler",
    model="gemini-2.0-flash",
    instruction="Use get_precise_location_info tool with user's address.",
    tools=[get_precise_location_info]
)

# Fallback handler
fallback_handler = Agent(
    name="fallback_handler",
    model="gemini-2.0-flash",
    instruction="""Check state['primary_location_failed'].
    If True, use get_general_area_info tool.
    If False, do nothing.""",
    tools=[get_general_area_info]
)

# Response agent
response_agent = Agent(
    name="response_agent",
    model="gemini-2.0-flash",
    instruction="Present location info from state['location_result'].",
    tools=[]
)

# Sequential agent with fallback
robust_agent = SequentialAgent(
```

```

    name="robust_location_agent",
    sub_agents=[primary_handler, fallback_handler, response_agent]
)

```

LANGCHAIN WITH ERROR HANDLING

```

from langchain.agents import AgentExecutor
from langchain_openai import ChatOpenAI

def safe_execute(agent_executor, input_data):
    """Execute agent with error handling."""
    try:
        return agent_executor.invoke(input_data)
    except Exception as e:
        logger.error(f"Agent execution failed: {e}")
        # Fallback response
        return {
            "output": "I encountered an error. Let me try an alternative approach.",
            "error": str(e)
        }

```

Key Takeaways

- **Core Concept:** Exception Handling and Recovery is essential for building robust and reliable agents that can operate effectively in unpredictable environments.
- **Best Practice:** Implement comprehensive error detection, logging, retry logic, fallbacks, and recovery mechanisms for production systems.
- **Common Pitfall:** Failing to handle exceptions leads to fragile agents that crash on unexpected errors; always implement error handling.
- **Performance Note:** Exception handling adds overhead but is essential for reliability; optimize detection and recovery paths for performance.

Related Patterns

This pattern works well with:

- **Reflection** - Exception handling can trigger reflective analysis to improve future attempts
- **Human-in-the-Loop** - Critical errors can be escalated to human operators
- **Goal Setting and Monitoring** - Exception handling ensures agents can recover and continue toward goals

This pattern is often combined with:

- **Tool Use** - Tool failures require exception handling and recovery
- **Planning** - Exceptions may trigger plan revision and replanning

References

- Code Complete by Steve McConnell
 - Fault Tolerance in Multi-Agent Systems: <https://arxiv.org/abs/2412.00534>
 - Google ADK Agents: <https://google.github.io/adk-docs/agents/>
-

Pattern: Human-in-the-Loop

Integrating human oversight, feedback, and decision-making into agent workflows for safety, quality, and trust.

Module ID: module-15

HUMAN-IN-THE-LOOP

Motivation

Apprentices learn under a master's guidance. Surgeons have assistants for critical steps. Editors review writers' work before publication. Humans naturally incorporate oversight, feedback, and collaboration into complex processes. The Human-in-the-Loop pattern brings this to agents: integrating human judgment, feedback, and decision-making into agent workflows for safety, quality, and trust, especially in high-stakes situations.

Pattern Overview

What it is: Human-in-the-Loop (HITL) is a pattern that strategically integrates human oversight, judgment, and intervention into AI agent workflows, creating a symbiotic partnership between human intelligence and AI capabilities.

When to use: Use this pattern when deploying AI in domains where errors have significant safety, ethical, or financial consequences, such as healthcare, finance, or autonomous systems. It is essential for tasks involving ambiguity and nuance that LLMs cannot reliably handle.

Why it matters: AI systems, including advanced LLMs, often struggle with tasks that require nuanced judgment, ethical reasoning, or a deep understanding of complex, ambiguous contexts. Deploying fully autonomous AI in high-stakes environments carries significant risks, as errors can lead to severe safety,

financial, or ethical consequences. HITL ensures that AI operates within ethical boundaries, adheres to safety protocols, and achieves objectives with optimal effectiveness.

The Human-in-the-Loop pattern represents a pivotal strategy in the development and deployment of Agents. It deliberately interweaves the unique strengths of human cognition—such as judgment, creativity, and nuanced understanding—with the computational power and efficiency of AI. This strategic integration is not merely an option but often a necessity, especially as AI systems become increasingly embedded in critical decision-making processes.

HITL acknowledges that even with rapidly advancing AI technologies, human oversight, strategic input, and collaborative interactions remain indispensable. The approach fundamentally revolves around the idea of synergy between artificial and human intelligence. Rather than viewing AI as a replacement for human workers, HITL positions AI as a tool that augments and enhances human capabilities. This augmentation can take various forms, from automating routine tasks to providing data-driven insights that inform human decisions.

HITL encompasses several key aspects: Human Oversight (monitoring AI performance and output), Intervention and Correction (humans rectifying errors or guiding agents), Human Feedback for Learning (collecting feedback to refine models), Decision Augmentation (AI provides analysis, humans make final decisions), Human-Agent Collaboration (cooperative interaction leveraging respective strengths), and Escalation Policies (protocols for when agents should escalate to humans).

Key Concepts

- **Human Oversight:** Monitoring AI agent performance and output to ensure adherence to guidelines and prevent undesirable outcomes.
- **Intervention and Correction:** Human operators rectifying errors, supplying missing data, or guiding agents when they encounter problems.
- **Human Feedback for Learning:** Collecting and using human feedback to refine AI models, prominently in reinforcement learning with human feedback.
- **Decision Augmentation:** AI provides analyses and recommendations, humans make final decisions, enhancing decision-making through AI-generated insights.
- **Escalation Policies:** Established protocols dictating when and how agents should escalate tasks to human operators.

How It Works

HITL works through structured interaction patterns. Agents operate autonomously for routine tasks but identify scenarios requiring human review. When such scenarios are detected, agents initiate escalation processes, transferring control or requesting input from human operators. Human operators provide validation, correction, guidance, or make final decisions. This feedback is then incorporated into the agent's context, potentially informing future behavior through learning mechanisms.

The pattern can be implemented in diverse ways: humans acting as validators reviewing AI outputs, humans actively guiding AI behavior in real-time, or humans collaborating with AI as partners through interactive dialog. Regardless of implementation, HITL maintains human control and oversight, ensuring AI systems remain aligned with human ethics, values, goals, and societal expectations.

When to Use This Pattern

Use this pattern when:

- **High-stakes decisions:** Errors have significant safety, ethical, or financial consequences.
- **Ambiguous scenarios:** Tasks involve nuance and ambiguity that LLMs cannot reliably handle.
- **Ethical considerations:** Decisions require ethical reasoning or moral judgment.
- **Quality requirements:** Outputs must meet high quality standards requiring human validation.
- **Learning from feedback:** You want to continuously improve AI models with high-quality human-labeled data.

Avoid this pattern when:

- **High-volume, low-stakes tasks:** The task requires scale that human oversight cannot provide.
- **Real-time constraints:** Human intervention adds unacceptable latency for time-sensitive applications.
- **Simple, deterministic tasks:** The task is straightforward enough that AI can handle it autonomously.
- **Cost constraints:** Human oversight is too expensive for the use case.
- **Privacy concerns:** Sensitive information cannot be exposed to human operators.

Decision Guidelines

Use HITL when the benefits of human judgment and oversight outweigh the costs of reduced scalability and increased latency. Consider: the criticality of decisions (more critical = more need for HITL), the ambiguity of tasks (more ambiguous = more need for HITL), and the availability of human expertise (expertise available = effective HITL). Be aware that HITL has significant caveats: lack of scalability, dependence on skilled operators, and privacy concerns requiring data anonymization. For production systems, implement hybrid approaches combining automation for scale with HITL for accuracy.

Practical Applications & Use Cases

The Human-in-the-Loop pattern is vital across a wide range of industries and applications, particularly where accuracy, safety, ethics, or nuanced understanding are paramount.

- **Content Moderation:** AI filters content rapidly, but ambiguous or borderline cases are escalated to human moderators for nuanced judgment.
- **Autonomous Driving:** Self-driving cars handle most tasks autonomously but hand over control to human drivers in complex or dangerous situations.
- **Financial Fraud Detection:** AI flags suspicious transactions, but high-risk or ambiguous alerts are sent to human analysts for investigation and final determination.
- **Legal Document Review:** AI scans and categorizes documents, but human legal professionals review findings for accuracy, context, and legal implications.
- **Customer Support:** Chatbots handle routine inquiries, but complex or emotionally charged issues are seamlessly handed over to human support agents.
- **Data Labeling:** Humans accurately label images, text, or audio to provide ground truth for AI training.
- **Generative AI Refinement:** Human editors review and refine AI-generated creative content to ensure it meets brand guidelines and quality standards.

Implementation

Prerequisites

```
pip install langchain langchain-openai  
# or  
pip install google-adk
```

Basic Example

```

from langchain_openai import ChatOpenAI
from typing import Dict, Optional
import json

class HITLAgent:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0)
        self.escalation_threshold = 0.7

    def should_escalate(self, task: str, confidence: float, context: Dict) → bool:
        """Determine if task should be escalated to human."""
        # Check confidence threshold
        if confidence < self.escalation_threshold:
            return True

        # Check for sensitive keywords
        sensitive_keywords = ["refund", "legal", "medical", "financial"]
        if any(keyword in task.lower() for keyword in sensitive_keywords):
            return True

        # Check for ambiguity indicators
        if context.get("ambiguity_score", 0) > 0.8:
            return True

        return False

    def process_with_hitl(self, task: str, context: Dict) → Dict:
        """Process task with human-in-the-loop when needed."""
        # AI processes task
        ai_response = self.llm.invoke(f"Process: {task}").content
        confidence = self._assess_confidence(ai_response, task)

        # Check if escalation needed

```

```

        if self.should_escalate(task, confidence, context):
            return {
                "status": "escalated",
                "ai_suggestion": ai_response,
                "requires_human_review": True
            }

        return {
            "status": "completed",
            "response": ai_response,
            "confidence": confidence
        }

def _assess_confidence(self, response: str, task: str) → float:
    """Assess AI response confidence."""
    # Simplified confidence assessment
    prompt = f"""Rate your confidence (0.0-1.0) in this response:
    Task: {task}
    Response: {response}
    Return only the confidence score."""

    result = self.llm.invoke(prompt).content
    try:
        return float(result.strip())
    except:
        return 0.5

# Usage
agent = HITLAgent()
result = agent.process_with_hitl(
    "Process customer refund request",
    {"ambiguity_score": 0.9}
)

```

```
if result["status"] == "escalated":  
    print("Escalated to human for review")  
    # Human reviews and provides final decision
```

Explanation: This example demonstrates basic HITL implementation with escalation logic. The agent processes tasks autonomously but escalates to humans when confidence is low, sensitive topics are involved, or ambiguity is high. This ensures human oversight for critical or uncertain scenarios.

Advanced Example

```

from langchain_openai import ChatOpenAI
from typing import Dict, List, Optional, Callable
import json
from enum import Enum

class EscalationLevel(Enum):
    NONE = "none"
    REVIEW = "review"
    APPROVAL = "approval"
    FULL_CONTROL = "full_control"

class AdvancedHITLSystem:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0)
        self.escalation_policies = {}
        self.human_feedback_log = []

    def set_escalation_policy(self, domain: str, policy: Dict):
        """Set escalation policy for a domain."""
        self.escalation_policies[domain] = policy

    def determine_escalation(self, task: str, domain: str, context: Dict) → EscalationLevel:
        """Determine escalation level based on policies."""
        policy = self.escalation_policies.get(domain, {})

        # Check risk level
        risk_level = context.get("risk_level", "low")
        if risk_level == "high":
            return EscalationLevel.FULL_CONTROL
        elif risk_level == "medium":
            return EscalationLevel.APPROVAL

        # Check confidence

```

```

confidence = context.get("confidence", 1.0)
if confidence < policy.get("confidence_threshold", 0.7):
    return EscalationLevel.REVIEW

# Check for policy violations
if self._check_policy_violations(task, policy):
    return EscalationLevel.APPROVAL

return EscalationLevel.NONE

def _check_policy_violations(self, task: str, policy: Dict) → bool:
    """Check if task violates policies."""
    restricted_keywords = policy.get("restricted_keywords", [])
    return any(keyword in task.lower() for keyword in restricted_keywords)

def process_with_escalation(self, task: str, domain: str, context: Dict) → Dict:
    """Process task with appropriate escalation level."""
    escalation_level = self.determine_escalation(task, domain, context)

    if escalation_level == EscalationLevel.NONE:
        # Autonomous processing
        response = self.llm.invoke(f"Process: {task}").content
        return {"status": "autonomous", "response": response}

    elif escalation_level == EscalationLevel.REVIEW:
        # AI processes, human reviews
        ai_response = self.llm.invoke(f"Process: {task}").content
        return {
            "status": "pending_review",
            "ai_response": ai_response,
            "requires_human_review": True
        }

    elif escalation_level == EscalationLevel.APPROVAL:
        # AI suggests, human approves
        ai_suggestion = self.llm.invoke(f"Suggest approach for: {task}").content

```

```

        return {
            "status": "pending_approval",
            "ai_suggestion": ai_suggestion,
            "requires_human_approval": True
        }

    else: # FULL_CONTROL
        # Human handles entirely
        return {
            "status": "human_control",
            "message": "Task requires human handling",
            "requires_human_action": True
        }

def incorporate_feedback(self, task_id: str, human_feedback: Dict):
    """Incorporate human feedback for learning."""
    feedback_entry = {
        "task_id": task_id,
        "feedback": human_feedback,
        "timestamp": time.time()
    }

    self.human_feedback_log.append(feedback_entry)

    # Use feedback to improve future responses
    # In production, this could update model weights or prompt templates

# Usage
hitl_system = AdvancedHITLSystem()

# Set escalation policy
hitl_system.set_escalation_policy("finance", {
    "confidence_threshold": 0.9,
    "restricted_keywords": ["refund", "chargeback", "dispute"]
})

# Process with escalation

```



```
result = hitl_system.process_with_escalation(  
    "Process refund request for order #12345",  
    "finance",  
    {"risk_level": "high", "confidence": 0.6}  
)  
  
if result["status"] == "pending_approval":  
    # Human reviews and approves  
    human_decision = "approved" # or "rejected"  
    hitl_system.incorporate_feedback("task_123", {  
        "decision": human_decision,  
        "notes": "Approved after verification"  
    })
```

Explanation: This advanced example implements a comprehensive HITL system with multiple escalation levels, domain-specific policies, and feedback incorporation. It demonstrates production-ready HITL with autonomous processing, review workflows, approval processes, and learning from human feedback.

Framework-Specific Examples

GOOGLE ADK WITH ESCALATION

```
from google.adk.agents import Agent
from google.adk.tools.tool_context import ToolContext

def escalate_to_human(issue_type: str) → dict:
    """Escalate issue to human operator."""
    return {
        "status": "escalated",
        "message": f"Escalated {issue_type} to human specialist"
    }

technical_support_agent = Agent(
    name="technical_support_specialist",
    model="gemini-2.0-flash",
    instruction="""You are a technical support specialist.
    For complex issues beyond basic troubleshooting:
    1. Use escalate_to_human to transfer to a human specialist.
    Maintain professional, empathetic tone.""",
    tools=[troubleshoot_issue, create_ticket, escalate_to_human]
)
```

LANGCHAIN WITH HUMAN REVIEW

```
from langchain.agents import AgentExecutor
from langchain.callbacks import HumanApprovalCallbackHandler

# Agent with human approval callback
callback = HumanApprovalCallbackHandler()

executor = AgentExecutor(
    agent=agent,
    tools=tools,
    callbacks=[callback],
    verbose=True
)

# Agent will request human approval for certain actions
result = executor.invoke({"input": "Send email to customer"})
```

Key Takeaways

- **Core Concept:** Human-in-the-Loop integrates human intelligence and judgment into AI workflows, ensuring safety, ethics, and effectiveness in complex scenarios.
- **Best Practice:** Implement clear escalation policies, confidence thresholds, and feedback mechanisms for effective HITL.
- **Common Pitfall:** HITL lacks scalability and depends on skilled operators; use hybrid approaches combining automation with selective human oversight.
- **Performance Note:** HITL adds latency and cost but is essential for high-stakes applications requiring human judgment and oversight.

Related Patterns

This pattern works well with:

- **Exception Handling** - Critical errors can be escalated to human operators
- **Guardrails and Safety** - HITL provides human oversight for safety-critical decisions
- **Learning and Adaptation** - Human feedback is used to improve AI models

This pattern is often combined with:

- **Goal Setting and Monitoring** - Human oversight ensures goals are met appropriately
- **Evaluation and Monitoring** - Human review is part of evaluation processes

References

- A Survey of Human-in-the-loop for Machine Learning: <https://arxiv.org/abs/2109.02840>
- Google ADK Agents: <https://google.github.io/adk-docs/agents/>
- LangChain Human Approval: https://python.langchain.com/docs/modules/callbacks/human_approval/

PART VIII

Knowledge & Communication

Retrieving knowledge and enabling agent communication

Pattern: Knowledge Retrieval (RAG)

Enabling LLMs to access external knowledge bases through Retrieval-Augmented Generation, vector databases, and semantic search.

Module ID: module-16

CHAPTER 14: KNOWLEDGE RETRIEVAL (RAG)

Motivation

When you need information, you don't rely solely on memory. You search the web, consult reference books, ask experts, or look through documentation. You retrieve relevant information and use it to answer questions or make decisions. Knowledge Retrieval (RAG) gives agents this capability: accessing external knowledge bases, finding relevant information, and augmenting their responses with retrieved context, just as humans look things up when needed.

LLMs exhibit substantial capabilities in generating human-like text. However, their knowledge base is typically confined to the data on which they were trained, limiting their access to real-time information, specific company data, or highly specialized details. **Knowledge Retrieval (RAG, or Retrieval Augmented Generation)**, addresses this limitation. RAG enables LLMs to access and integrate external, current, and context-specific information, thereby enhancing the accuracy, relevance, and factual basis of their outputs.

For AI agents, this is crucial as it allows them to ground their actions and responses in real-time, verifiable data beyond their static training. This capability enables them to perform complex tasks accurately, such as accessing the latest company policies to answer a specific question or checking current inventory before placing an order. By integrating external knowledge, RAG transforms agents from simple conversationalists into effective, data-driven tools capable of executing meaningful work.

Knowledge Retrieval (RAG) Pattern Overview

The Knowledge Retrieval (RAG) pattern significantly enhances the capabilities of LLMs by granting them access to external knowledge bases before generating a response. Instead of relying solely on their internal, pre-trained knowledge, RAG allows LLMs to “look up” information, much like a human might consult a book or search the internet. This process empowers LLMs to provide more accurate, up-to-date, and verifiable answers.

When a user poses a question or gives a prompt to an AI system using RAG, the query isn’t sent directly to the LLM. Instead, the system first scours a vast external knowledge base—a highly organized library of documents, databases, or web pages—for relevant information. This search is not a simple keyword match; it’s a **“semantic search”** that understands the user’s intent and the meaning behind their words. This initial search pulls out the most pertinent snippets or “chunks” of information. These extracted pieces are then “augmented,” or added, to the original prompt, creating a richer, more informed query. Finally, this enhanced prompt is sent to the LLM. With this additional context, the LLM can generate a response that is not only fluent and natural but also factually grounded in the retrieved data.

The RAG framework provides several significant benefits. It allows LLMs to access up-to-date information, thereby overcoming the constraints of their static training data. This approach also reduces the risk of **“hallucination”**—the generation of false information—by grounding responses in verifiable data. Moreover, LLMs can utilize specialized knowledge found in internal company documents or wikis. A vital advantage of this process is the capability to offer **“citations,”** which pinpoint the exact source of information, thereby enhancing the trustworthiness and verifiability of the AI’s responses.

To fully appreciate how RAG functions, it’s essential to understand a few core concepts (see Fig.1):

Embeddings

In the context of LLMs, embeddings are numerical representations of text, such as words, phrases, or entire documents. These representations are in the form of a vector, which is a list of numbers. The key idea is to capture the semantic meaning and the relationships between different pieces of text in a mathematical space. Words or phrases with similar meanings will have embeddings that are closer to each other in this vector space. For instance, imagine a simple 2D graph. The word “cat” might be represented by the coordinates (2, 3), while “kitten” would be very close at (2.1, 3.1). In contrast, the word “car” would have a

distant coordinate like (8, 1), reflecting its different meaning. In reality, these embeddings are in a much higher-dimensional space with hundreds or even thousands of dimensions, allowing for a very nuanced understanding of language.

Text Similarity

Text similarity refers to the measure of how alike two pieces of text are. This can be at a surface level, looking at the overlap of words (lexical similarity), or at a deeper, meaning-based level. In the context of RAG, text similarity is crucial for finding the most relevant information in the knowledge base that corresponds to a user's query. For instance, consider the sentences: "What is the capital of France?" and "Which city is the capital of France?". While the wording is different, they are asking the same question. A good text similarity model would recognize this and assign a high similarity score to these two sentences, even though they only share a few words. This is often calculated using the embeddings of the texts.

Semantic Similarity and Distance

Semantic similarity is a more advanced form of text similarity that focuses purely on the meaning and context of the text, rather than just the words used. It aims to understand if two pieces of text convey the same concept or idea. Semantic distance is the inverse of this; a high semantic similarity implies a low semantic distance, and vice versa. In RAG, semantic search relies on finding documents with the smallest semantic distance to the user's query. For instance, the phrases "a furry feline companion" and "a domestic cat" have no words in common besides "a". However, a model that understands semantic similarity would recognize that they refer to the same thing and would consider them to be highly similar. This is because their embeddings would be very close in the vector space, indicating a small semantic distance. This is the "smart search" that allows RAG to find relevant information even when the user's wording doesn't exactly match the text in the knowledge base.



RAG Core Concepts: Chunking, Embeddings, and Vector Database

Fig.1: RAG Core Concepts: Chunking, Embeddings, and Vector Database

Chunking of Documents

Chunking is the process of breaking down large documents into smaller, more manageable pieces, or "chunks." For a RAG system to work efficiently, it cannot feed entire large documents into the LLM. Instead, it processes these smaller chunks. The way documents are chunked is important for preserving the context and meaning of the information. For instance, instead of treating a 50-page user manual as a single block of

text, a chunking strategy might break it down into sections, paragraphs, or even sentences. For instance, a section on “Troubleshooting” would be a separate chunk from the “Installation Guide.” When a user asks a question about a specific problem, the RAG system can then retrieve the most relevant troubleshooting chunk, rather than the entire manual. This makes the retrieval process faster and the information provided to the LLM more focused and relevant to the user’s immediate need. Once documents are chunked, the RAG system must employ a retrieval technique to find the most relevant pieces for a given query.

The primary method is **vector search**, which uses embeddings and semantic distance to find chunks that are conceptually similar to the user’s question. An older, but still valuable, technique is **BM25**, a keyword-based algorithm that ranks chunks based on term frequency without understanding semantic meaning. To get the best of both worlds, **hybrid search** approaches are often used, combining the keyword precision of BM25 with the contextual understanding of semantic search. This fusion allows for more robust and accurate retrieval, capturing both literal matches and conceptual relevance.

Vector Databases

A vector database is a specialized type of database designed to store and query embeddings efficiently. After documents are chunked and converted into embeddings, these high-dimensional vectors are stored in a vector database. Traditional retrieval techniques, like keyword-based search, are excellent at finding documents containing exact words from a query but lack a deep understanding of language. They wouldn’t recognize that “furry feline companion” means “cat.” This is where vector databases excel. They are built specifically for semantic search. By storing text as numerical vectors, they can find results based on conceptual meaning, not just keyword overlap. When a user’s query is also converted into a vector, the database uses highly optimized algorithms (like **HNSW - Hierarchical Navigable Small World**) to rapidly search through millions of vectors and find the ones that are “closest” in meaning. This approach is far superior for RAG because it uncovers relevant context even if the user’s phrasing is completely different from the source documents. In essence, while other techniques search for words, vector databases search for meaning.

This technology is implemented in various forms, from managed databases like **Pinecone** and **Weaviate** to open-source solutions such as **Chroma DB**, **Milvus**, and **Qdrant**. Even existing databases can be augmented with vector search capabilities, as seen with **Redis**, **Elasticsearch**, and **Postgres** (using the pgvector extension). The core retrieval mechanisms are often powered by libraries like Meta AI’s **FAISS** or Google Research’s **ScaNN**, which are fundamental to the efficiency of these systems.

RAG's Challenges

Despite its power, the RAG pattern is not without its challenges. A primary issue arises when the information needed to answer a query is not confined to a single chunk but is spread across multiple parts of a document or even several documents. In such cases, the retriever might fail to gather all the necessary context, leading to an incomplete or inaccurate answer. The system's effectiveness is also highly dependent on the quality of the chunking and retrieval process; if irrelevant chunks are retrieved, it can introduce noise and confuse the LLM. Furthermore, effectively synthesizing information from potentially contradictory sources remains a significant hurdle for these systems.

Besides that, another challenge is that RAG requires the entire knowledge base to be pre-processed and stored in specialized databases, such as vector or graph databases, which is a considerable undertaking. Consequently, this knowledge requires periodic reconciliation to remain up-to-date, a crucial task when dealing with evolving sources like company wikis. This entire process can have a noticeable impact on performance, increasing latency, operational costs, and the number of tokens used in the final prompt.

Summary

In summary, the Retrieval-Augmented Generation (RAG) pattern represents a significant leap forward in making AI more knowledgeable and reliable. By seamlessly integrating an external knowledge retrieval step into the generation process, RAG addresses some of the core limitations of standalone LLMs. The foundational concepts of embeddings and semantic similarity, combined with retrieval techniques like keyword and hybrid search, allow the system to intelligently find relevant information, which is made manageable through strategic chunking. This entire retrieval process is powered by specialized vector databases designed to store and efficiently query millions of embeddings at scale. While challenges in retrieving fragmented or contradictory information persist, RAG empowers LLMs to produce answers that are not only contextually appropriate but also anchored in verifiable facts, fostering greater trust and utility in AI.

Graph RAG

GraphRAG is an advanced form of Retrieval-Augmented Generation that utilizes a knowledge graph instead of a simple vector database for information retrieval. It answers complex queries by navigating the explicit relationships (edges) between data entities (nodes) within this structured knowledge base. A key advantage is its ability to synthesize answers from information fragmented across multiple documents, a common failing of traditional RAG. By understanding these connections, GraphRAG provides more contextually accurate and nuanced responses.

Use cases include complex financial analysis, connecting companies to market events, and scientific research for discovering relationships between genes and diseases. The primary drawback, however, is the significant complexity, cost, and expertise required to build and maintain a high-quality knowledge graph. This setup is also less flexible and can introduce higher latency compared to simpler vector search systems. The system's effectiveness is entirely dependent on the quality and completeness of the underlying graph structure. Consequently, GraphRAG offers superior contextual reasoning for intricate questions but at a much higher implementation and maintenance cost. In summary, it excels where deep, interconnected insights are more critical than the speed and simplicity of standard RAG.

Agentic RAG

An evolution of this pattern, known as **Agentic RAG** (see Fig.2), introduces a reasoning and decision-making layer to significantly enhance the reliability of information extraction. Instead of just retrieving and augmenting, an “agent”—a specialized AI component—acts as a critical gatekeeper and refiner of knowledge. Rather than passively accepting the initially retrieved data, this agent actively interrogates its quality, relevance, and completeness, as illustrated by the following scenarios.



Agentic RAG introduces a reasoning agent that actively evaluates, reconciles, and refines retrieved information to ensure a more accurate and trustworthy final response.

Fig.2: Agentic RAG introduces a reasoning agent that actively evaluates, reconciles, and refines retrieved information to ensure a more accurate and trustworthy final response.

Reflection and Source Validation

First, an agent excels at reflection and source validation. If a user asks, “What is our company’s policy on remote work?” a standard RAG might pull up a 2020 blog post alongside the official 2025 policy document. The agent, however, would analyze the documents’ metadata, recognize the 2025 policy as the most current and authoritative source, and discard the outdated blog post before sending the correct context to the LLM for a precise answer.

Reconciling Knowledge Conflicts

Second, an agent is adept at reconciling knowledge conflicts. Imagine a financial analyst asks, “What was Project Alpha’s Q1 budget?” The system retrieves two documents: an initial proposal stating a €50,000 budget and a finalized financial report listing it as €65,000. An Agentic RAG would identify this contradiction, prioritize the financial report as the more reliable source, and provide the LLM with the verified figure, ensuring the final answer is based on the most accurate data.

Multi-Step Reasoning

Third, an agent can perform multi-step reasoning to synthesize complex answers. If a user asks, “How do our product’s features and pricing compare to Competitor X’s?” the agent would decompose this into separate sub-queries. It would initiate distinct searches for its own product’s features, its pricing, Competitor X’s features, and Competitor X’s pricing. After gathering these individual pieces of information, the agent would synthesize them into a structured, comparative context before feeding it to the LLM, enabling a comprehensive response that a simple retrieval could not have produced.

Identifying Knowledge Gaps and Using External Tools

Fourth, an agent can identify knowledge gaps and use external tools. Suppose a user asks, “What was the market’s immediate reaction to our new product launched yesterday?” The agent searches the internal knowledge base, which is updated weekly, and finds no relevant information. Recognizing this gap, it can then activate a tool—such as a live web-search API—to find recent news articles and social media sentiment. The agent then uses this freshly gathered external information to provide an up-to-the-minute answer, overcoming the limitations of its static internal database.

Challenges of Agentic RAG

While powerful, the agentic layer introduces its own set of challenges. The primary drawback is a significant increase in complexity and cost. Designing, implementing, and maintaining the agent's decision-making logic and tool integrations requires substantial engineering effort and adds to computational expenses. This complexity can also lead to increased latency, as the agent's cycles of reflection, tool use, and multi-step reasoning take more time than a standard, direct retrieval process. Furthermore, the agent itself can become a new source of error; a flawed reasoning process could cause it to get stuck in useless loops, misinterpret a task, or improperly discard relevant information, ultimately degrading the quality of the final response.

In summary: Agentic RAG represents a sophisticated evolution of the standard retrieval pattern, transforming it from a passive data pipeline into an active, problem-solving framework. By embedding a reasoning layer that can evaluate sources, reconcile conflicts, decompose complex questions, and use external tools, agents dramatically improve the reliability and depth of the generated answers. This advancement makes the AI more trustworthy and capable, though it comes with important trade-offs in system complexity, latency, and cost that must be carefully managed.

Practical Applications & Use Cases

Knowledge Retrieval (RAG) is changing how Large Language Models (LLMs) are utilized across various industries, enhancing their ability to provide more accurate and contextually relevant responses.

Applications include:

- **Enterprise Search and Q&A:** Organizations can develop internal chatbots that respond to employee inquiries using internal documentation such as HR policies, technical manuals, and product specifications. The RAG system extracts relevant sections from these documents to inform the LLM's response.
- **Customer Support and Helpdesks:** RAG-based systems can offer precise and consistent responses to customer queries by accessing information from product manuals, frequently asked questions (FAQs), and support tickets. This can reduce the need for direct human intervention for routine issues.
- **Personalized Content Recommendation:** Instead of basic keyword matching, RAG can identify and retrieve content (articles, products) that is semantically related to a user's preferences or previous interactions, leading to more relevant recommendations.

- **News and Current Events Summarization:** LLMs can be integrated with real-time news feeds. When prompted about a current event, the RAG system retrieves recent articles, allowing the LLM to produce an up-to-date summary.

By incorporating external knowledge, RAG extends the capabilities of LLMs beyond simple communication to function as knowledge processing systems.

Pattern: Inter-Agent Communication (A2A)

Enabling agents to communicate, coordinate, and collaborate with each other through standardized protocols and interfaces.

Module ID: module-17

INTER-AGENT COMMUNICATION (A2A)

Motivation

Teams communicate through meetings, shared documents, and protocols. Doctors hand off patients with standardized reports. Businesses coordinate through contracts and agreements. Humans use shared languages, protocols, and interfaces to collaborate effectively. Inter-Agent Communication brings this to agent systems: enabling agents to communicate, coordinate, and collaborate through standardized protocols and interfaces, just as humans do in collaborative work.

Pattern Overview

What it is: Inter-Agent Communication (A2A) is an open, HTTP-based protocol that enables communication and collaboration between different AI agents, regardless of their underlying frameworks, facilitating seamless coordination, task delegation, and information exchange.

When to use: Use this pattern when you need to orchestrate collaboration between two or more AI agents, especially if they are built using different frameworks (e.g., Google ADK, LangGraph, CrewAI). It is ideal for building complex, modular applications where specialized agents handle specific parts of a workflow.

Why it matters: Individual AI agents often face limitations when tackling complex, multifaceted problems, even with advanced capabilities. To overcome this, Inter-Agent Communication enables diverse AI agents, potentially built with different frameworks, to collaborate effectively. This collaboration involves seamless coordination, task delegation, and information exchange.

Individual AI agents, especially those built on different frameworks, often struggle with complex, multifaceted problems on their own. The primary challenge is the lack of a common language or protocol that allows them to communicate and collaborate effectively. This isolation prevents the creation of sophisticated systems where multiple specialized agents can combine their unique skills to solve larger tasks. Without a standardized approach, integrating these disparate agents is costly, time-consuming, and hinders the development of more powerful, cohesive AI solutions.

The Inter-Agent Communication (A2A) protocol provides an open, standardized solution for this problem. It is an HTTP-based protocol that enables interoperability, allowing distinct AI agents to coordinate, delegate tasks, and share information seamlessly, regardless of their underlying technology. A core component is the Agent Card, a digital identity file that describes an agent's capabilities, skills, and communication endpoints, facilitating discovery and interaction. A2A defines various interaction mechanisms, including synchronous and asynchronous communication, to support diverse use cases.

Google's A2A protocol is supported by a range of technology companies and service providers, including Atlassian, Box, LangChain, MongoDB, Salesforce, SAP, and ServiceNow. Microsoft plans to integrate A2A into Azure AI Foundry and Copilot Studio, demonstrating its commitment to open protocols. As an open-source protocol, A2A welcomes community contributions to facilitate its evolution and widespread adoption.

Key Concepts

- **Agent Card:** A digital identity file (usually JSON) that describes an agent's capabilities, skills, endpoint URL, version, and authentication requirements.
- **Agent Discovery:** Mechanisms for clients to find Agent Cards, including Well-Known URI, Curated Registries, or Direct Configuration.
- **Tasks and Messages:** Communication is structured around asynchronous tasks with unique identifiers, moving through states (submitted, working, completed).
- **Interaction Mechanisms:** Multiple methods including Synchronous Request/Response, Asynchronous Polling, Streaming Updates (SSE), and Push Notifications (Webhooks).
- **Security:** Built-in mechanisms including Mutual TLS (mTLS), comprehensive audit logs, and credential handling via OAuth 2.0 or API keys.

How It Works

A2A operates on a client-server architecture with three main actors: User (initiates requests), A2A Client (acts on user's behalf), and A2A Server (provides HTTP endpoint to process requests). The interaction flow involves: (1) Discovery—client queries server to learn available capabilities via Agent Card, (2) Request Formulation—LLM determines it needs a tool/resource and formulates a request, (3) Client Communication—client sends standardized call to server, (4) Server Execution—server authenticates, validates, and executes action, (5) Response—server sends standardized response back, updating LLM context.

Communication is structured around asynchronous tasks, which represent fundamental units of work for long-running processes. Each task has a unique identifier and moves through states. Messages contain attributes (key-value metadata) and parts (actual content like text, files, or JSON). Artifacts are tangible outputs generated by agents during tasks. All communication uses HTTP(S) with JSON-RPC 2.0 protocol for payloads, and a server-generated contextId maintains continuity across multiple interactions.

When to Use This Pattern

✓ Use this pattern when:

- **Multi-framework collaboration:** You need agents built with different frameworks (ADK, LangGraph, CrewAI) to work together.
- **Modular architectures:** You want specialized agents handling specific parts of workflows that can be combined dynamically.
- **Dynamic capability discovery:** Agents need to discover and consume capabilities of other agents without redeployment.
- **Distributed systems:** Agents are deployed across different services, networks, or organizations.
- **Enterprise workflows:** You need to orchestrate complex workflows with agents from different vendors or systems.

✗ Avoid this pattern when:

- **Single framework, single agent:** All agents use the same framework and can communicate through framework-specific mechanisms.
- **Tightly coupled systems:** Agents are closely integrated and don't benefit from standardized communication.

- **Simple, single-step tasks:** The task doesn't require multi-agent collaboration.
- **Performance-critical:** The protocol overhead adds unacceptable latency for real-time applications.

Decision Guidelines

Use A2A when the benefits of interoperability and modularity outweigh the implementation complexity. Consider: the number of different frameworks (more frameworks = more benefit from A2A), the need for dynamic discovery (dynamic = use A2A), and the requirement for distributed deployment (distributed = use A2A). Be aware that A2A requires proper security implementation including authentication, authorization, and audit logging. For production systems, implement comprehensive security measures and monitoring.

Practical Applications & Use Cases

Inter-Agent Communication is indispensable for building sophisticated AI solutions across diverse domains, enabling modularity, scalability, and enhanced intelligence.

- **Multi-Framework Collaboration:** Enable independent AI agents from different frameworks to communicate and collaborate on complex problems.
- **Automated Workflow Orchestration:** Facilitate complex workflows where agents delegate and coordinate tasks across different stages.
- **Dynamic Information Retrieval:** Agents communicate to retrieve and exchange real-time information from specialized data-fetching agents.
- **Enterprise Integration:** Integrate agents from different vendors or systems into unified workflows.
- **Distributed Agent Networks:** Deploy agents across different services or organizations while maintaining communication.

Implementation

Prerequisites

```
pip install google-adk  
# For A2A server implementation  
pip install a2a-python
```

Basic Example

```

from google.adk.agents import LlmAgent
from google.adk.a2a import AgentCard, AgentSkill, AgentCapabilities

# Define agent skills
skill = AgentSkill(
    id='check_availability',
    name='Check Availability',
    description="Checks a user's availability using Google Calendar",
    tags=['calendar'],
    examples=['Am I free from 10am to 11am tomorrow?'],
)

# Create Agent Card
agent_card = AgentCard(
    name='Calendar Agent',
    description="An agent that can manage a user's calendar",
    url='http://localhost:8000/',
    version='1.0.0',
    defaultInputModes=['text'],
    defaultOutputModes=['text'],
    capabilities=AgentCapabilities(streaming=True),
    skills=[skill],
)

# Create ADK agent
calendar_agent = LlmAgent(
    name='calendar_agent',
    model='gemini-2.0-flash',
    description="An agent that can help manage a user's calendar",
    instruction="Help users manage their calendar using provided tools.",
    tools=[calendar_tools]
)

```

Explanation: This example demonstrates creating an A2A-compliant agent with an Agent Card defining its capabilities and skills. The Agent Card serves as the digital identity that other agents can discover and use to communicate with this agent.

Advanced Example

```

from a2a import A2AServer, AgentCard, AgentSkill
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
import asyncio

class A2AAgentServer:
    def __init__(self, agent: LlmAgent, agent_card: AgentCard):
        self.agent = agent
        self.agent_card = agent_card
        self.runner = Runner(
            app_name=agent_card.name,
            agent=agent,
            session_service=InMemorySessionService(),
            memory_service=InMemoryMemoryService()
        )

    async def handle_task(self, task_id: str, message: dict, context_id: str) → dict:
        """Handle incoming A2A task."""
        # Process message with agent
        response = await self.runner.run_async(
            user_id=context_id,
            session_id=task_id,
            new_message=message
        )

        # Return A2A-compliant response
        return {
            "task_id": task_id,
            "status": "completed",
            "artifacts": [{
                "parts": [{"type": "text", "text": response}]
            }]
        }

```

```

    async def start_server(self, host: str, port: int):
        """Start A2A server."""
        # In production, use proper A2A server implementation
        print(f"A2A server started at http://{host}:{port}")
        # Server would handle HTTP requests and route to handle_task

# Create agent and card
agent = LlmAgent(
    name="research_agent",
    model="gemini-2.0-flash",
    instruction="Research topics and provide summaries"
)

card = AgentCard(
    name="Research Agent",
    description="Researches topics and provides summaries",
    url="http://localhost:8001/",
    version="1.0.0",
    skills=[AgentSkill(
        id="research_topic",
        name="Research Topic",
        description="Research a topic and provide summary"
    )]
)

# Start server
server = A2AAgentServer(agent, card)
asyncio.run(server.start_server("localhost", 8001))

```

Explanation: This advanced example implements an A2A server that can receive and process tasks from other agents. It demonstrates the server-side implementation with task handling, agent execution, and A2A-compliant response formatting.

Framework-Specific Examples

A2A CLIENT REQUEST

```
import requests
import json

# Synchronous request
def send_a2a_task(server_url: str, task: dict) → dict:
    """Send task to A2A server."""
    response = requests.post(
        f"{server_url}/tasks/send",
        json={
            "jsonrpc": "2.0",
            "id": "1",
            "method": "sendTask",
            "params": {
                "id": task["id"],
                "sessionId": task["session_id"],
                "message": {
                    "role": "user",
                    "parts": [{"type": "text", "text": task["query"]}]}
            }
        }
    )
    return response.json()

# Streaming request
def send_a2a_task_stream(server_url: str, task: dict):
    """Send streaming task to A2A server."""
    response = requests.post(
        f"{server_url}/tasks/sendSubscribe",
        json={
            "jsonrpc": "2.0",
            "id": "2",
```



```

        "method": "sendTaskSubscribe",
        "params": {
            "id": task["id"],
            "sessionId": task["session_id"],
            "message": {
                "role": "user",
                "parts": [{"type": "text", "text": task["query"]}]}
        }
    },
    stream=True
)
# Handle streaming response
for line in response.iter_lines():
    if line:
        yield json.loads(line)

```

AGENT DISCOVERY

```

import requests

def discover_agent(agent_url: str) → dict:
    """Discover agent capabilities via Agent Card."""
    # Well-Known URI approach
    response = requests.get(f"{agent_url}/.well-known/agent.json")
    return response.json()

# Usage
agent_card = discover_agent("http://calendar-agent.example.com")
print(f"Agent: {agent_card['name']}")
print(f"Skills: {[skill['name'] for skill in agent_card['skills']]}")

```

Key Takeaways

- **Core Concept:** A2A is an open, HTTP-based protocol enabling communication between AI agents built with different frameworks.
- **Best Practice:** Use Agent Cards for discovery, implement proper security (mTLS, authentication), and choose appropriate interaction mechanisms (synchronous, streaming, polling).
- **Common Pitfall:** Failing to implement proper security can expose agents to unauthorized access; always implement authentication and authorization.
- **Performance Note:** A2A adds protocol overhead but enables interoperability and modularity essential for complex multi-agent systems.

Related Patterns

This pattern works well with:

- **Multi-Agent** - A2A enables communication between agents in multi-agent systems
- **Model Context Protocol (MCP)** - A2A complements MCP by facilitating agent-to-agent communication
- **Routing** - A2A can route tasks to appropriate agents based on capabilities

This pattern is often combined with:

- **Tool Use** - Agents can expose tools via A2A for other agents to use
- **Planning** - A2A enables agents to coordinate plans and delegate tasks

References

- Google A2A Protocol: <https://a2a-protocol.org/>
- A2A GitHub Repository: <https://github.com/google-a2a/A2A>
- A2A Samples: <https://github.com/google-a2a/a2a-samples>
- Google ADK A2A: <https://google.github.io/adk-docs/>

PART IX

Optimization & Safety

Optimizing performance and ensuring safe operation

Resource-Aware Optimization

Optimizing agent behavior considering computational, temporal, and financial resource constraints.

Module ID: module-18

RESOURCE-AWARE OPTIMIZATION

Introduction

LLM-based applications can be expensive and slow. Selecting the best model or tool for every task is often inefficient, creating a fundamental trade-off between output quality and the resources required to produce it. Resource-aware optimization enables agents to dynamically manage computational, temporal, and financial resources, making intelligent decisions about resource allocation to achieve goals within specified budgets.

This chapter provides an overview of resource-aware optimization approaches for agentic systems. We'll explore different optimization strategies, cost-performance trade-offs, and when these techniques are most valuable. For specific implementation patterns, see the pattern modules referenced throughout this chapter.

The Resource Optimization Challenge

Agentic systems consume various resources:

- **Financial Resources:** API costs for LLM calls, tool usage, and external services

- **Computational Resources:** Processing power, memory, and storage
- **Temporal Resources:** Latency and response time
- **Energy Resources:** Battery life for edge devices, computational energy

Without dynamic resource management, systems cannot adapt to varying task complexities or operate within budgetary and performance constraints. Every decision—which model to use, how many reasoning steps to take, which tools to call—has resource implications.

Key Optimization Strategies

Dynamic Model Switching

Strategic selection of LLMs based on task complexity and available resources. Use lightweight, fast models for simple tasks and powerful, slower models for complex reasoning.

Example: A customer service agent might use a fast, inexpensive model for simple FAQ queries, but switch to a more capable model for complex technical support issues.

Router-Based Optimization

A router agent classifies incoming requests and routes them to appropriate models or tools based on complexity, budget, and time constraints. This enables automatic optimization without manual intervention.

Characteristics:

- Analyzes query complexity and requirements
- Considers available budget and time constraints
- Selects optimal model or tool for each request
- Can learn and improve routing decisions over time

Cost-Sensitive Decision Making

Agents make decisions considering financial costs, computational costs, and latency requirements. This involves:

- **Cost-Benefit Analysis:** Evaluating whether additional resources justify improved quality

- **Budget Management:** Tracking and managing resource consumption against budgets
- **Priority-Based Allocation:** Allocating more resources to high-priority tasks

Graceful Degradation

Systems automatically fall back to alternative strategies when resource constraints are severe, maintaining operation at reduced capacity rather than failing completely.

Example: If a preferred model is unavailable or overloaded, the system automatically switches to a backup model, ensuring service continuity.

When Resource Optimization Is Valuable

Resource-aware optimization is most valuable when:

- **Budget Constraints:** Strict financial limits for API calls or computational resources
- **Latency-Sensitive Applications:** Quick response times are critical for user experience
- **Resource-Constrained Hardware:** Agents deployed on edge devices with limited capabilities
- **Quality-Cost Trade-offs:** Need to balance response quality with operational costs
- **Variable Task Complexity:** Different tasks have varying resource requirements

It may be less critical when:

- **Unlimited Resources:** Budget and computational resources are not constraints
- **Fixed Quality Requirements:** All tasks require the same high-quality model
- **Simple, Uniform Tasks:** All tasks have similar complexity and resource requirements
- **Deterministic Workflows:** Resource allocation can be predetermined

Optimization Techniques

Adaptive Tool Selection

Choosing efficient tools based on task requirements and resource constraints. For example, using a lightweight search API for simple queries and a more comprehensive search for complex research.

Contextual Pruning and Summarization

Managing token counts through context compression (see **Context Compression: Managing the Finite Window**). Reducing context size directly reduces costs and latency.

Proactive Resource Prediction

Anticipating resource demands and pre-allocating resources accordingly. This enables more efficient resource utilization and better performance.

Parallelization and Distributed Computing

Using parallel execution and distributed systems to improve throughput and efficiency, though this must be balanced against increased complexity and coordination costs.

Learned Resource Allocation

Using machine learning to optimize resource allocation policies based on historical performance data. This enables adaptive optimization that improves over time.

Cost-Performance Trade-offs

Resource optimization requires balancing multiple competing objectives:

Quality vs. Cost: More expensive models often produce better results, but may not be necessary for simple tasks.

Speed vs. Quality: Faster models may sacrifice some quality, but provide better user experience for time-sensitive applications.

Completeness vs. Efficiency: Comprehensive analysis may be more accurate but slower and more expensive than targeted approaches.

Reliability vs. Cost: Redundant systems and fallbacks improve reliability but increase costs.

Integration with Other Capabilities

Resource-aware optimization integrates with other agent capabilities:

- **Pattern: Routing** - Routing decisions can optimize resource allocation
- **Context Compression** - Reducing context size directly optimizes resource usage
- **Pattern: Parallelization** - Parallel execution can improve resource efficiency
- **Evaluation and Monitoring** - Monitoring resource usage guides optimization decisions
- **Pattern: Prioritization** - Prioritizing tasks enables efficient resource allocation

Key Insights

1. **Optimization is not optional for production:** Real-world systems must operate within resource constraints. Optimization is essential for viability.
2. **Dynamic allocation is key:** Static resource allocation cannot adapt to varying task complexity. Dynamic systems optimize continuously.
3. **Trade-offs are inevitable:** Every optimization decision involves trade-offs. Understand your priorities (cost, quality, speed) and optimize accordingly.
4. **Monitoring enables optimization:** Without visibility into resource usage, optimization is impossible. Implement comprehensive monitoring.
5. **Graceful degradation maintains service:** When resources are constrained, graceful degradation is better than complete failure.

Next Steps

This chapter provided an overview of resource-aware optimization concepts. For detailed implementation guidance, see:

- **Pattern: Routing** - How routing can optimize resource allocation
- **Context Compression: Managing the Finite Window** - Techniques for reducing token usage
- **Pattern: Parallelization** - How parallel execution can improve efficiency
- **Evaluation and Monitoring** - How to monitor and optimize resource usage

Resource-aware optimization is essential for building viable, production-ready agentic systems. Understanding these concepts enables you to build agents that operate efficiently within real-world constraints.

Guardrails/Safety Patterns

Implementing safety mechanisms, content filters, and compliance checks to ensure agents operate within defined boundaries.

Module ID: module-20

GUARDRAILS AND SAFETY PATTERNS

Introduction

As agentic systems become more autonomous and integrated into critical applications, ensuring they operate safely, ethically, and as intended becomes paramount. Guardrails—also referred to as safety patterns—are crucial mechanisms that guide agent behavior and prevent harmful, biased, or undesirable outputs.

This chapter provides an overview of guardrails and safety mechanisms for agentic systems. We'll explore different types of guardrails, implementation approaches, and when they are most critical. For specific implementation patterns, see the pattern modules referenced throughout this chapter.

Why Guardrails Matter

Without guardrails, agentic systems may be unconstrained, unpredictable, and potentially hazardous. As agents gain more autonomy and capability, the risks increase:

- **Harmful Outputs:** Agents may generate toxic, biased, or inappropriate content
- **Safety Violations:** Agents may take actions that violate safety protocols or regulations

- **Ethical Concerns:** Agents may behave in ways that violate ethical guidelines
- **Legal Compliance:** Agents may fail to meet regulatory requirements
- **Reputational Risk:** Poor agent behavior damages trust and reputation

The primary aim of guardrails is not to restrict an agent's capabilities but to ensure its operation is robust, trustworthy, and beneficial. They function as both a safety measure and a guiding influence, vital for constructing responsible AI systems.

Types of Guardrails

Input Validation and Sanitization

Filtering and cleaning incoming data before agent processing to detect inappropriate prompts and ensure structured inputs adhere to predefined rules. This first line of defense prevents malicious or problematic inputs from reaching the agent.

Techniques:

- Content moderation APIs to detect toxic or inappropriate input
- Schema validation to ensure inputs match expected formats
- Sanitization to remove or neutralize potentially harmful content
- Rate limiting to prevent abuse

Output Filtering and Post-Processing

Analyzing generated responses for toxicity, bias, or policy violations, flagging and redacting problematic content before it reaches users.

Techniques:

- LLM-based content analysis to detect violations
- Specialized models for toxicity or bias detection
- Policy compliance checking
- Automatic redaction or blocking of problematic content

Behavioral Constraints

Using prompt-level instructions to guide agent behavior and reduce unintended outputs. System prompts and instructions set boundaries and guide agent decision-making.

Techniques:

- Clear behavioral guidelines in system prompts
- Explicit constraints on agent capabilities
- Ethical guidelines and principles
- Role definitions that limit agent scope

Tool Use Restrictions

Limiting agent capabilities by restricting access to certain tools or functions. This prevents agents from taking actions they shouldn't.

Techniques:

- Tool allowlists and blocklists
- Permission-based access control
- Capability masking (see **Pattern: Constrained Tool Use**)
- Runtime tool availability management

External Moderation

Using specialized APIs or services for content moderation and safety checks. External services provide additional layers of protection and specialized expertise.

Techniques:

- Content moderation APIs (e.g., Perspective API)
- Safety classification services
- Compliance checking services
- Human review workflows

Human Oversight

Integrating human-in-the-loop processes for validation and intervention when guardrails detect issues. Humans provide judgment for complex or high-stakes decisions.

Techniques:

- Human approval for critical actions
- Escalation workflows for detected issues
- Human review of flagged content
- Manual override capabilities

Multi-Layer Protection

Effective guardrails operate through multiple layers of protection:

1. **Input Layer:** Validate and sanitize inputs before processing
2. **Processing Layer:** Guide agent behavior through prompts and constraints
3. **Output Layer:** Filter and validate outputs before delivery
4. **External Layer:** Additional checks through specialized services
5. **Human Layer:** Human oversight for critical decisions

These layers work together to create comprehensive protection while maintaining agent functionality.

When Guardrails Are Critical

Guardrails are essential when:

- **User-Facing Applications:** Agents interact directly with users
- **Sensitive Domains:** Healthcare, finance, legal, or education where errors have serious consequences
- **Content Generation:** Systems generating content that must adhere to guidelines
- **Public Deployment:** Public-facing systems where reputation and trust are critical
- **Regulatory Compliance:** Applications subject to regulations requiring safety measures

Guardrails may be less critical when:

- **Internal, Controlled Environments:** Highly controlled environments with trusted users only
- **Research/Prototyping:** Early research phases where guardrails add unnecessary complexity
- **Performance-Critical Systems:** Systems where guardrail overhead is prohibitive (rare)
- **Over-Constrained Systems:** Systems where guardrails would prevent legitimate functionality

Key Design Principles

Defense in Depth

Implement multiple layers of guardrails rather than relying on a single mechanism. If one layer fails, others provide backup protection.

Fail-Safe Defaults

Design systems to fail safely—when in doubt, err on the side of caution. Block questionable content rather than allowing potentially harmful outputs.

Continuous Monitoring

Guardrails require ongoing monitoring, evaluation, and refinement to adapt to new threats and maintain effectiveness. Regular evaluation ensures guardrails remain effective as threats evolve.

Balance Safety with Functionality

Guardrails should prevent harm without unnecessarily constraining legitimate functionality. Overly restrictive guardrails can make agents unusable.

Transparency

Users and developers should understand what guardrails are in place and how they work. Transparency builds trust and enables debugging.

Integration with Other Capabilities

Guardrails integrate with other agent capabilities:

- **Pattern: Human-in-the-Loop** - Human oversight provides a critical guardrail layer
- **Pattern: Exception Handling** - Guardrails can trigger exception handling when violations are detected
- **Evaluation and Monitoring** - Monitoring detects when guardrails are triggered and evaluates their effectiveness
- **Pattern: Constrained Tool Use** - Tool restrictions are a form of guardrail
- **Pattern: Reflection** - Agents can use reflection to self-check for policy violations

Key Insights

1. **Guardrails are not optional for production systems:** Any agent interacting with users or operating in sensitive domains requires guardrails.
2. **Multiple layers are essential:** Relying on a single guardrail mechanism is insufficient. Implement defense in depth.
3. **Guardrails require maintenance:** Threats evolve, and guardrails must adapt. Regular evaluation and refinement are critical.
4. **Balance is key:** Overly restrictive guardrails can prevent legitimate functionality. Find the right balance for your use case.
5. **Human oversight is valuable:** For complex or high-stakes decisions, human judgment provides essential guardrail protection.

Next Steps

This chapter provided an overview of guardrails and safety concepts. For detailed implementation guidance, see:

- **Pattern: Human-in-the-Loop** - Integrating human oversight into agent workflows
- **Pattern: Constrained Tool Use** - Restricting tool access as a safety mechanism
- **Pattern: Exception Handling and Recovery** - Handling safety violations and errors
- **Evaluation and Monitoring** - Monitoring guardrail effectiveness

Guardrails are essential for building responsible, trustworthy agentic systems. Understanding these concepts and implementing appropriate guardrails is critical for safe deployment in production environments.

Evaluation and Monitoring

Systematic assessment of agent performance, monitoring progress, and detecting operational anomalies in production environments.

Module ID: module-21

EVALUATION AND MONITORING

Introduction

Agentic systems operate in complex, dynamic environments where performance can degrade over time. Their probabilistic and non-deterministic nature means that traditional software testing is insufficient for ensuring reliability. Continuous evaluation and monitoring are essential for measuring an agent's effectiveness, detecting issues, and driving improvements.

This chapter provides an overview of evaluation and monitoring approaches for agentic systems. We'll explore different evaluation methods, key metrics, and monitoring strategies. For specific implementation patterns, see the pattern modules referenced throughout this chapter.

Why Evaluation and Monitoring Matter

Unlike deterministic software, agentic systems face unique challenges:

- **Non-Determinism:** The same input can produce different outputs, making traditional unit testing inadequate
- **Complex Behaviors:** Agent actions involve multi-step reasoning, tool usage, and dynamic decision-making
- **Performance Drift:** Agent performance can degrade over time due to data drift, environmental changes, or model updates

- **Subjective Quality:** Many agent outputs require subjective evaluation (helpfulness, relevance, tone) that automated metrics miss

Evaluation and monitoring address these challenges by providing systematic ways to assess agent performance, detect anomalies, and ensure ongoing reliability.

Types of Evaluation

Objective Metrics

Objective metrics provide quantifiable measures of agent performance:

Accuracy: The correctness of agent outputs, measured against ground truth or expected results

Latency: Response time from input to final output, critical for user-facing applications

Token Usage: The number of tokens consumed, directly impacting cost

Error Rate: Frequency of failures, exceptions, or invalid outputs

Success Rate: Percentage of tasks completed successfully

These metrics are straightforward to measure and provide clear performance indicators, but they may miss nuanced aspects of agent behavior.

Subjective Evaluation

Many agent outputs require subjective evaluation that automated metrics cannot capture:

Helpfulness: Does the response actually help the user?

Relevance: Is the response relevant to the query?

Tone and Style: Is the communication appropriate and well-crafted?

Completeness: Does the response fully address the question?

User Satisfaction: Overall user experience and satisfaction

LLM-as-a-Judge

A powerful approach for subjective evaluation is using LLMs themselves as evaluators. LLM-as-a-Judge leverages the advanced linguistic capabilities of LLMs to provide nuanced, human-like assessments.

Advantages:

- **Consistency:** More consistent than human evaluators
- **Scalability:** Can evaluate large volumes of outputs
- **Nuance:** Captures subtle aspects that automated metrics miss
- **Efficiency:** Faster and cheaper than human evaluation

Limitations:

- May have biases or limitations similar to the agent being evaluated
- Requires careful prompt engineering for reliable evaluation
- May not perfectly match human judgment

Example: An LLM judge can evaluate agent responses on criteria like accuracy, helpfulness, and clarity, providing structured scores and feedback that guide improvements.

Trajectory Analysis

Trajectory analysis evaluates not just the final output, but the sequence of steps taken to reach a solution. This is crucial for understanding agent decision-making quality.

Metrics:

- **Exact Match:** Does the agent's action sequence exactly match the expected sequence?
- **In-Order Match:** Are the correct actions taken in the right order (allowing extra steps)?
- **Precision/Recall:** What percentage of actions were correct? What percentage of required actions were taken?

Trajectory analysis reveals whether agents are making good decisions throughout the process, not just producing correct final outputs.

Key Monitoring Concepts

Performance Metrics

Effective monitoring requires clear metrics tailored to the agent's domain:

- **Accuracy:** Correctness of outputs
- **Latency:** Response time
- **Resource Consumption:** Token usage, API calls, computational resources
- **User Satisfaction:** Feedback scores, engagement metrics

Drift Detection

Agent performance can degrade over time due to:

- **Concept Drift:** Changes in input data distribution
- **Environmental Shifts:** Changes in the operating environment
- **Model Updates:** Changes to underlying models
- **Tool Changes:** Updates to external tools or APIs

Drift detection monitors performance trends and alerts when degradation occurs, enabling proactive intervention.

Anomaly Detection

Anomaly detection identifies unusual or unexpected agent behavior that might indicate:

- **Errors:** Systematic failures or bugs
- **Security Issues:** Malicious attacks or unauthorized access
- **Emergent Behavior:** Unintended agent behaviors
- **Tool Failures:** Issues with external dependencies

Compliance and Safety Audits

For regulated or high-stakes domains, automated audit reports track:

- **Ethical Compliance:** Adherence to ethical guidelines

- **Regulatory Compliance:** Meeting legal and regulatory requirements
- **Safety Protocols:** Following safety procedures and constraints

These audits provide documentation and enable verification of agent behavior over time.

Evaluation Approaches

Continuous Monitoring

Real-time monitoring tracks agent performance as it operates, providing immediate visibility into:

- Current performance metrics
- Error rates and types
- Resource consumption
- User interactions

This enables rapid detection and response to issues.

A/B Testing

A/B testing systematically compares different agent versions or strategies to identify optimal approaches:

- **Version Comparison:** Compare different agent implementations
- **Strategy Testing:** Test different reasoning or planning approaches
- **Model Comparison:** Evaluate different underlying models
- **Prompt Testing:** Compare different prompt strategies

A/B testing provides data-driven insights for improving agent performance.

Benchmark Evaluation

Benchmark evaluation tests agents against standardized test suites:

- **Task-Specific Benchmarks:** Domain-specific evaluation datasets
- **General Capability Tests:** Broad capability assessments
- **Safety Benchmarks:** Tests for safety and compliance

Benchmarks provide objective comparisons and track progress over time.

Implementation Considerations

Evaluation Infrastructure

Effective evaluation requires infrastructure for:

- **Data Collection:** Logging interactions, metrics, and outcomes
- **Storage:** Time-series databases, log files, or observability platforms
- **Analysis:** Tools for processing and analyzing evaluation data
- **Reporting:** Dashboards and reports for stakeholders

Evaluation Frequency

The frequency of evaluation depends on:

- **Criticality:** More critical systems require more frequent evaluation
- **Change Rate:** Systems that change frequently need more evaluation
- **Resource Constraints:** Balance evaluation thoroughness with available resources

Feedback Loops

Evaluation should create feedback loops that drive improvement:

- **Performance Monitoring** → **Issue Detection** → **Investigation** → **Fix** → **Verification**
- **A/B Testing** → **Results Analysis** → **Strategy Selection** → **Deployment**
- **User Feedback** → **Analysis** → **Agent Improvement** → **Re-evaluation**

Integration with Other Capabilities

Evaluation and monitoring integrate with other agent capabilities:

- **Goal Setting and Monitoring:** Evaluation metrics are often tied to goal achievement
- **Reflection:** Agents can use evaluation results to improve their own performance

- **Human-in-the-Loop:** Human evaluators provide ground truth for training evaluation systems
- **Learning and Adaptation:** Evaluation results drive learning and adaptation processes
- **Exception Handling:** Monitoring helps detect exceptions and trigger recovery mechanisms

Key Insights

1. **Evaluation is not optional:** Agentic systems require continuous evaluation to ensure reliability and performance. Traditional testing is insufficient.
2. **Multiple evaluation methods are needed:** Combine objective metrics, subjective evaluation (LLM-as-a-Judge), and trajectory analysis for comprehensive assessment.
3. **Trajectory analysis is critical:** Evaluating only final outputs misses important insights about decision-making quality. Always include trajectory analysis.
4. **Monitoring enables proactive intervention:** Continuous monitoring detects issues early, enabling rapid response before problems escalate.
5. **Evaluation drives improvement:** Effective evaluation creates feedback loops that systematically improve agent performance over time.

Next Steps

This chapter provided an overview of evaluation and monitoring concepts. For detailed implementation guidance, see:

- **Pattern: Goal Setting and Monitoring** - How to set goals and track progress
- **Pattern: Reflection** - How agents can use evaluation to improve themselves
- **Pattern: Human-in-the-Loop** - Integrating human evaluators into evaluation processes
- **Pattern: Exception Handling and Recovery** - How monitoring detects and triggers error recovery

Effective evaluation and monitoring are essential for building reliable, production-ready agentic systems. Understanding these concepts enables you to build systems that maintain performance, detect issues, and continuously improve.

Exploration and Discovery

Enabling agents to actively seek out novel information, uncover new possibilities, and identify unknown unknowns.

Module ID: module-23

EXPLORATION AND DISCOVERY

Introduction

Most agentic systems optimize within known solution spaces or follow predetermined paths. But some problems require agents to venture into the unknown—to actively seek out novel information, uncover new possibilities, and identify “unknown unknowns.” Exploration and Discovery enables agents to move beyond simple optimization to proactive, agentic exploration that expands the system’s own understanding and capabilities.

This chapter provides an overview of exploration and discovery approaches for agentic systems. We’ll explore how agents can proactively explore problem spaces, generate hypotheses, and discover novel solutions. For specific implementation patterns, see the pattern modules referenced throughout this chapter.

The Need for Exploration

AI agents often operate within predefined knowledge, limiting their ability to tackle novel situations or open-ended problems. In complex and dynamic environments, static, pre-programmed information is insufficient for true innovation or discovery.

Exploration vs. Optimization:

- **Optimization:** Finding the best solution within a known solution space

- **Exploration:** Actively seeking out new information and possibilities beyond known boundaries

Exploration is crucial for:

- **Scientific Research:** Discovering new materials, drug candidates, or scientific principles
- **Creative Tasks:** Generating novel content, strategies, or solutions
- **Market Research:** Identifying trends, opportunities, or insights in evolving domains
- **Security Research:** Discovering vulnerabilities or attack vectors
- **Open-Ended Problems:** Problems where the solution space is not fully defined

Exploration Approaches

Proactive Exploration

Agents actively seek out new information rather than waiting for explicit instructions or reacting to known problems. This involves:

- **Broad Search:** Exploring multiple directions simultaneously
- **Novel Path Discovery:** Venturing into unfamiliar territories
- **Information Gathering:** Actively collecting data from diverse sources
- **Hypothesis Generation:** Formulating testable hypotheses about the problem space

Hypothesis-Driven Exploration

Agents formulate testable hypotheses and design experiments to validate or refute them. This mirrors the scientific method:

1. **Hypothesis Generation:** Create testable hypotheses about the problem
2. **Experimental Design:** Design experiments to test hypotheses
3. **Execution and Analysis:** Run experiments and analyze results
4. **Refinement:** Refine hypotheses based on results
5. **Iteration:** Repeat the cycle to deepen understanding

Multi-Agent Exploration

Specialized agents work together, each with specific roles (generation, reflection, ranking, evolution) to emulate effective exploration processes:

- **Generation Agent:** Produces initial hypotheses, ideas, or strategies
- **Reflection/Critique Agent:** Evaluates hypotheses for correctness, novelty, quality, and feasibility
- **Ranking Agent:** Compares and ranks hypotheses using scoring systems
- **Evolution Agent:** Refines top-ranked hypotheses through iterative improvement
- **Clustering Agent:** Identifies relationships between ideas to explore systematically

This multi-agent approach is detailed in the **Multi-Agent Architectures** chapter.

The Exploration Process

Exploration and Discovery operates through structured, iterative processes:

1. **Generation:** Explore the problem space broadly, generating diverse possibilities
2. **Evaluation:** Assess generated ideas for quality, novelty, and feasibility
3. **Ranking:** Compare and prioritize the most promising directions
4. **Refinement:** Iteratively improve top-ranked ideas
5. **Synthesis:** Combine insights from multiple explorations to generate novel understanding

This “generate, debate, and evolve” approach creates a self-improving cycle where hypotheses undergo systematic assessment and refinement.

When Exploration Is Valuable

Exploration and Discovery is most valuable when:

- **Open-Ended Problems:** The solution space is not fully defined or known in advance
- **Novel Discovery Needed:** The objective is to uncover “unknown unknowns” rather than optimize known processes
- **Scientific Research:** Tasks involve hypothesis generation, experimental design, and knowledge discovery

- **Creative Tasks:** Generating novel content, strategies, or solutions
- **Market Research:** Identifying trends, opportunities, or insights in complex, evolving domains

It is **not** ideal when:

- **Well-Defined Problems:** The solution space is known and optimization is sufficient
- **Deterministic Tasks:** Tasks with clear, predetermined solution paths
- **Time-Critical Operations:** When exploration overhead is prohibitive
- **Resource Constraints:** When computational costs of exploration exceed benefits

Exploration Strategies

Exhaustive Exploration

Thoroughly exploring the entire problem space. This is comprehensive but expensive and may be impractical for large spaces.

Targeted Exploration

Focusing exploration on promising areas based on heuristics or prior knowledge. This is more efficient but may miss opportunities in unexplored regions.

Multi-Agent Exploration

Using specialized agents to explore different aspects simultaneously. This balances thoroughness with efficiency through parallel exploration and specialized expertise.

Iterative Refinement

Starting with broad exploration and progressively narrowing focus based on discoveries. This enables efficient exploration of large problem spaces.

Integration with Other Capabilities

Exploration and Discovery integrates with other agent capabilities:

- **Multi-Agent Architectures** - Multi-agent systems enable specialized exploration roles
- **Pattern: Reflection** - Reflection enables evaluation and refinement of discovered ideas
- **Pattern: Prioritization** - Prioritization helps focus exploration on promising directions
- **Pattern: Planning** - Planning helps structure exploration processes
- **Reasoning Techniques** - Reasoning enables hypothesis generation and evaluation

Key Insights

1. **Exploration is expensive:** Systematic exploration requires significant computational resources. Use it when the value of discovery justifies the cost.
2. **Multi-agent systems excel at exploration:** Specialized agents working together can explore more effectively than single agents.
3. **Hypothesis-driven exploration is powerful:** Formulating and testing hypotheses provides structure to exploration processes.
4. **Balance exploration with exploitation:** Too much exploration wastes resources; too little misses opportunities. Find the right balance.
5. **Evaluation is critical:** Without evaluation, exploration is aimless. Systematic evaluation guides exploration toward valuable discoveries.

Next Steps

This chapter provided an overview of exploration and discovery concepts. For detailed implementation guidance, see:

- **Multi-Agent Architectures** - How multi-agent systems enable effective exploration
- **Pattern: Reflection** - How to evaluate and refine discovered ideas
- **Pattern: Prioritization** - How to focus exploration on promising directions
- **Reasoning Techniques** - How reasoning enables hypothesis generation

Exploration and Discovery enables agents to tackle open-ended problems and discover novel solutions. Understanding these concepts enables you to build agents that can venture beyond known solution spaces to discover new possibilities.